

收缩圆柱药材烘干的变物性热质传递建模

摘 要

圆柱形中药材热风烘干同时受时变空气边界、温湿相关物性和尺寸收缩影响，表面状态难以代表内部温度-含水率场。围绕“预热场重构-全程变物性演化-固定域终点-收缩域终点”四层任务，本文以 Fourier 导热和 Fick 扩散为基础，构建一维轴对称等效导热-扩散模型。问题一按题给参数分别求解温度场与非线性含水率场，问题二至问题四再通过状态相关物性形成顺序耦合；干燥终点统一取给定时间网格上全域最大干基含水率严格低于 0.15 kg/kg 的首个时间层。

针对问题一，采用分段线性插值重构时变空气边界，建立径向有限体积模型，对温度方程作全隐式离散，对非线性含水率方程作 Picard 迭代。1800 s 时，中心与表面温度分别为 33.5756°C 和 36.7857°C ，干基含水率分别为 2.5500 kg/kg 和 1.5103 kg/kg ；表面升温 and 失水均先于轴线，中心-表面含水率差为 1.0397 kg/kg 。

针对问题二，构建变物性顺序耦合模型，在每个时间层交替更新温度、含水率和物性，首步采用后向 Euler 格式，后续采用二阶向后差分格式。3 h 时，中心与表面温度分别为 49.8495°C 和 49.9664°C ，干基含水率分别为 1.7662 kg/kg 和 1.0081 kg/kg ；径向温差已缩小至 0.1169°C ，含水率差仍为 0.7581 kg/kg ，表明温度场先于含水率场趋于均匀。

针对问题三，采用稳定段均值延拓与全节点逐秒阈值判定：以 9000~14400 s 的 91 个观测值均值延拓空气边界，并逐秒检查全部径向节点。在烘房于 14400 s 后继续维持同一平台工况的假设下，得到固定半径烘干时长为 57.4322 h，终点由轴线控制，终点中心含水率的未舍入值为 0.14999973 kg/kg 。在稳定段均值上下各扰动一个总体标准差及末条记录替代等五个确定性长期空气边界情景中，时长最大绝对相对变化为 0.5117%；空间网格加密后的时长相对差为 0.09044%。

针对问题四，将实测半径作为移动边界，建立移动材料坐标有效扩散模型，在固定单位区间内更新几何权重和附录四物性。模型在 51.0775 h 达到终点，此时半径为 1.2000 cm；同附录四物性的固定半径反事实需 129.7964 h。在空气边界、物性和离散设定相同的模型内对照下，引入实测半径边界后，计算终点提前 78.7189 h，相对缩短 60.6480%；该结果仅表示本文有效闭合模型内的几何边界影响，不解释为真实收缩的独立因果效应。取 $a_* = \pm 0.1, \pm 0.2$ 时最大结构偏差为 0.3143%；将 h 和 h_m 分别作 $\pm 20\%$ 单因素扰动，终点最大绝对相对变化分别为 0.0353% 和 1.3454%。空间网格与时间步加密的终点绝对差分别为 13 s 和 0.5 s。离散平衡、时空步加密及边界与结构敏感性结果共同表明，关键结论在本文设定的数值与情景范围内稳定。

关键字：圆柱药材烘干；径向有限体积法；变物性热质耦合；移动材料坐标；全域阈值判定

一、 问题重述

1.1 问题背景

中药材热风干燥由表面对流换热、传质与药材内部的热量传导、水分扩散共同驱动，通常经历预热平衡和恒温干燥两个阶段。烘干过程中，烘房温度与空气水分浓度随时间变化，药材的热物性和水分扩散能力又受温度、含水率影响，持续失水还会引起外形尺寸收缩。针对药用根类材料，北美西洋参根干燥实验表明，根体可随失水发生显著体积收缩，且表皮会阻碍水分外逸并影响收缩过程 [1]。因此，仅凭表面状态难以判断药材内部各处的温度和干基含水率是否均达到工艺要求。圆柱物料的热风干燥具有典型的非稳态热质传递特征 [2, 3]。题目据此给出烘房环境记录、药材物性关系及半径变化数据，要求刻画内部温度场与含水率场的时空演化，并依据全域含水率约束确定烘干终点。

图 1 展示药材处理场景，为本文讨论的干燥工艺提供背景。



图 1 药材处理场景

1.2 问题要求

本文需要解决以下四个问题：

问题一：以长 25 cm、初始半径 2 cm 的圆柱形药材为研究对象，根据给定的烘房温度、空气水分浓度和热质传递参数，建立预热平衡阶段的温度与干基含水率变化模型。药材初始温度为 28℃，初始干基含水率为 2.55 kg/kg。除计算指定时刻和指定径向位置的结果外，还需按 1 s 的时间间隔、0.1 cm 的空间间隔输出前 1 800 s 的完整结果。

问题二：在问题一的基础上，区分预热平衡与恒温干燥阶段的参数，并考虑药材物性随温度和含水率的变化，建立约 2–3 天全程烘干过程中温度与干基含水率的变化模型。给出前 3 h 内每隔 0.5 h、距轴线每隔 0.5 cm 的计算结果，并按 1 s 的时间间隔、0.1 cm 的空间间隔输出完整结果。

问题三：沿用问题二的全程烘干模型，在药材尺寸保持不变的条件下，以各处干基

含水率均严格低于 0.15 kg/kg 作为结束判据，确定烘干时长。给出每隔 6 h 、距轴线每隔 0.5 cm 的含水率，并按 60 s 的时间间隔、 0.1 cm 的空间间隔输出完整结果。

问题四：进一步考虑失水引起的尺寸变化，将实测半径变化及相应的物性关系纳入模型，沿用问题三的结束判据重新确定烘干时长。给出每隔 6 h 、距轴线每隔 0.5 cm 以及药材表面的含水率，并按 60 s 的时间间隔、 0.1 cm 的空间间隔输出完整结果。

二、 问题分析

四个问题以圆柱形药材内部温度场和干基含水率场为共同研究对象，整体呈现“定物性预热建模—变物性全程扩展—固定尺寸终点判定—收缩边界修正”的递进关系。问题一建立基本的径向传热传质框架；问题二引入温度、含水率与物性参数之间的反馈；问题三在固定几何条件下将场变量求解转化为干燥终点判定；问题四进一步把实测半径变化纳入移动边界。后续各问均在上一问模型的基础上增加新的物理机制，从而保持控制方程、边界条件和数值方法的一致性。

2.1 问题一：预热阶段径向场分析

问题一要求根据离散记录的烘房温度和空气水分浓度，计算前 1800 s 内药材各径向位置的温度与干基含水率。求解的关键是将逐分钟的环境记录转换为每个时间步均可调用的边界条件，并在保留主要传递特征的前提下降低空间维数。考虑到题目仅要求按距轴线位置输出结果，且药材长径比较大，本文将其近似为轴对称一维圆柱，并对两类环境数据作分段线性插值。随后分别依据 Fourier 导热定律和 Fick 扩散定律建立径向非稳态控制方程，在表面施加对流换热与对流传质边界，采用径向有限体积法和全隐式时间格式求解预热阶段的温度场与含水率场。

2.2 问题二：全程变物性分析

问题二由预热平衡阶段扩展至恒温干燥阶段，并要求考虑药材物性随局部状态变化。此时，含水率决定密度、比热容和导热系数，温度与含水率共同影响水分扩散系数，原有定物性方程因参数反馈转化为非线性顺序耦合问题。本文沿用问题一的径向控制体和表面对流边界，在每个时间层根据当前场值更新热物性与扩散系数；首层采用后向 Euler 格式，其余时间层采用二阶向后差分，并通过 Picard 迭代交替求解温度场和含水率场，直至两类场量同时满足收敛条件，由此建立全程变物性传热传质模型。

2.3 问题三：固定尺寸干燥终点分析

问题三要求在药材尺寸不变时，确定所有位置的干基含水率均严格低于 0.15 kg/kg 的最早时刻。该问的关键一是为实测时段以后的烘房状态构造有依据的边界延拓，二是将“各处达标”转化为可计算的全域事件判据。本文在实测区间内继续采用分段线性插值，并以末段稳定工况的均值延拓后续空气边界；在此基础上继承问题二的变物性模

型，以每个时间层全部径向节点含水率的最大值作为判定量，逐秒搜索其首次低于阈值的时刻。该方法不预设中心点必为控制位置，并使终点判定精度不受 60 s 结果输出间隔的限制。

2.4 问题四：收缩边界干燥终点分析

问题四进一步考虑失水引起的半径变化，计算域由固定圆柱转化为随时间收缩的移动区域。直接在物理坐标中删减外层网格会引起控制体突变，因此求解的关键是将移动边界映射到固定计算域，同时保持材料点、物性参数和表面通量之间的对应关系。本文对实测半径作分段线性插值，并引入归一化材料坐标 $x = r/R(t)$ ，把移动圆柱映射到单位区间；每个时间层同步更新几何权重、热物性和扩散系数，再沿用有限体积离散、Picard 迭代及全域最大含水率判据确定干燥终点。为检验结论对收缩闭合方式的依赖，还设置同物性固定半径对照和非均匀内部映射扰动，以区分几何变化与物性变化的影响。

三、 模型假设

结合题设条件，作如下假设：

假设 1：药材视为轴对称圆柱，温度和干基含水率仅随径向位置与时间变化，忽略轴向梯度和端面传递。

假设 2：药材视为均匀连续介质，内部传热与水分迁移分别服从 Fourier 定律和 Fick 定律。

假设 3：忽略化学反应热、辐射及托盘接触传热；因题目未给出相变参数，温度方程不显式计入汽化潜热。

假设 4：烘房空气在药材侧壁附近均匀，换热与传质系数恒定；实测结束后，以末段稳定均值延拓空气边界。

假设 5：问题一至三中药材半径不变；问题四仅考虑径向收缩，长度不变，半径分段线性变化，材料点的归一化坐标 $x = r/R(t)$ 保持不变。

四、 符号说明

正文以 T 表示绝对温度。仅含温度导数和温差的导热方程使用摄氏温度计算会得到相同结果；附录三和附录四的 Arrhenius 型扩散系数则必须代入绝对温度。表格统一输出摄氏温度，含水率采用题目给出的干基定义。表 1 列出正文使用的核心符号，离散上下标只在有限体积小节内解释。

表 1 主要符号及含义

符号	含义	单位
t	烘干开始后的时间	s
r	到药材轴线的径向距离	m
x	收缩模型中的归一化径向坐标	1
ξ	非均匀收缩模型中的材料坐标	1
L	药材长度	m
R	固定尺寸模型中的药材半径	m
$R(t)$	收缩模型中的药材半径	m
r_ξ	材料映射关于 ξ 的雅可比	m
a_*	内部非均匀收缩的结构参数	1
$T(r, t)$	药材绝对温度	K
$T_c(r, t)$	药材摄氏温度	°C
$T_a(t)$	烘房空气绝对温度	K
$C(r, t)$	药材干基含水率	kg/kg
$C_a(t)$	烘房空气水分浓度	kg/kg
ρ	药材密度	kg/m ³
c_p	药材比热容	J/(kg K)
k	药材导热系数	W/(m K)
D	有效水分扩散系数	m ² /s
h	表面对流换热系数	W/(m ² K)
h_m	表面对流传质系数	m/s
V_i	第 i 个径向控制体体积	m ³
$A_{i+\frac{1}{2}}$	相邻控制体之间的控制面面积	m ²
Δr	径向网格步长	m
Δt	时间步长	s

式中 $T_c = T - 273.15$ 。当正文给出附件一的原始温度时使用摄氏温度，代入附录三和附录四的 Arrhenius 型扩散系数时使用绝对温度，避免因温标混用造成数量级错误。

五、 模型求解

5.1 问题一的径向场求解

5.1.1 问题一的几何简化

二维轴对称圆柱模型把温度和含水率写成 $T(r, z, t)$ 与 $C(r, z, t)$ ，侧壁和端面分别设置对流边界 [4]。题目没有给出轴向观测位置，所有输出点均由到轴线的距离确定。端面总面积与侧面积之比为

$$\frac{2\pi R^2}{2\pi RL} = \frac{R}{L} = 0.08. \quad (1)$$

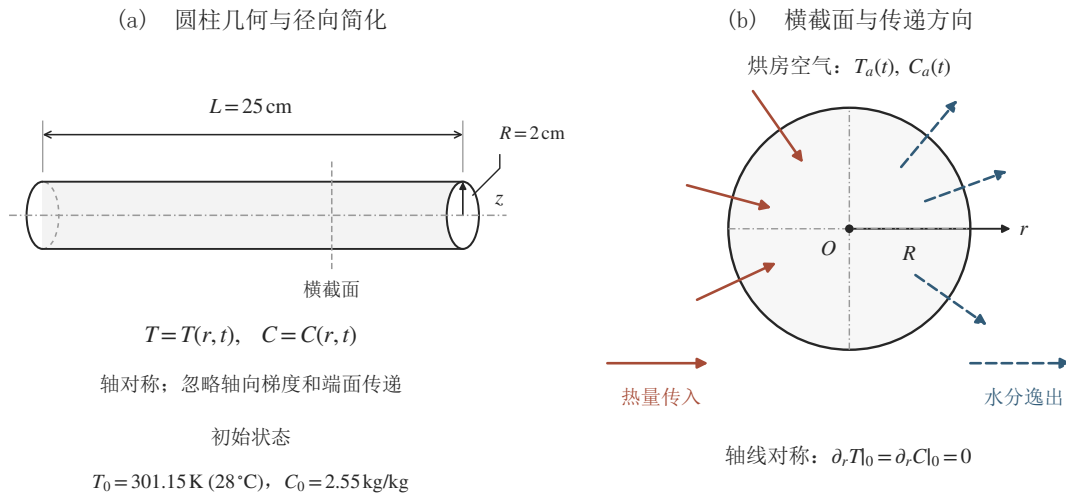
结合题定径向输出与式 1 的几何量级，本文把固定尺寸药材近似为一维径向圆柱。该面积比不是误差估计，端部效应仍属于模型结构不确定性。

降维后定义

$$T = T(r, t), \quad C = C(r, t), \quad 0 \leq r \leq R. \quad (2)$$

式 2 保留径向梯度，能够直接输出题目指定的五个位置。该简化不代表轴向温差恒为零，其偏差范围尚需由二维抽样计算或端部测温数据确定。

图 2 给出圆柱几何、径向坐标和边界传递方向；箭头表示升温失水工况，不代表通量大小。



侧壁边界（径向向外为正）

$$-k \frac{\partial T}{\partial r} \Big|_R = h[T(R, t) - T_a(t)]$$

$$-D \frac{\partial C}{\partial r} \Big|_R = h_m[C(R, t) - C_a(t)]$$

箭头示升温失水工况；横截面放大绘制，传质边界采用题设经验口径。

图 2 圆柱药材几何简化与径向传热传质示意图

5.1.2 问题一的温度模型

半径为 r 的圆柱面上，向外径向热流密度满足 Fourier 定律

$$q_r = -k \frac{\partial T}{\partial r}. \quad (3)$$

将式 3 代入环形微元的能量守恒关系，得到一维径向非稳态导热方程

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \quad 0 < r < R. \quad (4)$$

问题一取

$$\rho = 820 \text{ kg/m}^3, \quad c_p = 2600 \text{ J/(kg K)}, \quad k = 0.36 \text{ W/(m K)}. \quad (5)$$

药材初始温度为

$$T(r, 0) = 301.15 \text{ K}. \quad (6)$$

轴线处由对称性得到零梯度条件

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0. \quad (7)$$

表面热流由烘房空气和药材表面的温差驱动，故有

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h [T(R, t) - T_a(t)], \quad h = 25 \text{ W/(m}^2 \text{ K)}. \quad (8)$$

附件一给出的摄氏温度先作分段线性插值。若 $t_j \leq t \leq t_{j+1}$ ，则

$$T_a(t) = 273.15 + T_{a,j}^c + \frac{T_{a,j+1}^c - T_{a,j}^c}{t_{j+1} - t_j} (t - t_j). \quad (9)$$

式 9 保证记录时刻的边界值与附件一完全一致。

附件一在预热阶段的原始记录如图 3 所示。三十分钟内，烘房温度由 28°C 升至 41.513°C ，空气水分浓度由 0.01963 kg/kg 升至 0.03307 kg/kg 。两条边界曲线均保留观测点之间的局部斜率变化，没有用单一升温速率替代实测过程。

5.1.3 问题一的水分模型

把 $C(r, t)$ 定义为药材干基含水率。在固定域及恒定干固体参考密度的假设下，定义等效干基含水率通量

$$J_C = -D(C) \frac{\partial C}{\partial r}. \quad (10)$$

环形微元内的水分守恒给出

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C) \frac{\partial C}{\partial r} \right], \quad 0 < r < R. \quad (11)$$

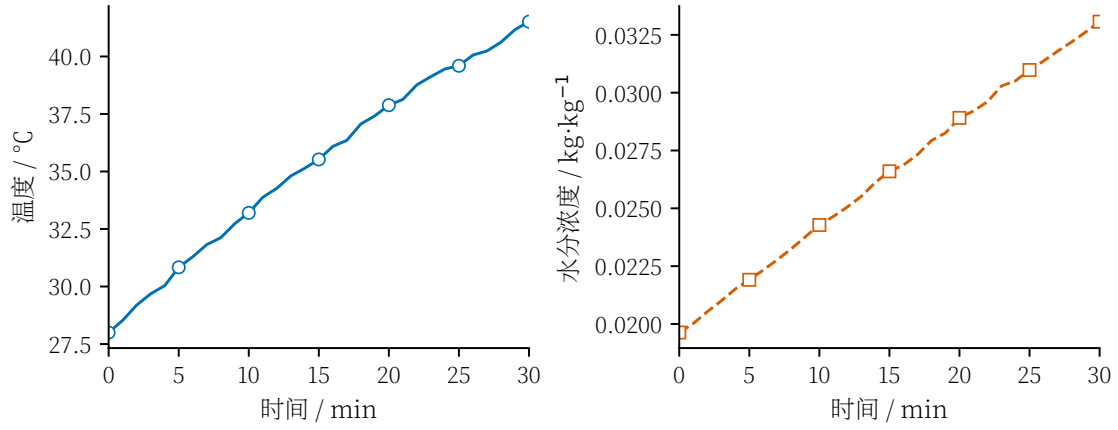


图3 问题一烘房环境边界的原始记录

附录二的经验式应按题面写为

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right). \quad (12)$$

式 12 的指数含 C 的倒数。含水率下降时，指数项随之减小，有效扩散系数降低，模型由此描述低含水率阶段逐渐增大的内部迁移阻力。水分初始条件和轴线边界为

$$C(r, 0) = 2.55, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0. \quad (13)$$

药材表面采用第三类传质边界

$$-D(C) \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m [C(R, t) - C_a(t)], \quad h_m = 8 \times 10^{-7} \text{ m/s}. \quad (14)$$

附件一中的空气水分浓度按式 9 的同类形式作分段线性插值。式 14 是题设口径下的经验 Robin 边界，用于描述表面水分离开药材的等效速率；它不表示药材干基含水率与一般湿空气含湿量天然具有同一浓度基准。

5.1.4 问题一的数值求解

在 $0 \leq r \leq R$ 上设置节点 $r_i = i\Delta r$ ，其中 $i = 0, 1, \dots, N$ 。节点两侧控制面为 $r_{i-\frac{1}{2}}$ 与 $r_{i+\frac{1}{2}}$ 。内部控制体体积及控制面面积分别为

$$V_i = \pi L \left(r_{i+\frac{1}{2}}^2 - r_{i-\frac{1}{2}}^2 \right), \quad (15)$$

$$A_{i\pm\frac{1}{2}} = 2\pi L r_{i\pm\frac{1}{2}}. \quad (16)$$

中心控制体取 $0 \leq r \leq \Delta r/2$ ，其内侧控制面面积为零。表面控制体取 $R - \Delta r/2 \leq r \leq R$ 。

时间层记为 $t_n = n\Delta t$ 。内部节点的全隐式温度离散式为

$$\begin{aligned} \frac{\rho c_p V_i}{\Delta t} (T_i^{n+1} - T_i^n) = & \frac{k A_{i-\frac{1}{2}}}{\Delta r} (T_{i-1}^{n+1} - T_i^{n+1}) \\ & + \frac{k A_{i+\frac{1}{2}}}{\Delta r} (T_{i+1}^{n+1} - T_i^{n+1}). \end{aligned} \quad (17)$$

中心节点只保留外侧控制面通量，表面节点在内侧导热通量外加入

$$h A_R (T_a^{n+1} - T_N^{n+1}), \quad A_R = 2\pi R L. \quad (18)$$

由此得到包含 T_0^{n+1} 至 T_N^{n+1} 的完整三对角方程组。

水分方程采用相同控制体。第 ℓ 次非线性迭代中，以当前含水率计算

$$D_{i+\frac{1}{2}}^{(\ell)} = \frac{D(C_i^{(\ell)}) + D(C_{i+1}^{(\ell)})}{2}, \quad (19)$$

再求解

$$\begin{aligned} \frac{V_i}{\Delta t} (C_i^{n+1,\ell+1} - C_i^n) = & \frac{D_{i-\frac{1}{2}}^{(\ell)} A_{i-\frac{1}{2}}}{\Delta r} (C_{i-1}^{n+1,\ell+1} - C_i^{n+1,\ell+1}) \\ & + \frac{D_{i+\frac{1}{2}}^{(\ell)} A_{i+\frac{1}{2}}}{\Delta r} (C_{i+1}^{n+1,\ell+1} - C_i^{n+1,\ell+1}). \end{aligned} \quad (20)$$

表面控制体再加入

$$-h_m A_R (C_N^{n+1,\ell+1} - C_a^{n+1}). \quad (21)$$

相邻两次迭代的含水率最大差不超过 10^{-10} 后进入下一时间层，单步最多迭代五十次。计算过程中实际最多进行四次迭代，全部时间层均达到收敛容差。线性方程组采用 Thomas 算法求解，基准网格下热量矩阵与水分矩阵的条件数分别为 1.889×10^3 和 1.254×10^3 ，该量级未显示明显的病态放大风险。条件数不单独作为精度结论，时间和空间离散误差仍由后述加密计算评估。

基准计算取 $\Delta r = 0.000125 \text{ m}$ 与 $\Delta t = 0.25 \text{ s}$ ，对应一百六十个径向小区间和七千二百个时间步。由于 1 s 等于四个内部时间步， 0.1 cm 等于八个径向步长，题目要求的输出时刻和位置均落在计算网格上，可直接抽取而无须额外插值。温度以摄氏度输出，数值按题目模板显示四位小数；该位数是输出格式，实际离散误差由后述加密差值量化。

5.1.5 问题一的模型计算结果

表 2 与表 3 分别对应题面表 1 和表 2，给出指定位置的计算值。药材表面温度在 1800 s 时达到 36.7857°C ，中心温度为 33.5756°C ，径向温差为 3.2101°C 。同一时刻表面含水率降至 1.5103 kg/kg ，中心含水率按四位小数仍为 2.5500 kg/kg ，两者相差 1.0397 kg/kg 。

表 2 三十分钟内药材温度（对应题面表 1）

时间	距轴线距离对应温度 °C				
s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	28.0001	28.0004	28.0041	28.0328	28.1802
300	28.0410	28.0637	28.1516	28.3683	28.8491
600	28.4537	28.5364	28.8043	29.3162	30.1654
900	29.3248	29.4587	29.8759	30.6165	31.7306
1200	30.5432	30.7102	31.2227	32.1129	33.4279
1500	31.9961	32.1871	32.7664	33.7465	35.1204
1800	33.5756	33.7723	34.3644	35.3624	36.7857

温度表显示外表面始终先于内部升温，且径向温差随加热过程逐渐向轴线传播，这与外表面对流换热、内部导热的边界条件和传递方向一致。

表 3 三十分钟内药材干基含水率（对应题面表 2）

时间	距轴线距离对应含水率 kg/kg				
s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	2.5500	2.5500	2.5500	2.5500	2.2476
300	2.5500	2.5500	2.5500	2.5492	2.0511
600	2.5500	2.5500	2.5500	2.5352	1.8771
900	2.5500	2.5500	2.5497	2.5045	1.7548
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5788
1800	2.5500	2.5497	2.5383	2.3755	1.5103

图 4 展示完整时空场。温度响应从外表面向轴线逐渐传播，含水率下降集中在靠近表面的区域，两个场的扩散深度存在差异。温度边界与内部场的响应方向一致，含水率始终由内部向表面递减，表面对流项的符号与物理过程相符。

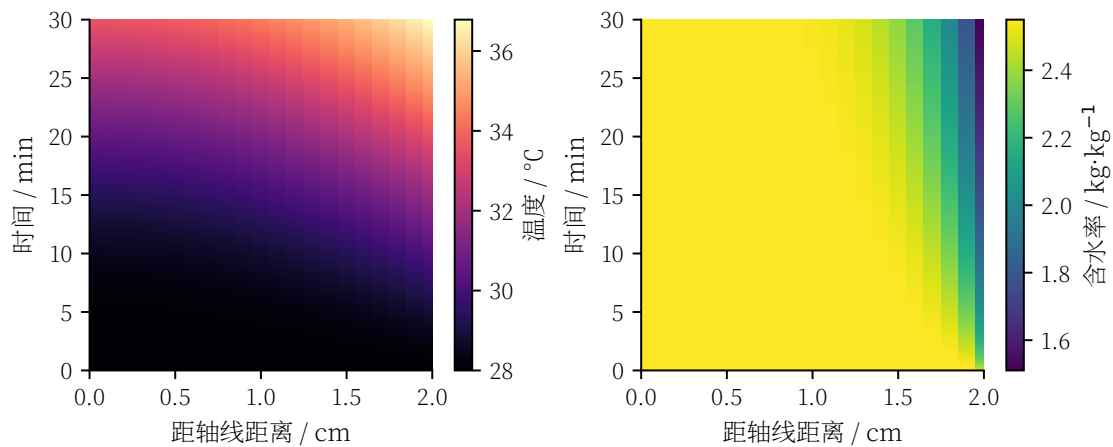


图 4 问题一温度与干基含水率径向场

将时间步缩小至 0.125 s 后，指定时空点的温度与含水率最大绝对差分别为 $2.389 \times 10^{-4} \text{ °C}$ 和 $5.139 \times 10^{-5} \text{ kg/kg}$ 。将空间步缩小至 0.000 062 5 m 后，对应差值分别为 $2.300 \times$

$10^{-5} \text{ }^{\circ}\text{C}$ 和 $3.505 \times 10^{-4} \text{ kg/kg}$ 。图 5 给出两类加密误差，含水率对表面附近的空分分辨率更敏感。上述差值表明，表格显示四位小数并不意味着所有末位数字都已在当前网格下稳定；本问结果应理解为给定基准离散方案的计算值。

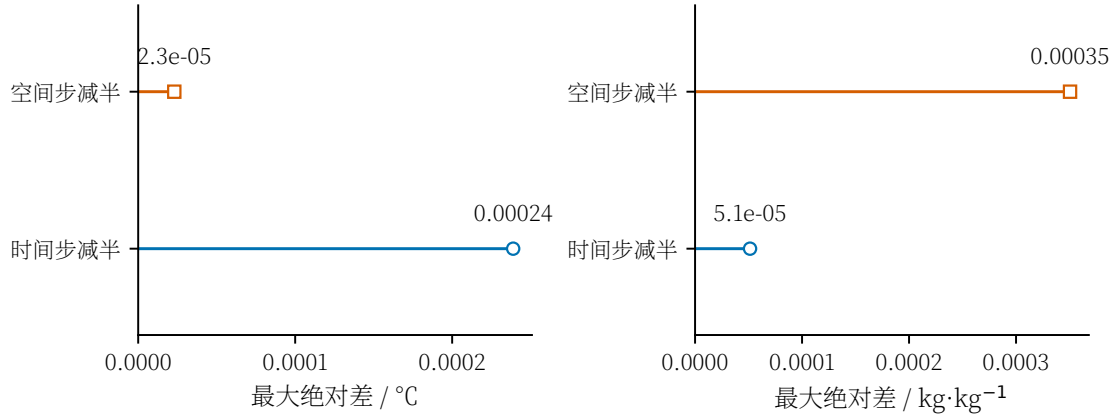


图 5 问题一数值加密误差

由控制体内等效含水率总变化与表面对流通量计算离散整体平衡，所得相对残差为 2.35×10^{-14} 。该数值接近双精度浮点运算的舍入尺度，说明相邻控制体共享通量在离散求和中能够抵消，表面通量与域内离散总量变化保持一致。这一残差用于检查数值实现，不构成对真实干固体与水分总质量守恒的独立验证。

5.2 问题二的全程变物性模型

附件一在前三小时内提供一百八十一条环境记录，采样间隔为 60 s。计算时刻始终落在实测区间内，空气温度 $T_a(t)$ 和空气水分浓度 $C_a(t)$ 均由相邻记录线性插值得到。图 6 显示两类边界量总体上升并伴随逐分钟波动，3 h 时的观测值分别为 $50.195 \text{ }^{\circ}\text{C}$ 和 0.04977 kg/kg 。

附录三没有另行给出表面对流系数，计算沿用附录二的 $h = 25 \text{ W}/(\text{m}^2 \text{ K})$ 和 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。这项处理属于新增假设，故在基准结果外分别把两个系数调整为原值的 80% 和 120%，检验边界参数对内部场的影响。

5.2.1 物性反馈机制

附录三把密度、比热容和导热系数写成局部含水率的函数，并令水分扩散系数同时受含水率和温度控制。各关系式为

$$\rho(C) = 650 + 128C, \quad (22)$$

$$c_p(C) = 1450 + 2736 \frac{C}{C+1}, \quad (23)$$

$$k(C) = 0.21 + 0.38 \frac{C}{C+1}, \quad (24)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T}\right). \quad (25)$$

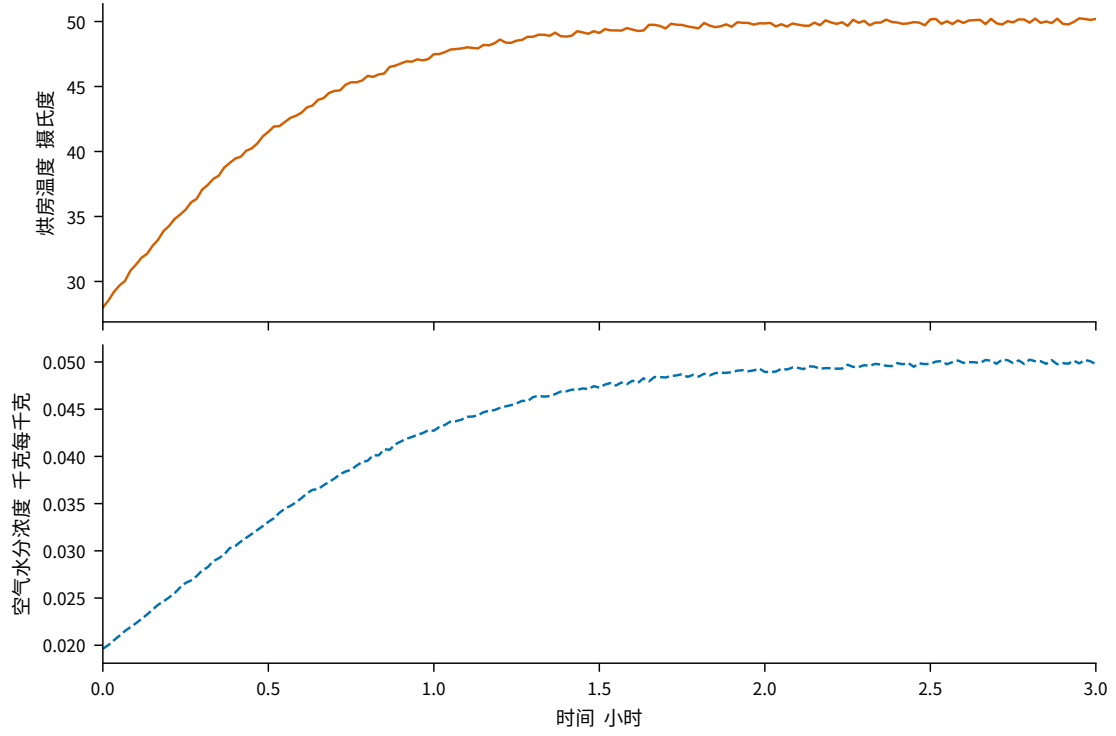


图 6 问题二前三小时的空气边界

式 25 中的 T 采用 K。每次 Picard 迭代先用当前含水率更新 ρ 、 c_p 和 k ，再用更新后的温度和含水率计算 D 。

三小时计算中， ρ 的取值范围为 779.0391 至 976.4000 kg/m³， c_p 为 2823.5301 至 3415.2958 J/(kg K)， k 为 0.400768 至 0.482958 W/(m K)。扩散系数 D 处于 5.5475×10^{-9} 至 1.2522×10^{-8} m²/s。这些数值在计算域内保持有限且为正，离散系统没有出现物性奇点。

5.2.2 有效场方程的控制体离散

对温度和含水率的有效场方程分别作环形控制体积分，可写成统一形式

$$\beta_i^{n+1,m} V_i \delta_t u_i^{n+1,m+1} = \sum_{f=i \pm \frac{1}{2}} \frac{\Gamma_f^{n+1,m} A_f}{\Delta r} \left(u_{\text{nb}(f)}^{n+1,m+1} - u_i^{n+1,m+1} \right). \quad (26)$$

温度方程取 $u = T$ 、 $\beta = \rho c_p$ 、 $\Gamma = k$ ，水分方程取 $u = C$ 、 $\beta = 1$ 、 $\Gamma = D$ 。相邻节点的物性算术平均值代表控制面物性，同一内部控制面的通量在相邻控制体中符号相反，求和后能够抵消。有限体积法以控制体积分和界面通量平衡为基本构造 [5]。这一性质保证所建有效场方程在离散层面的通量平衡；由于模型未另行定义干固体质量库存和含湿总焓，它不表示已经验证真实水分质量或总焓守恒。

轴线控制面的面积为零，式 26 自然满足对称条件。表面控制体把外侧通量改写为

$$S_{T,N} = h A_R (T_a - T_N), \quad S_{C,N} = -h_m A_R (C_N - C_a). \quad (27)$$

空气温度高于表面温度时，热源项为正。表面含水率高于空气水分浓度时，水分源项为负，两类符号均与药材升温和失水的方向一致。

5.2.3 时间层内迭代

首个时间层采用后向 Euler 格式，得到二阶格式需要的第二个历史状态。从第二个时间层起，式 26 中的时间导数采用

$$\delta_t u_i^{n+1} = \frac{3u_i^{n+1} - 4u_i^n + u_i^{n-1}}{2\Delta t}. \quad (28)$$

每次物性固定后，温度和含水率各形成一个三对角线性系统，并用追赶法求解。时间离散和非线性迭代的数值分析依据参见 Quarteroni 等的专著 [6]。

同一时间层内依次更新温度、扩散系数、含水率和热物性。当相邻两轮温度的最大绝对差低于 10^{-8}°C ，且含水率的最大绝对差低于 10^{-10} kg/kg 时停止迭代。程序把迭代上限设为四十次，实际有四千二百九十个时间层经过三次迭代收敛，六千五百一十个时间层经过四次迭代收敛，未触及上限。

基准计算把半径等分为一百六十个区间， $\Delta r = 0.000125 \text{ m}$ ，时间步取 $\Delta t = 1 \text{ s}$ 。程序保存一秒至一万零八百秒的每个时间层。由于 0.1 cm 等于八个径向步长，零至二厘米的二十一个输出位置均与网格节点重合，程序直接抽取相应节点结果，无须额外空间插值。温度表和水分浓度表各含一万零八百行数据及二十二列，表中数值保留四位小数。

5.2.4 径向场结果

表 4 和表 5 选取六个时刻及五个径向位置。选定位置覆盖轴线、内层、外层和表面，可直接观察场量从边界向内部传播的过程。

表 4 问题二指定时空点的温度

时间	不同半径处温度 $^{\circ}\text{C}$				
	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5137	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

0.5 h 时表面温度为 35.4130°C ，中心温度为 32.1892°C ，径向温差为 3.2238°C 。到 3 h，对应温度升至 49.9664°C 和 49.8495°C ，温差缩小到 0.1169°C 。图 7 表明高温区由表面向轴线推进，后段径向温度趋于均匀。

该温度场进一步表明：早期热量主要积聚在近表层，随后等温线逐渐变平，因而径

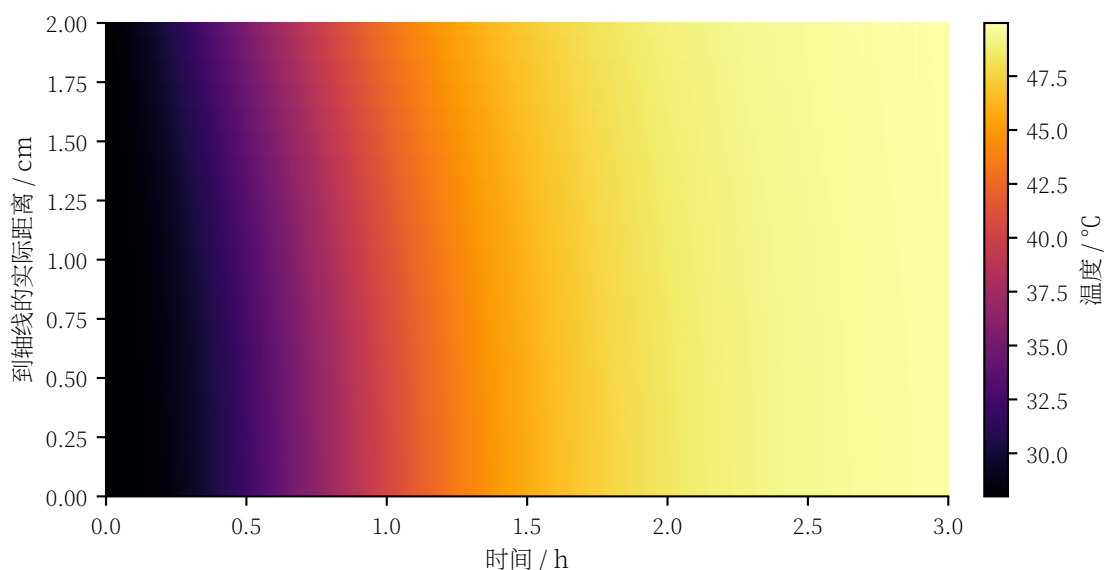


图 7 问题二前三小时的径向温度场

向温差在后段明显收敛。

表 5 问题二指定时空点的干基含水率

时间	不同半径处干基含水率 kg/kg				
h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5	2.5499	2.5489	2.5255	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

0.5 h 时中心含水率为 2.5499 kg/kg，表面含水率已降至 1.6486 kg/kg。到 3 h，两处数值分别为 1.7662 kg/kg 和 1.0081 kg/kg，中心比表面高 0.7581 kg/kg。图 8 显示低含水率区持续向内部扩展，轴线附近的扩散滞后仍未消失。

图 9 将两类径向差值分开比较。温差在早期达到峰值后总体缩小，而含水率差在三小时末仍保持较大幅度。这一差异直接表明温度场先于含水率场趋于均匀，不能仅凭温度接近均匀就判断内部失水已经完成。

5.2.5 数值精度检验

把时间步缩小至 0.5 s 后，指定时空点的温度和含水率最大绝对差分别为 $1.9005 \times 10^{-6} ^\circ\text{C}$ 和 $2.0140 \times 10^{-8} \text{ kg/kg}$ ，四位小数结果均未改变。把径向区间数增加至三百二十后，对应差值为 $1.3031 \times 10^{-5} ^\circ\text{C}$ 和 $1.1647 \times 10^{-5} \text{ kg/kg}$ 。温度表没有变化，含水率表仅有两个单元格的末位改变一个单位，全部差值低于四位小数舍入半单位 5×10^{-5} 。

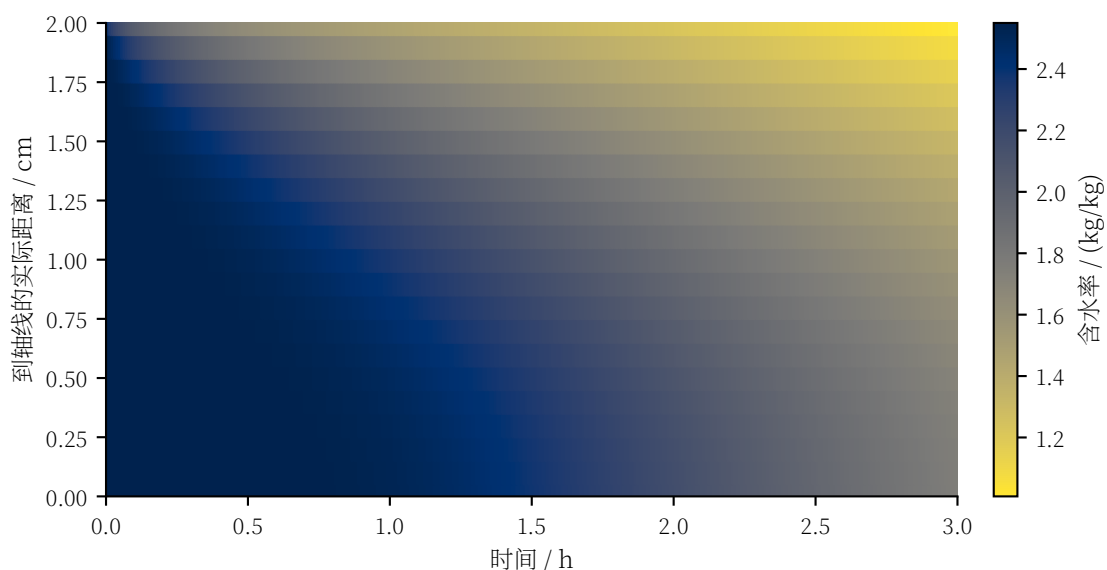


图 8 问题二前三小时的径向干基含水率场

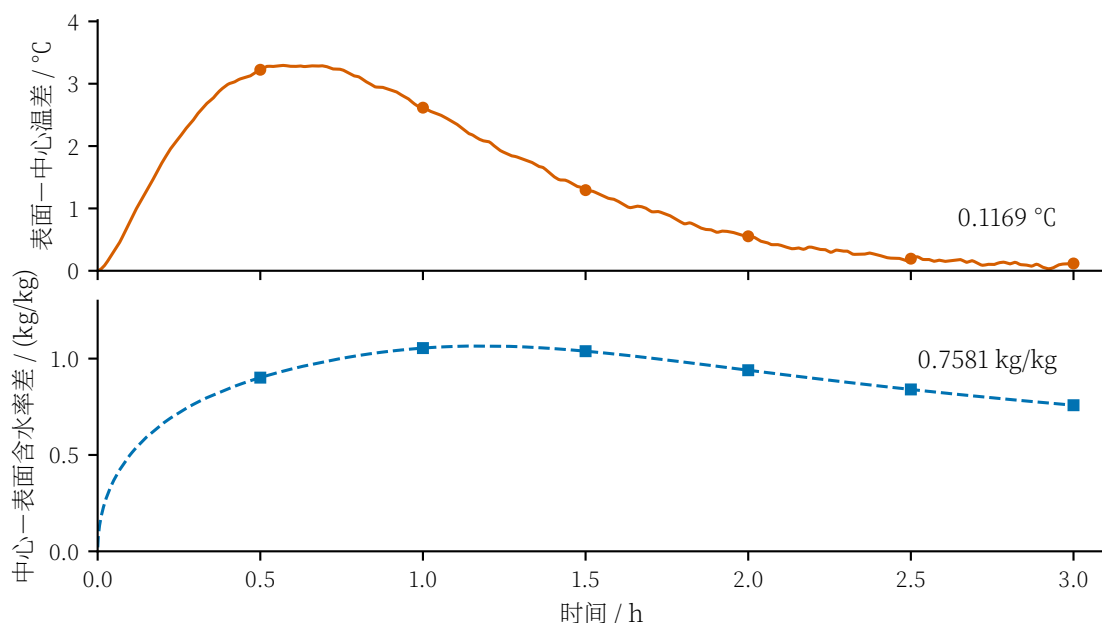


图 9 问题二中心与表面之间的温差和含水率差；标记对应每隔半小时的报告时刻

等效含水率场离散整体平衡的最大相对残差为 1.6932×10^{-11} 。温度和含水率三对角系统的最大线性残差分别为 6.9221×10^{-15} 和 7.5490×10^{-16} ，末时刻矩阵条件数分别为 2189.789 和 1243.281。这些数值用于检查离散装配、线性求解和非线性迭代的一致性，不构成真实水分质量或总焓守恒的独立证明。

在题目要求的六个时刻和五个径向位置组成的三十个报告点上，换热系数取基准值的 80% 和 120% 时，温度最大变化分别为 1.0817°C 和 0.7885°C ，含水率最大变化分别为 0.0154 kg/kg 和 0.0105 kg/kg 。传质系数采用相同比例扰动时，温度最大变化为 0.0567°C 和 0.0492°C ，含水率最大变化为 0.1694 kg/kg 和 0.1348 kg/kg 。图 10 表明，在上述单因素扰动和报告点范围内，温度对换热系数更敏感，含水率对传质系数更敏感。

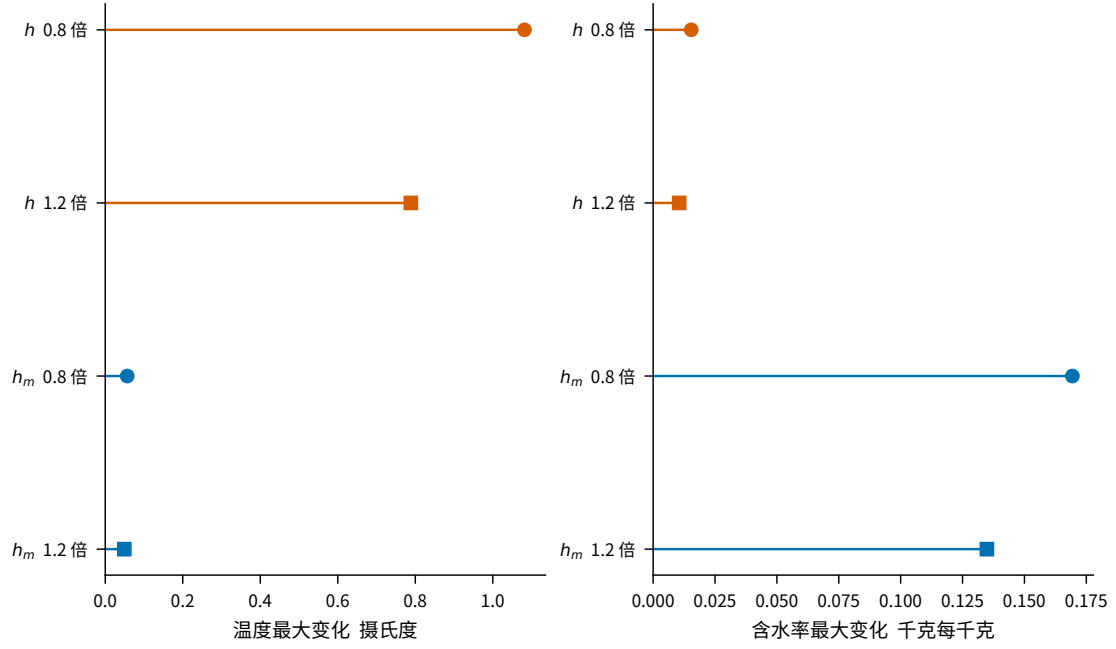


图 10 问题二边界参数在三十个报告点上的确定性灵敏度

5.3 问题三的固定域求解

5.3.1 稳定边界的取值

附件一后段围绕平台水平波动。选取 9 000 s 至 14 400 s 的九十一个观测点，得到温度均值 50.004 945 °C，总体标准差 0.154 016 °C；空气水分浓度均值为 0.049 993 736 kg/kg，总体标准差为 0.000 153 463 kg/kg。

稳定段温度从 49.757 °C 变化至 50.246 °C，空气水分浓度从 0.049 75 kg/kg 变化至 0.050 25 kg/kg。这些观测量给出均值延拓的波动尺度，也为后续确定性敏感性检验提供扰动幅度。

已观测时段仍对附件数据作分段线性插值。后续边界取稳定段均值，写为

$$(T_a(t), C_a(t)) = \begin{cases} (T_{a,\text{lin}}(t), C_{a,\text{lin}}(t)), & 0 \leq t \leq 14\,400 \text{ s}, \\ (323.154\,945 \text{ K}, 0.049\,993\,736 \text{ kg/kg}), & t > 14\,400 \text{ s}. \end{cases} \quad (29)$$

式 29 只固定烘房状态，药材内部场仍按非稳态方程推进。该延拓以烘房在实测结束后继续维持同一平台工况为前提。末条记录另作替代工况，用来考察已设情景内边界取值对干燥时长的影响，而不把该情景差异解释为统计置信区间。

5.3.2 全域阈值的事件判定

问题三继承式 22 至式 25 的物性关系、式 26 的有效场有限体积离散式和式 27 的表面通量。对时间层 $t_n = n\Delta t$ 及全部径向节点定义

$$C_{\max}^n = \max_{0 \leq i \leq N} C_i^n, \quad t_{\text{dry}}^{\text{fixed}, \Delta t} = \min \{t_n = n\Delta t : C_{\max}^n < 0.15\}. \quad (30)$$

式 30 在全部径向节点上取最大值，不预设控制位置。它给出时间网格上的首次达标时刻；本文不对相邻时间层之间的连续阈值根作插值。早期近似均匀场存在数值并列，径向梯度形成后中心值等于全域最大值。

首个时间层用后向 Euler 格式，其余时间层采用式 28 的二阶向后差分。每层 Picard 迭代在温度最大变化不超过 $1 \times 10^{-8}^{\circ}\text{C}$ 且含水率最大变化不超过 $1 \times 10^{-10} \text{ kg/kg}$ 时停止，迭代上限为四十次。程序以 1 s 时间步检查式 30，以 60 s 间隔写入结果，终点不落在常规记录时刻时单独追加。

空间网格采用三百二十个径向区间。输出表从 0 cm 到 2 cm 每隔 0.1 cm 给出含水率，加上时间列共二十二列。表内数值统一保留四位小数，完整文件含三千四百四十六条时间记录。

5.3.3 固定尺寸下的干燥历程

在 $\Delta t = 1 \text{ s}$ 的时间网格上，全域最大含水率于 206 756 s 首次低于阈值，对应固定半径烘干时长 57.4322 h。前一常规记录 206 700 s 的最大含水率为 $0.150\,016\,07 \text{ kg/kg}$ ，终点值为 $0.149\,999\,73 \text{ kg/kg}$ 。图 11 给出全程轨迹，并将前一常规记录及终点的 $C_{\max}^n - 0.15$ 单独绘出。终点偏差为 $-2.669 \times 10^{-7} \text{ kg/kg}$ ，严格低于零；该时长是逐秒检查得到的首个离散满足时刻，不解释为连续时间阈值方程的精确根，阈值跨越由中心含水率控制。

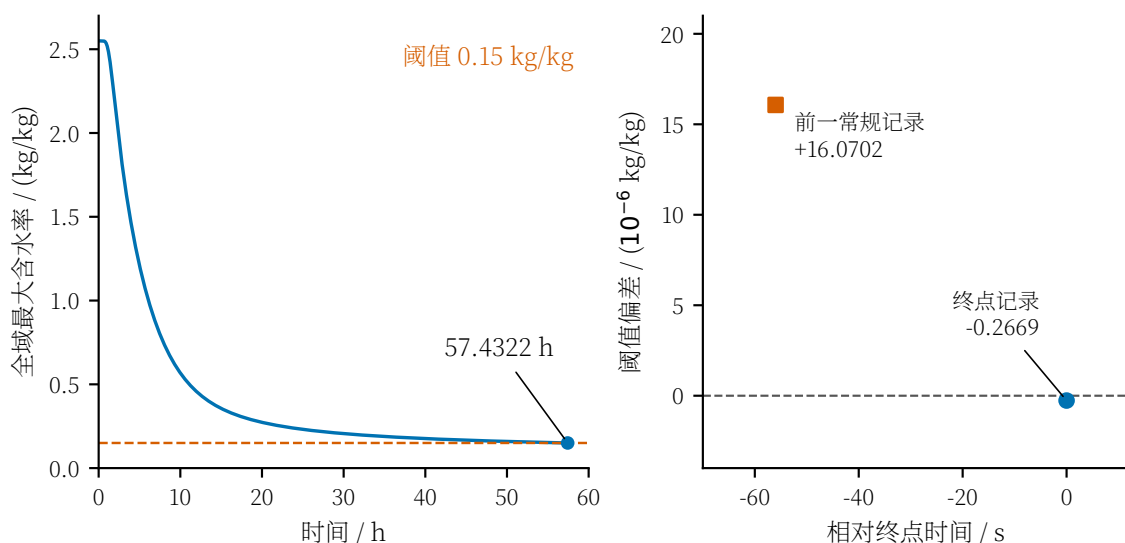


图 11 问题三全域最大含水率及终点阈值偏差；右图为相隔 56 s 的两条已保存记录，首次时刻由 1 s 步长检测确定

表 6 列出每隔六小时及烘干结束时的五个指定位置结果。表面含水率在 18 h 已降至 $0.084\,5 \text{ kg/kg}$ ，中心仍为 $0.298\,8 \text{ kg/kg}$ 。到 54 h，中心含水率为 $0.153\,8 \text{ kg/kg}$ ，离结束条件只差 $0.003\,8 \text{ kg/kg}$ 。

终点行的中心值按题目要求显示为四位小数，未舍入值为 $0.149\,999\,73 \text{ kg/kg}$ ，因此满足严格小于阈值的条件。图 12 显示外层先进入低含水率区，中心曲线下降最慢。中心与表面的先后差异说明体积平均值不能替代全域判据。

表 6 问题三指定位置的干基含水率

时间	不同半径处干基含水率 kg/kg				
	h	0 cm	0.5 cm	1.0 cm	1.5 cm
6.0000	1.0169	0.9881	0.9015	0.7550	0.5328
12.0000	0.4561	0.4431	0.4030	0.3297	0.1642
18.0000	0.2988	0.2915	0.2687	0.2253	0.0845
24.0000	0.2378	0.2327	0.2167	0.1855	0.0665
30.0000	0.2058	0.2018	0.1891	0.1640	0.0598
36.0000	0.1858	0.1825	0.1718	0.1503	0.0566
42.0000	0.1720	0.1691	0.1597	0.1406	0.0548
48.0000	0.1618	0.1591	0.1506	0.1334	0.0537
54.0000	0.1538	0.1514	0.1436	0.1276	0.0530
57.4322	0.1500	0.1477	0.1402	0.1249	0.0527

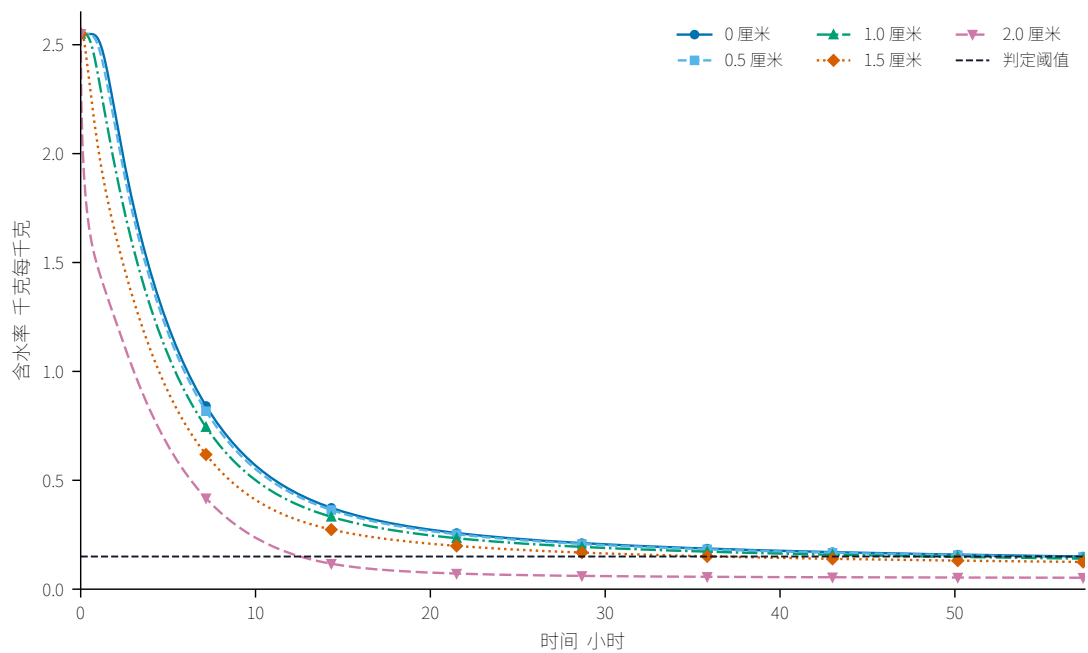


图 12 问题三五个指定径向位置的含水率历程

5.3.4 数值可靠性检验

时间步检验固定一百六十个径向区间。1 s、0.5 s 和 0.25 s 对应的烘干时长分别为 206 569 s、206 568.5 s 和 206 568.5 s，前两者相对最细时间步的绝对差为 0.5 s 和零，分别满足 1 s 与 0.5 s 的预设门槛。空间检验固定 1 s 时间步，一百六十与三百二十区间的烘干时长相对差为 0.090 44 %。指定表值的最大差为 $1.030\ 7 \times 10^{-4}$ kg/kg，超过四位小数舍入半单位，故最终取三百二十区间。

基准解的最大含水率反向增量为 $3.996\ 8 \times 10^{-15}$ kg/kg，有效含水率场离散整体平

衡的最大相对残差为 6.8716×10^{-10} 。温度与含水率线性系统的最大相对残差分别为 3.0249×10^{-14} 和 2.4208×10^{-15} ，末时刻条件数分别为 8386.07 和 1771.93。全部时间层最多用四次 Picard 迭代。这些指标用于检查离散装配、线性求解和非线性迭代的一致性，且未改变离散时间步上的终点判定；它们不等同于真实水分质量守恒检验。

稳定温度减少和增加一个总体标准差时，烘干时长分别变化 0.4904% 和 -0.4875%；空气水分浓度作相同方式扰动时，变化幅度不超过 0.0063%。采用附件末条记录会使时长缩短 0.5117%。另将换热系数和传质系数同时放大 8%，作为集总边界参数的应力情景，所得时长缩短 0.8372%。图 13 仅表明终点时长在上述长期空气边界情景和集总系数扰动下的变化均未超过百分之一；8% 系数扰动不构成端面通量上界，也不能据此限定真实二维端面效应。

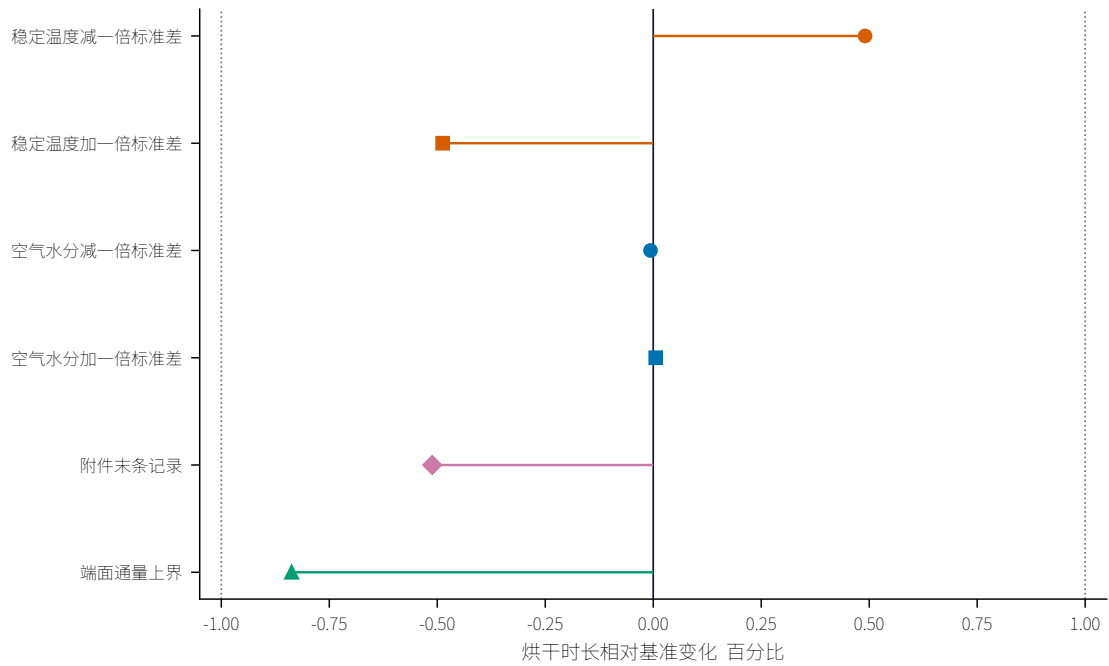


图 13 问题三长期空气边界与集总系数扰动情景

5.4 问题四的收缩域求解

5.4.1 移动域控制方程

附件二的半径记录覆盖零至七十二小时。全部一百四十五个观测点按时间顺序连接，半径由 2 cm 减小至 1.198 cm，收缩速度在后段趋近于零。计算在相邻记录间作分段线性插值，既保留观测值，又避免额外拟合参数。图 14 给出模型实际调用的半径边界。

设相邻观测点为 (t_j, R_j) 和 (t_{j+1}, R_{j+1}) ，计算中的半径边界取

$$R(t) = R_j + \frac{R_{j+1} - R_j}{t_{j+1} - t_j}(t - t_j), \quad t_j \leq t \leq t_{j+1}. \quad (31)$$

该式在每个观测时刻严格返回附件记录值，主模型终点位于观测区间内，不需要对半径作区间外延拓。

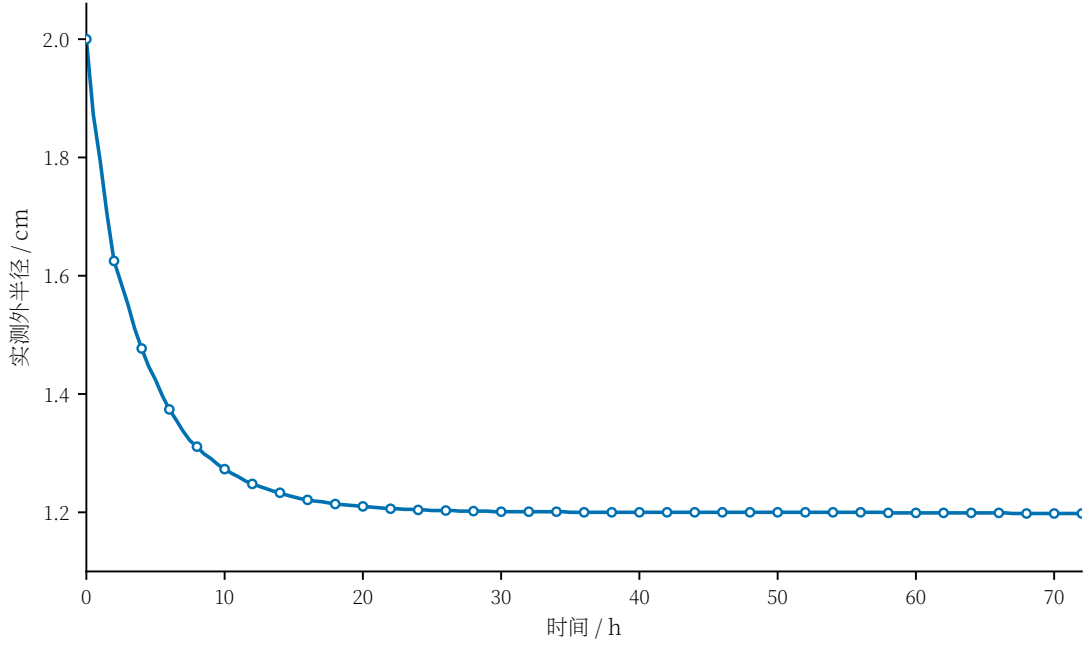


图 14 附件二半径记录构成的移动边界

固定物理坐标会随外表面内移而退出药材区域。食品干燥研究可在移动坐标中统一描述内部传递和几何变化 [7]。为保持网格拓扑不变，令

$$x = \frac{r}{R(t)}, \quad 0 \leq x \leq 1, \quad (32)$$

相似收缩假设规定材料点保持固定的 x ，因此 x 同时是计算坐标和材料标记。该假设只由外半径记录闭合；题面没有提供内部位移观测，因而不能据此辨识真实的局部收缩场。材料坐标中的温度及干基含水率分别记为 $\tilde{T}(x, t)$ 和 $\tilde{C}(x, t)$ 。材料点的径向速度为

$$v_r(r, t) = \frac{rR'(t)}{R(t)}. \quad (33)$$

在本文采用的有效标量扩散闭合下， C 表示随材料点运动的干基含水率状态量。由于题面缺少干固体局部密度、孔隙率以及它们与变形雅可比之间的质量闭合关系，以下方程不用于声称严格的局部水分质量守恒，而是在给定半径轨迹和相似收缩速度下描述等效传递。移动物理域中的有效控制方程写成材料形式

$$\rho c_p \left(\frac{\partial T}{\partial t} \Big|_r + v_r \frac{\partial T}{\partial r} \right) = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \quad (34a)$$

$$\frac{\partial C}{\partial t} \Big|_r + v_r \frac{\partial C}{\partial r} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right). \quad (34b)$$

两式左端包含固定物理位置的时间变化和材料运动引起的对流变化，不能从静止域方程直接删去后一项。

对任一场量 u 定义 $\tilde{u}(x, t) = u(xR(t), t)$ 。时间链式法则和径向算子变换给出

$$\left. \frac{\partial \tilde{u}}{\partial t} \right|_x = \left. \frac{\partial u}{\partial t} \right|_r + xR'(t) \frac{\partial u}{\partial r} = \left. \frac{\partial u}{\partial t} \right|_r + v_r \frac{\partial u}{\partial r}, \quad (35a)$$

$$\frac{\partial u}{\partial r} = \frac{1}{R(t)} \frac{\partial \tilde{u}}{\partial x}, \quad (35b)$$

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \Gamma \frac{\partial u}{\partial r} \right) = \frac{1}{xR^2(t)} \frac{\partial}{\partial x} \left(x \Gamma \frac{\partial \tilde{u}}{\partial x} \right). \quad (35c)$$

其中 Γ 在温度方程中取 k ，在水分方程中取 D 。第一式使用了 $xR'(t) = v_r$ 。材料运动项与固定位置导数合并为固定 x 的时间导数，并非在换元时被忽略。把这些关系代入式 34，得到

$$\rho c_p \frac{\partial \tilde{T}}{\partial t} = \frac{1}{xR^2(t)} \frac{\partial}{\partial x} \left(xk \frac{\partial \tilde{T}}{\partial x} \right), \quad (36)$$

$$\frac{\partial \tilde{C}}{\partial t} = \frac{1}{xR^2(t)} \frac{\partial}{\partial x} \left(xD \frac{\partial \tilde{C}}{\partial x} \right). \quad (37)$$

其中 $R^{-2}(t)$ 表示扩散距离缩短在本文有效闭合下产生的几何作用，不代表从实验中独立识别出的收缩机理。中心保持零通量，表面边界为

$$-\frac{k}{R(t)} \left. \frac{\partial \tilde{T}}{\partial x} \right|_{x=1} = h \left[\tilde{T}(1, t) - T_a(t) \right], \quad (38)$$

$$-\frac{D}{R(t)} \left. \frac{\partial \tilde{C}}{\partial x} \right|_{x=1} = h_m \left[\tilde{C}(1, t) - C_a(t) \right]. \quad (39)$$

附录四给出的物性关系随局部状态更新

$$\rho(C) = 760 + 90C, \quad (40)$$

$$c_p(C) = 1850 + 2150 \frac{C}{C+1}, \quad (41)$$

$$k(C) = 0.12 + 0.20 \frac{C}{C+1}, \quad (42)$$

$$D(C, T) = 4.2 \times 10^{-4} \exp \left(-\frac{0.30}{C} \right) \exp \left(-\frac{3850}{T} \right). \quad (43)$$

扩散系数中的温度取 K。空气边界沿用问题三的实测插值和稳定段均值，换热系数与传质系数保持不变，使问题四只引入附录四物性和半径边界。

5.4.2 内部映射检验

附件二只记录外半径，无法由题面唯一确定内部材料点的位移。相似收缩作为主模型闭合，不把局部密度反演成真实干固体运动。为检验这一闭合对终点的影响，引入初

始材料坐标 ξ ，并构造保持同一外半径的映射族

$$r(\xi, t) = R(t) [\xi + a_* g(t) \xi (1 - \xi)], \quad (44)$$

$$g(t) = \frac{1 - R(t)/R_0}{1 - R_{\text{end}}/R_0}, \quad (45)$$

$$r_\xi = R(t) [1 + a_* g(t) (1 - 2\xi)]. \quad (46)$$

映射在中心取零，在表面取 $R(t)$ 。参数 a_* 取 $-0.2, -0.1, 0, 0.1, 0.2$ ，计算区间内均有 $r_\xi > 0$ ，材料点的径向顺序保持不变。

一般映射下的热质方程可写为统一形式

$$b \frac{\partial u}{\partial t} = \frac{1}{rr_\xi} \frac{\partial}{\partial \xi} \left(\frac{r\Gamma}{r_\xi} \frac{\partial u}{\partial \xi} \right), \quad (47)$$

温度方程取 $b = \rho c_p$ 、 $u = T$ 和 $\Gamma = k$ ，水分方程取 $b = 1$ 、 $u = C$ 和 $\Gamma = D$ 。当 $a_* = 0$ 时，式 47 退化为式 36 及式 37，上述五个参数情景属于同一模型族。

固定尺寸对照同样采用附录四物性、空气边界、初始条件和数值算法，仅将半径恒定为初始值 R_0 。设相似收缩时长为 $t_{\text{dry}}^{(4)}$ ，固定尺寸时长为 $t_{\text{dry}}^{(4,0)}$ ，净几何效应定义为

$$\eta_R = \frac{t_{\text{dry}}^{(4)} - t_{\text{dry}}^{(4,0)}}{t_{\text{dry}}^{(4,0)}} \times 100\%. \quad (48)$$

该对照将半径固定为初始值，不使用问题三的物性结果，因而能够分离附录四体系内的几何贡献。

5.4.3 离散求解

单位材料区间划分为三百二十个径向控制体。控制体体积和控制面面积在每个时间层按当前映射更新，内部通量在相邻控制体间取相反符号。首个时间层采用后向 Euler 格式，其余时间层采用式 28 的二阶向后差分，正式时间步为 1 s。

记第 i 个控制体的环形体积权重为 w_i^n ，相邻节点间的扩散导通系数为 $G_{i+1/2}^n$ 。统一变量 u 的离散方程写为

$$w_i^n = \frac{\left(r_{i+1/2}^n\right)^2 - \left(r_{i-1/2}^n\right)^2}{2}, \quad (49)$$

$$G_{i+1/2}^n = \frac{r_{i+1/2}^n \Gamma_{i+1/2}^n}{r_{i+1}^n - r_i^n}, \quad (50)$$

$$\frac{b_i^n w_i^n}{\Delta t_{\text{eff}}} (u_i^n - H_{u,i}^n) = G_{i-1/2}^n (u_{i-1}^n - u_i^n) + G_{i+1/2}^n (u_{i+1}^n - u_i^n). \quad (51)$$

首层取 $H_{u,i}^n = u_i^{n-1}$ 和 $\Delta t_{\text{eff}} = \Delta t$ ，后续时间层取 $H_{u,i}^n = (4u_i^{n-1} - u_i^{n-2})/3$ 和 $\Delta t_{\text{eff}} =$

$2\Delta t/3$ 。对表面节点，完整离散方程写为

$$\frac{b_N^n w_N^n}{\Delta t_{\text{eff}}} (u_N^n - H_{u,N}^n) = G_{N-1/2}^n (u_{N-1}^n - u_N^n) + R^n \gamma_u (u_a^n - u_N^n), \quad (52)$$

$$\left(\frac{b_N^n w_N^n}{\Delta t_{\text{eff}}} + G_{N-1/2}^n + R^n \gamma_u \right) u_N^n = \frac{b_N^n w_N^n}{\Delta t_{\text{eff}}} H_{u,N}^n + G_{N-1/2}^n u_{N-1}^n + R^n \gamma_u u_a^n. \quad (53)$$

温度方程取 $(u, u_a, b, \Gamma, \gamma_u) = (T, T_a, \rho c_p, k, h)$ 。

水分方程取 $(u, u_a, b, \Gamma, \gamma_u) = (C, C_a, 1, D, h_m)$ 。因此表面对流项在通量形式中位于等式右侧；移项后，矩阵主对角线增加 $R^n \gamma_u$ ，右端项增加 $R^n \gamma_u u_a^n$ ，与程序装配一致。

每个时间层先依据当前含水率更新 ρ 、 c_p 和 k ，求得温度后再更新 D 并求解含水率。温度及含水率相邻两轮的最大绝对差分别低于 10^{-8}°C 和 10^{-10} kg/kg 时停止 Picard 迭代，迭代上限为四十次。主模型全部时间层最多迭代四次，其中九万三千四百四十九层用三次，九万零四百三十层用四次。

干燥终点在每个内部时间步检查。对时间层 $t_n = n\Delta t$ 定义

$$t_{\text{dry}}^{(4)} = \min \left\{ t_n = n\Delta t : \max_{0 \leq i \leq N} \tilde{C}_i^n < 0.15 \right\}. \quad (54)$$

式 54 给出离散时间网格上的首次达标时刻，不对相邻时间层之间的连续阈值根作插值。结果文件每隔 60 s 保存一次，并额外保存终点。固定实际位置通过 $x = r/R(t)$ 映射到当前网格，超出当前半径的位置记为域外，另设一列记录实时表面值。

5.4.4 干燥终点结果

主模型在 183 879 s 首次满足全域严格阈值，对应 51.077 5 h。前一秒的最大含水率为 0.150 000 202 481 kg/kg，终点值为 0.149 999 688 772 kg/kg，控制位置位于轴线。

计算终点的半径为 1.200 0 cm，早于附件二七十二小时观测结束时刻，计算没有使用半径末值延拓。

把两次判定值减去阈值可得

$$\Delta C_{\text{pre}} = 2.02481 \times 10^{-7} > 0, \quad \Delta C_{\text{end}} = -3.11228 \times 10^{-7} < 0. \quad (55)$$

符号变化表明终点由未舍入结果确定，表 7 中显示的四位小数不参与阈值判定。

图 15 显示低含水率区域由表面向轴线推进，药材区域随外表面内移而缩小。白色部分表示固定物理位置已经位于药材域外，不属于数据缺失。径向梯度在后段减弱，中心始终是最晚越过阈值的位置。

表 7 给出六小时间隔及终点处的代表性结果。六小时至终点期间，半径由 1.374 0 cm 减小至 1.200 0 cm，中心含水率由 1.719 5 kg/kg 降至阈值以下。固定位置 1.5 cm 在这些时刻均位于药材域外，表面值另列，避免把被收缩移出的空间点误写成表面值。

同附录四物性的固定半径反事实在 467 267 s 达到阈值，对应 129.796 4 h。式 48 给

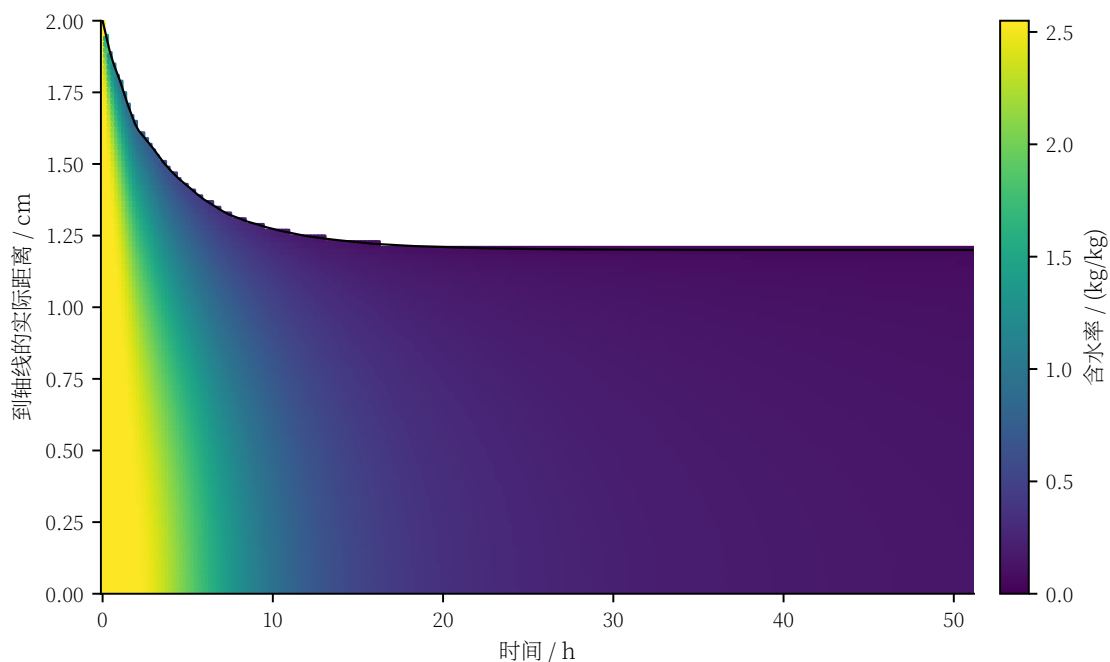


图 15 收缩域内干基含水率的时空演化

表 7 考虑尺寸收缩的药材干基含水率

时间 h	半径 cm	0 cm	0.5 cm	1.0 cm	1.5 cm	药材表面
6.0000	1.3740	1.7195	1.5377	1.0226	域外	0.4207
12.0000	1.2480	0.7376	0.6545	0.4083	域外	0.1669
18.0000	1.2140	0.4087	0.3686	0.2397	域外	0.0892
24.0000	1.2040	0.2851	0.2610	0.1790	域外	0.0673
30.0000	1.2010	0.2263	0.2093	0.1492	域外	0.0595
36.0000	1.2000	0.1928	0.1796	0.1316	域外	0.0560
42.0000	1.2000	0.1712	0.1603	0.1199	域外	0.0541
48.0000	1.2000	0.1561	0.1468	0.1115	域外	0.0530
51.0775	1.2000	0.1500	0.1413	0.1081	域外	0.0526

终点中心值按四位小数显示为 0.1500，判定使用的未舍入值严格小于 0.15。

出 $\eta_R = -60.6480\%$ ：在空气边界、物性与离散参数均相同的模型内对照下，引入实测半径边界后，计算终点较固定半径反事实提前 78.7189 h。图 16 显示两个情景使用同一阈值，但由于所设几何边界不同，终点相隔 283 388 s。

计算结果代入几何效应指标后得到

$$\Delta t_R = t_{\text{dry}}^{(4)} - t_{\text{dry}}^{(4,0)} = -78.7189 \text{ h}, \quad \eta_R = -60.6480\%. \quad (56)$$

该式是本文有效模型内部的反事实比较：除半径边界外，物性、空气边界和离散参数均保持相同。它用于量化模型中的几何边界影响，不等同于由实验识别出的真实收缩独立因果效应。

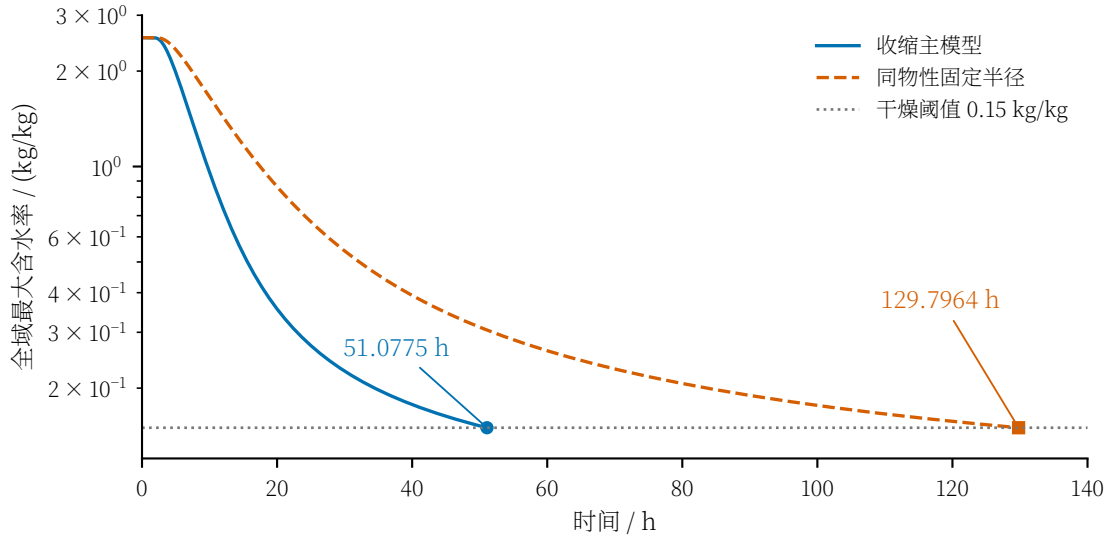


图 16 同附录四物性下收缩模型与固定半径对照的阈值跨越；纵轴采用对数刻度

5.4.5 结构敏感性检验

为避免只检验单侧扰动，取 $a_* = -0.2, -0.1, 0.1, 0.2$ 并分别重新求解完整模型。对应干燥时长为 50.9222 h、50.9992 h、51.1572 h 和 51.2381 h，相对主模型分别变化 -0.3040%、-0.1534%、0.1561% 和 0.3143%。图 17 展示正向扰动的代表性结果；左幅给出主模型终点时刻的假设映射偏离 $r/R - \xi$ ，两端均为零，表示各映射共享中心和实测外半径，这不是内部位移的观测结果。

记参数为 a_* 时的干燥时长为 $t_{\text{dry}}(a_*)$ 。内部映射的相对时长变化和最大结构偏差定义为

$$\eta_a(a_*) = \frac{t_{\text{dry}}(a_*) - t_{\text{dry}}(0)}{t_{\text{dry}}(0)} \times 100\%, \quad E_{\text{shape}} = \max_{a_* \in \{-0.2, -0.1, 0.1, 0.2\}} |\eta_a(a_*)|. \quad (57)$$

其中 $a_* = 0$ 对应相似收缩主映射。四组双侧扰动给出 $E_{\text{shape}} = 0.3143\%$ 。由于这些 a_* 取值并非由内部位移数据标定，该指标只表示所设映射族和扰动区间内的稳定性，不是所有收缩机制的误差上界。

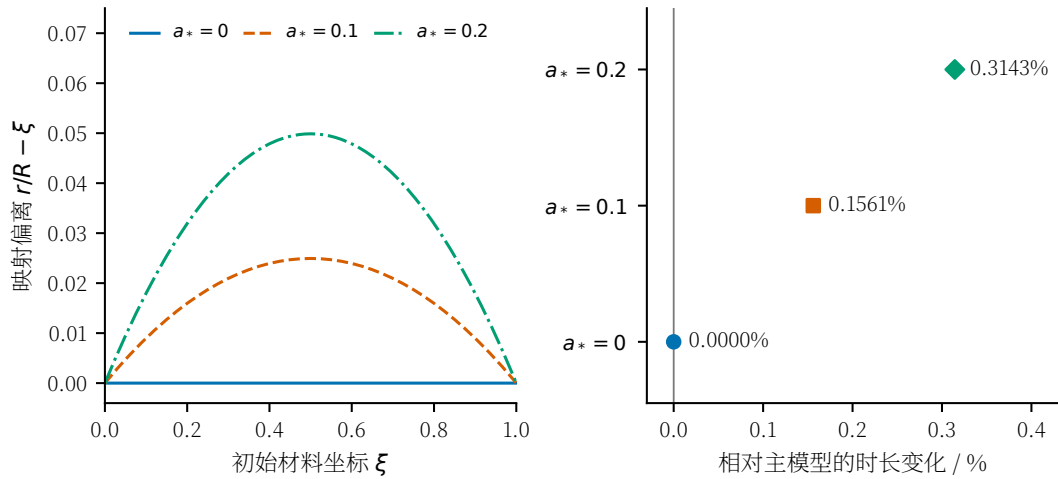


图 17 内部映射正向扰动的代表性结果；左图取 $t = 51.0775$ h，右图为未标定情景的计算结果，双侧扰动数值见正文

5.4.6 换热与传质系数敏感性

以 $h = 25 \text{ W}/(\text{m}^2 \text{ K})$ 、 $h_m = 8.0 \times 10^{-7} \text{ m/s}$ 为基准，每次只将一个系数上下扰动 20%，其余参数、空气边界、半径边界和终点判据均保持不变，并重新求解至终点。对 $p \in \{h, h_m\}$ ，定义

$$\eta_p(\lambda) = \frac{t_{\text{dry}}(p = \lambda p_0) - t_{\text{dry}}(p_0)}{t_{\text{dry}}(p_0)} \times 100\%, \quad \lambda \in \{0.8, 1.2\}. \quad (58)$$

表 8 问题四终点时长对 h 和 h_m 的单因素敏感性

情景	参数取值	终点时长/h	相对变化/%
基准	见正文基准值	51.0775	0.0000
$0.8h$	$20 \text{ W}/(\text{m}^2 \text{ K})$	51.0956	0.0353
$1.2h$	$30 \text{ W}/(\text{m}^2 \text{ K})$	51.0656	-0.0234
$0.8h_m$	$6.4 \times 10^{-7} \text{ m/s}$	51.7647	1.3454
$1.2h_m$	$9.6 \times 10^{-7} \text{ m/s}$	50.6839	-0.7706

由表 8，在所设 $\pm 20\%$ 区间内， h 与 h_m 引起的终点最大绝对相对变化分别为 $E_h = 0.0353\%$ 和 $E_{h_m} = 1.3454\%$ 。增大两类系数均使终点提前；在所检验的两类边界系数扰动中，终点对 h_m 的响应更明显，但这不足以证明全部干燥过程只受传质控制。正负扰动的响应不完全对称，来自温湿相关扩散系数与全域阈值判定的非线性；这些结果是给定区间内的确定性情景响应，不是参数置信区间。

5.4.7 空气边界检验

稳定段均值延拓仍含边界取值误差。计算分别把稳定段温度和空气水分浓度向下、向上移动一个总体标准差，并以附件末条记录替代两项均值，共形成五个边界情景。设

情景 s 的干燥时长为 $t_{\text{dry}}(s)$ ，相对主模型的变化及最大绝对变化定义为

$$\eta_{\text{air}}(s) = \frac{t_{\text{dry}}(s) - t_{\text{dry}}(0)}{t_{\text{dry}}(0)} \times 100\%, \quad (59)$$

$$E_{\text{air}} = \max_s |\eta_{\text{air}}(s)|. \quad (60)$$

温度下移一个总体标准差时，时长相对主模型变化 0.4840%。温度上移相同幅度时，对应变化为 -0.4813%。

空气水分浓度下移和上移一个总体标准差时，时长相对变化分别为 -0.0071% 和 0.0071%。末条记录替代情景使时长从主模型的 183 879 s 变为 182 949 s，即 50.8192 h。代入式 60 得到相对变化 -0.5058%，因此 $E_{\text{air}} = 0.5058\%$ 。

指标 E_{air} 检验长期空气边界的取值，指标 E_{shape} 检验同一外半径下的内部收缩映射， E_h 与 E_{h_m} 则检验表面传递系数。各指标采用相同主模型时长作比较基准，但扰动对象和情景集合不同。

5.4.8 数值精度检验

空间网格增至六百四十个区间后，终点相差 13 s，相对差为 0.007069%，共有时空点上的含水率最大差为 $1.715844 \times 10^{-5} \text{ kg/kg}$ 。时间步减至 0.5 s 后，终点相差 0.5 s，该含水率最大差为 $8.575106 \times 10^{-8} \text{ kg/kg}$ 。两类加密误差均低于四位小数结果的半单位门槛 $5 \times 10^{-5} \text{ kg/kg}$ 。

令 $\mathcal{S}$ 表示两组结果共有的表格时刻和有效物理位置，网格误差与时间步误差取

$$E_C^N = \max_{(r,t) \in \mathcal{S}} |C_{320}(r,t) - C_{640}(r,t)|, \quad (61)$$

$$E_C^{\Delta t} = \max_{(r,t) \in \mathcal{S}} |C_{1s}(r,t) - C_{0.5s}(r,t)|, \quad (62)$$

$$E_t = \frac{|t_{\text{dry}}^{\text{coarse}} - t_{\text{dry}}^{\text{fine}}|}{t_{\text{dry}}^{\text{fine}}} \times 100\%. \quad (63)$$

该定义排除已经离开收缩域的固定位置，不把结构空值计作数值误差。

两类加密试验的统一判定量为

$$E_C^{\max} = \max \{E_C^N, E_C^{\Delta t}\} = 1.715844 \times 10^{-5} < 5 \times 10^{-5} \text{ kg/kg}. \quad (64)$$

该不等式说明表 7 保留四位小数时，网格和时间步误差均未越过预定门槛。

水分和热量的离散平衡残差分别写为

$$\mathcal{B}_C^n = \sum_i w_i^n \delta_t C_i^n + R^n h_m (C_N^n - C_a^n), \quad (65)$$

$$\mathcal{B}_T^n = \sum_i \rho_i^n c_{p,i}^n w_i^n \delta_t T_i^n + R^n h (T_N^n - T_a^n). \quad (66)$$

程序按储存项和边界项的量级归一化 B_C^n ，并用各热量项绝对量之和归一化 B_T^n 。线性方程组的相对残差采用

$$\varepsilon_{\text{lin},u} = \max_n \frac{\|A_u^n u^n - b_u^n\|_\infty}{\|b_u^n\|_\infty}. \quad (67)$$

水分方程的离散平衡相对残差为 1.7883×10^{-11} ，热方程归一残差为 9.7424×10^{-16} 。热量和水分线性系统的最大相对残差分别为 3.5831×10^{-14} 和 1.2016×10^{-15} 。该组残差检验有效扩散方程的离散一致性，不表示真实干固体和水分总质量守恒。

六、 模型评价

6.1 结构特点

一维径向控制体直接对应题目要求的空间坐标，环形体积和控制面面积保留了圆柱几何特征。中心控制体的内表面积为零，无需在 $r = 0$ 处直接计算 $1/r$ 。相邻控制体的共享通量在离散求和时相消，从而满足所建有效场方程的局部离散通量平衡。

变物性模型把温度和含水率通过扩散系数及热物性联系起来，能够从预热阶段连续推进到恒温阶段。移动坐标把收缩域转化为固定单位区间，避免反复增删网格，并使问题三和问题四采用同一终点判据。

题面数据直接构成时间边界和半径边界，模型没有为缺失的实验量拟合额外参数。分段线性插值保持原始记录值，结果文件所需的时间和空间分辨率可以由数值网格直接输出。

6.2 适用边界

径向模型忽略端面传递，面积比只能说明端部贡献受限，不能证明所有轴向位置都满足相同场分布。一维模型和二维模型可能给出不同的局部场分布 [8]。靠近端面的温度和含水率需用二维模型抽样或端部测温数据评估偏差。

题面没有提供汽化潜热和局部相变速率，Hussain 和 Dincer 的参考方程也未显式列出潜热源项，因此本文没有在温度方程中另设相变源项 [4]。这一取舍来自参数缺失和参考模型口径，不能据此判断潜热影响的大小。Da Silva 等人在圆柱形香蕉的 58.6°C 干燥数值实验中比较了潜热设置。忽略汽化潜热后，卡方统计量由 28.9 增至 4146.1，决定系数由 0.98843 降至 0.75502。采用自由水的汽化潜热后，两项指标分别为 29.6 和 0.98832[9]。香蕉实验的材料、温度和边界条件不同，上述指标不能直接换算为本题药材的误差。该对照说明潜热设置会改变同类耦合干燥模型的拟合结果。本文给出的温度、扩散系数和干燥时长均以无显式潜热源项为条件，不把有效水分扩散系数解释为对汽化能耗的补偿。

题面把药材干基含水率和空气水分浓度统一记为 kg/kg ，却没有分别说明二者的基准物质。直接作差的传质边界只适用于题设经验口径，不能外推为一般的湿空气平衡关系。

问题三的长时空气边界采用稳定段均值,其结论以烘房在实测结束后继续维持同一平台工况为前提。总体标准差扰动和末条记录替代使终点时长变化不超过 0.5117%;这些确定性情景只反映所设范围内的响应,不构成统计置信区间。将换热系数和传质系数同时放大 8% 的集总应力情景使时长缩短 0.8372%,但不能作为真实二维端面效应的误差上界。问题四按式 60 检验五个空气边界情景,最大绝对时长变化为 $E_{\text{air}} = 0.5058\%$,对应附件末条记录替代情景。按式 58 对 h 和 h_m 分别作 $\pm 20\%$ 单因素扰动时,最大绝对时长变化为 0.0353% 和 1.3454%,仅适用于该扰动范围。主模型在 51.0775 h 结束,早于半径观测终点,因而没有调用末值保持规则。问题四只观测外表面半径,内部映射参数没有实验标定。 $a_* = \pm 0.1, \pm 0.2$ 四组结构扰动给出 $E_{\text{shape}} = 0.3143\%$,该数值只适用于式 44 的映射族。缺少内部位移和热质通量观测时,现有数据不能辨识局部孔隙演化或多相传递参数,因此该指标不是所有收缩机制的不确定性上界。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具,主要用于程序调试、图表制作、论文结构起草、语言校核及排版,详细使用情况见支撑材料。

参考文献

- [1] SOKHANSANJ S, BAILEY W G, VAN DALFSEN K B. Drying of North American ginseng roots (*Panax quinquefolius* L.)[J]. *Drying Technology*, 1999, 17(10): 2293–2308. DOI: 10.1080/07373939908917684.
- [2] 刘格含, 王鹏, 吴小华, 等. 农产品热风干燥传热传质数值模拟研究进展 [J]. *食品工业科技*, 2020, 41(22): 342–350, 357. DOI: 10.13386/j.issn1002-0306.2020010245.
- [3] TZEMPELIKOS D A, MITRAKOS D, VOUIROS A P, et al. Numerical modeling of heat and mass transfer during convective drying of cylindrical quince slices[J]. *Journal of Food Engineering*, 2015, 156: 10–21. DOI: 10.1016/j.jfoodeng.2015.01.017.
- [4] HUSSAIN M M, DINCER I. Two-dimensional heat and moisture transfer analysis of a cylindrical moist object subjected to drying: A finite-difference approach[J]. *International Journal of Heat and Mass Transfer*, 2003, 46(21): 4033–4039. DOI: 10.1016/S0017-9310(03)00229-1.
- [5] EYMARD R, GALLOUËT T, HERBIN R. Finite volume methods[M]//CIARLET P G, LIONS J L. *Handbook of Numerical Analysis*. Amsterdam: Elsevier, 2000, 7: 713–1018. DOI: 10.1016/S1570-8659(00)07005-8.
- [6] QUARTERONI A, SACCO R, SALERI F. *Numerical Mathematics*[M]. 2nd ed. Berlin, Heidelberg: Springer, 2007. DOI: 10.1007/b98885.

- [7] ADROVER A, BRASIELLO A, PONSO G. A moving boundary model for food isothermal drying and shrinkage: General setting[J]. Journal of Food Engineering, 2019, 244: 178–191. DOI: 10.1016/j.jfoodeng.2018.09.018.
- [8] ADROVER A, BRASIELLO A. A moving boundary model for food isothermal drying and shrinkage: One-dimensional versus two-dimensional approaches[J]. Journal of Food Process Engineering, 2019, 42(6): e13178. DOI: 10.1111/jfpe.13178.
- [9] DA SILVA W P, E SILVA C M D P S, GAMA F J A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. Journal of Food Engineering, 2014, 141: 65–73. DOI: 10.1016/j.jfoodeng.2014.05.010.

A 支撑材料清单

支撑材料压缩包为 支撑材料.zip，解压后的文件均不含参赛者身份信息，主要内容见表 9。

表 9 支撑材料清单

文件或目录	内容说明
README.txt	运行环境、目录结构、复现顺序和结果核对说明
requirements.txt	Python 依赖及建议版本
药材烘干求解.py	问题一径向有限体积求解、结果表生成与冒烟测试
全程干燥求解.py	问题二、问题三的变物性全程模型及对照检验
问题 4_ 求解.py、问题 4_ 复现.py	问题四移动边界求解、敏感性分析与一键复现
问题 1_ 绘图.py-问题 4_ 绘图.py	四个问题的论文图件生成程序
论文图件优化.py、问题 4_ 素材整理.py	重点图件重绘与问题四结果材料整理
utils/、figure_tools/	本项目内置的绘图样式、导出与版面检查辅助模块
repro_manifest.py	复现清单与文件哈希生成程序
problem A/附件/	为保证离线复现而随包保留的赛题原始附件
results/	四问结果工作簿、关键结果、敏感性与收敛性数据、复现清单
figures/	正文实际引用的 17 幅图件（16 幅 PDF 与 1 幅 PNG）
AI 工具使用详情.pdf、AI 使用报告.docx	按 2026 年试行规定整理的 AI 工具使用详情及其可编辑审核稿，提交前由参赛队逐项复核
SHA256SUMS.txt	支撑材料各文件的 SHA-256 校验值

B 计算程序

以下程序与支撑材料中的同名文件一致。完整运行顺序、参数和输出位置见支撑材料中的 README.txt。

2.1 问题一求解程序

```
1 from __future__ import annotations
2
```

```

3 import argparse
4 import hashlib
5 import json
6 import platform
7 from dataclasses import dataclass
8 from pathlib import Path
9
10 import numpy as np
11 import openpyxl
12
13
14 PROJECT_ROOT = Path(__file__).resolve().parent
15 AIR_PATH = PROJECT_ROOT / "problem A" / "附件" / "附件1.xlsx"
16 RADIUS_PATH = PROJECT_ROOT / "problem A" / "附件" / "附件2.xlsx"
17 TEMPLATE_DIR = PROJECT_ROOT / "problem A" / "附件" / "附件3"
18 RESULTS_DIR = PROJECT_ROOT / "results"
19
20
21 @dataclass(frozen=True)
22 class AirBoundary:
23     time_s: np.ndarray
24     temperature_c: np.ndarray
25     moisture: np.ndarray
26
27     def at(self, time_s: float) -> tuple[float, float]:
28         temperature = float(
29             np.interp(
30                 time_s,
31                 self.time_s,
32                 self.temperature_c,
33                 left=self.temperature_c[0],
34                 right=self.temperature_c[-1],
35             )
36         )
37         moisture = float(
38             np.interp(
39                 time_s,
40                 self.time_s,
41                 self.moisture,
42                 left=self.moisture[0],
43                 right=self.moisture[-1],
44             )
45         )
46         return temperature, moisture
47
48
49 @dataclass(frozen=True)
50 class RadialGrid:

```

```

51     radius_m: float
52     length_m: float
53     nodes_m: np.ndarray
54     volumes_m3: np.ndarray
55     internal_areas_m2: np.ndarray
56     surface_area_m2: float
57     spacing_m: float
58
59
60 @dataclass
61 class SimulationResult:
62     time_s: np.ndarray
63     radius_m: np.ndarray
64     temperature_c: np.ndarray
65     moisture: np.ndarray
66     max_picard_iterations: int
67     moisture_balance_relative_error: float
68     heat_matrix_condition_number: float
69     moisture_matrix_condition_number: float
70
71
72 def sha256_file(path: Path) -> str:
73     digest = hashlib.sha256()
74     with path.open("rb") as stream:
75         for block in iter(lambda: stream.read(1024 * 1024), b''):
76             digest.update(block)
77     return digest.hexdigest()
78
79
80 def read_air_boundary(path: Path = AIR_PATH) -> AirBoundary:
81     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
82     worksheet = workbook.active
83     rows = list(worksheet.iter_rows(values_only=True))
84     workbook.close()
85     if len(rows) != 242:
86         raise ValueError(f"附件1有效行数应为242，实际为{len(rows)}")
87     header = tuple(rows[0])
88     if header != ("时间", "温度", "水分浓度"):
89         raise ValueError(f"附件1表头异常: {header}")
90     data = np.asarray(rows[1:], dtype=float)
91     if data.shape != (241, 3):
92         raise ValueError(f"附件1数据形状异常: {data.shape}")
93     if data[0, 0] != 0 or data[-1, 0] != 14400:
94         raise ValueError("附件1时间范围不是0至14400秒")
95     if not np.allclose(np.diff(data[:, 0]), 60.0):
96         raise ValueError("附件1时间间隔不是60秒")
97     if not np.all(np.isfinite(data)):
98         raise ValueError("附件1含有非有限数")

```

```

99     return AirBoundary(data[:, 0], data[:, 1], data[:, 2])
100
101
102 def make_grid(radius_m: float, intervals: int, length_m: float = 0.25) -> RadialGrid:
103     if intervals < 2:
104         raise ValueError("径向小区间数至少为2")
105     nodes = np.linspace(0.0, radius_m, intervals + 1)
106     spacing = radius_m / intervals
107     faces = np.empty(intervals + 2, dtype=float)
108     faces[0] = 0.0
109     faces[-1] = radius_m
110     faces[1:-1] = 0.5 * (nodes[:-1] + nodes[1:])
111     volumes = np.pi * length_m * (faces[1:] ** 2 - faces[:-1] ** 2)
112     internal_areas = 2.0 * np.pi * length_m * faces[1:-1]
113     surface_area = 2.0 * np.pi * radius_m * length_m
114     return RadialGrid(
115         radius_m=radius_m,
116         length_m=length_m,
117         nodes_m=nodes,
118         volumes_m3=volumes,
119         internal_areas_m2=internal_areas,
120         surface_area_m2=surface_area,
121         spacing_m=spacing,
122     )
123
124
125 def solve_tridiagonal(
126     lower: np.ndarray,
127     diagonal: np.ndarray,
128     upper: np.ndarray,
129     right_hand_side: np.ndarray,
130 ) -> np.ndarray:
131     n = diagonal.size
132     c_prime = np.empty(n - 1, dtype=float)
133     d_prime = np.empty(n, dtype=float)
134     pivot = diagonal[0]
135     if abs(pivot) < 1.0e-30:
136         raise np.linalg.LinAlgError("三对角矩阵首个主元接近零")
137     c_prime[0] = upper[0] / pivot
138     d_prime[0] = right_hand_side[0] / pivot
139     for i in range(1, n):
140         pivot = diagonal[i] - lower[i - 1] * c_prime[i - 1]
141         if abs(pivot) < 1.0e-30:
142             raise np.linalg.LinAlgError(f"三对角矩阵第{i}个主元接近零")
143         if i < n - 1:
144             c_prime[i] = upper[i] / pivot
145         d_prime[i] = (right_hand_side[i] - lower[i - 1] * d_prime[i - 1]) / pivot
146     solution = np.empty(n, dtype=float)

```

```

147     solution[-1] = d_prime[-1]
148     for i in range(n - 2, -1, -1):
149         solution[i] = d_prime[i] - c_prime[i] * solution[i + 1]
150     return solution
151
152
153 def assemble_implicit_system(
154     grid: RadialGrid,
155     old_state: np.ndarray,
156     storage_coefficient: np.ndarray,
157     diffusion_coefficient: np.ndarray,
158     time_step_s: float,
159     boundary_coefficient: float,
160     ambient_value: float,
161 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
162     storage = storage_coefficient * grid.volumes_m3 / time_step_s
163     face_diffusion = 0.5 * (
164         diffusion_coefficient[:-1] + diffusion_coefficient[1:]
165     )
166     conductance = face_diffusion * grid.internal_areas_m2 / grid.spacing_m
167     diagonal = storage.copy()
168     diagonal[:-1] += conductance
169     diagonal[1:] += conductance
170     diagonal[-1] += boundary_coefficient * grid.surface_area_m2
171     lower = -conductance.copy()
172     upper = -conductance.copy()
173     right_hand_side = storage * old_state
174     right_hand_side[-1] += (
175         boundary_coefficient * grid.surface_area_m2 * ambient_value
176     )
177     return lower, diagonal, upper, right_hand_side
178
179
180 def dense_from_tridiagonal(
181     lower: np.ndarray, diagonal: np.ndarray, upper: np.ndarray
182 ) -> np.ndarray:
183     matrix = np.diag(diagonal)
184     matrix += np.diag(lower, k=-1)
185     matrix += np.diag(upper, k=1)
186     return matrix
187
188
189 def ql_diffusivity(moisture: np.ndarray) -> np.ndarray:
190     safe_moisture = np.maximum(moisture, 1.0e-8)
191     return 7.0e-9 * np.exp(-0.89 / safe_moisture)
192
193
194 def simulate_ql(

```

```

195     end_time_s: float,
196     time_step_s: float = 1.0,
197     intervals: int = 20,
198     picard_tolerance: float = 1.0e-10,
199     picard_max_iterations: int = 50,
200 ) -> SimulationResult:
201     steps_float = end_time_s / time_step_s
202     steps = int(round(steps_float))
203     if not np.isclose(steps, steps_float):
204         raise ValueError("结束时刻必须是时间步长的整数倍")
205     air = read_air_boundary()
206     grid = make_grid(radius_m=0.02, intervals=intervals)
207     node_count = intervals + 1
208     time_axis = np.linspace(0.0, end_time_s, steps + 1)
209     temperature_history = np.empty((steps + 1, node_count), dtype=float)
210     moisture_history = np.empty((steps + 1, node_count), dtype=float)
211     temperature = np.full(node_count, 28.0, dtype=float)
212     moisture = np.full(node_count, 2.55, dtype=float)
213     temperature_history[0] = temperature
214     moisture_history[0] = moisture
215
216     density = 820.0
217     heat_capacity = 2600.0
218     conductivity = 0.36
219     heat_transfer_coefficient = 25.0
220     mass_transfer_coefficient = 8.0e-7
221     heat_storage = np.full(node_count, density * heat_capacity, dtype=float)
222     heat_diffusion = np.full(node_count, conductivity, dtype=float)
223     heat_lower, heat_diagonal, heat_upper, _ = assemble_implicit_system(
224         grid,
225         temperature,
226         heat_storage,
227         heat_diffusion,
228         time_step_s,
229         heat_transfer_coefficient,
230         28.0,
231     )
232     heat_condition = float(
233         np.linalg.cond(
234             dense_from_tridiagonal(heat_lower, heat_diagonal, heat_upper)
235         )
236     )
237
238     initial_inventory = float(np.dot(grid.volumes_m3, moisture))
239     cumulative_surface_outflow = 0.0
240     max_picard_iterations_used = 0
241     moisture_condition = float("nan")
242

```

```

243 for step in range(1, steps + 1):
244     current_time = step * time_step_s
245     ambient_temperature, ambient_moisture = air.at(current_time)
246     _, _, heat_rhs = assemble_implicit_system(
247         grid,
248         temperature,
249         heat_storage,
250         heat_diffusion,
251         time_step_s,
252         heat_transfer_coefficient,
253         ambient_temperature,
254     )
255     temperature = solve_tridiagonal(
256         heat_lower, heat_diagonal, heat_upper, heat_rhs
257     )
258
259     moisture_guess = moisture.copy()
260     converged = False
261     for iteration in range(1, picard_max_iterations + 1):
262         diffusivity = ql_diffusivity(moisture_guess)
263         lower, diagonal, upper, rhs = assemble_implicit_system(
264             grid,
265             moisture,
266             np.ones(node_count, dtype=float),
267             diffusivity,
268             time_step_s,
269             mass_transfer_coefficient,
270             ambient_moisture,
271         )
272         moisture_new = solve_tridiagonal(lower, diagonal, upper, rhs)
273         update_norm = float(np.max(np.abs(moisture_new - moisture_guess)))
274         moisture_guess = moisture_new
275         if update_norm <= picard_tolerance:
276             converged = True
277             break
278     if not converged:
279         raise RuntimeError(
280             f"问题一-水分Picard迭代在t={current_time:g}秒未收敛"
281         )
282     if step == 1:
283         moisture_condition = float(
284             np.linalg.cond(dense_from_tridiagonal(lower, diagonal, upper))
285         )
286     max_picard_iterations_used = max(max_picard_iterations_used, iteration)
287     moisture = moisture_guess
288     cumulative_surface_outflow += (
289         time_step_s
290         * mass_transfer_coefficient

```

```

291         * grid.surface_area_m2
292         * (moisture[-1] - ambient_moisture)
293     )
294     temperature_history[step] = temperature
295     moisture_history[step] = moisture
296
297     final_inventory = float(np.dot(grid.volumes_m3, moisture))
298     balance_residual = (
299         final_inventory - initial_inventory + cumulative_surface_outflow
300     )
301     relative_balance_error = abs(balance_residual) / max(
302         abs(initial_inventory - final_inventory), 1.0e-30
303     )
304     return SimulationResult(
305         time_s=time_axis,
306         radius_m=grid.nodes_m,
307         temperature_c=temperature_history,
308         moisture=moisture_history,
309         max_picard_iterations=max_picard_iterations_used,
310         moisture_balance_relative_error=float(relative_balance_error),
311         heat_matrix_condition_number=heat_condition,
312         moisture_matrix_condition_number=moisture_condition,
313     )
314
315
316 def sample_field(
317     result: SimulationResult,
318     field: np.ndarray,
319     sample_times_s: list[float],
320     sample_radii_cm: list[float],
321 ) -> np.ndarray:
322     sampled = np.empty((len(sample_times_s), len(sample_radii_cm)), dtype=float)
323     radius_cm = 100.0 * result.radius_m
324     for i, sample_time in enumerate(sample_times_s):
325         time_index = int(np.argmin(np.abs(result.time_s - sample_time)))
326         if not np.isclose(result.time_s[time_index], sample_time):
327             raise ValueError(f"计算结果不含要求时刻{sample_time}秒")
328         sampled[i] = np.interp(sample_radii_cm, radius_cm, field[time_index])
329     return sampled
330
331
332 def write_q1_workbook(result: SimulationResult, output_path: Path) -> None:
333     template_path = TEMPLATE_DIR / "result1.xlsx"
334     workbook = openpyxl.load_workbook(template_path)
335     radii_cm = [round(0.1 * i, 1) for i in range(21)]
336     output_times_s = np.arange(1.0, 1801.0, 1.0)
337     time_indices = np.searchsorted(result.time_s, output_times_s)
338     if not np.allclose(result.time_s[time_indices], output_times_s):

```

```

339     raise ValueError("内部时间网格未包含result1要求的整数秒时刻")
340 for sheet_name, field in (
341     ("温度", result.temperature_c),
342     ("水分浓度", result.moisture),
343 ):
344     worksheet = workbook[sheet_name]
345     if worksheet.max_row > 1:
346         worksheet.delete_rows(2, worksheet.max_row - 1)
347     worksheet.cell(1, 1, "时间\\到药材中心的距离")
348     for column, radius in enumerate(radii_cm, start=2):
349         worksheet.cell(1, column, radius)
350     for row, (time_value, time_index) in enumerate(
351         zip(output_times_s, time_indices, strict=True), start=2
352     ):
353         worksheet.cell(row, 1, int(round(float(time_value))))
354         values = np.interp(radii_cm, 100.0 * result.radius_m, field[time_index])
355         for column, value in enumerate(values, start=2):
356             worksheet.cell(row, column, round(float(value), 4))
357     worksheet.freeze_panes = "B2"
358 output_path.parent.mkdir(parents=True, exist_ok=True)
359 temporary_path = output_path.with_suffix(".tmp.xlsx")
360 workbook.save(temporary_path)
361 workbook.close()
362 temporary_path.replace(output_path)
363
364
365 def write_key_table_csv(
366     path: Path,
367     sample_times_s: list[float],
368     sample_radii_cm: list[float],
369     temperature: np.ndarray,
370     moisture: np.ndarray,
371 ) -> None:
372     lines = ["变量,时间_s," + ",".join(f"r_{r:g}_cm" for r in sample_radii_cm)]
373     for name, values in (("温度_C", temperature), ("水分浓度_kg_per_kg", moisture)):
374         for time_value, row in zip(sample_times_s, values, strict=True):
375             lines.append(
376                 f"{name}, {time_value:g}," + ",".join(f"{value:.8f}" for value in row)
377             )
378     path.write_text("\n".join(lines) + "\n", encoding="utf-8-sig")
379
380
381 def run_smoke() -> None:
382     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
383     result = simulate_q1(end_time_s=120.0, time_step_s=1.0, intervals=20)
384     ambient_120 = read_air_boundary().at(120.0)
385     report = {
386         "mode": "P1_q1_smoke",

```

```

387     "input": {
388         "path": str(AIR_PATH.relative_to(PROJECT_ROOT)),
389         "sha256": sha256_file(AIR_PATH),
390         "rows": 241,
391         "first_time_s": 0,
392         "last_time_s": 14400,
393     },
394     "parameters": {
395         "end_time_s": 120.0,
396         "time_step_s": 1.0,
397         "radial_spacing_m": 0.001,
398         "radial_intervals": 20,
399         "initial_temperature_c": 28.0,
400         "initial_moisture_kg_per_kg": 2.55,
401         "diffusivity_formula": "7e-9*exp(-0.89/C)",
402         "picard_tolerance": 1.0e-10,
403         "picard_max_iterations": 50,
404     },
405     "boundary_at_120_s": {
406         "temperature_c": ambient_120[0],
407         "moisture_kg_per_kg": ambient_120[1],
408     },
409     "result_at_120_s": {
410         "temperature_center_c": float(result.temperature_c[-1, 0]),
411         "temperature_surface_c": float(result.temperature_c[-1, -1]),
412         "moisture_center_kg_per_kg": float(result.moisture[-1, 0]),
413         "moisture_surface_kg_per_kg": float(result.moisture[-1, -1]),
414     },
415     "checks": {
416         "temperature_min_c": float(np.min(result.temperature_c)),
417         "temperature_max_c": float(np.max(result.temperature_c)),
418         "moisture_min_kg_per_kg": float(np.min(result.moisture)),
419         "moisture_max_kg_per_kg": float(np.max(result.moisture)),
420         "moisture_balance_relative_error": result.moisture_balance_relative_error,
421         "heat_matrix_condition_number": result.heat_matrix_condition_number,
422         "moisture_matrix_condition_number": result.moisture_matrix_condition_number,
423         "max_picard_iterations": result.max_picard_iterations,
424         "finite": bool(
425             np.all(np.isfinite(result.temperature_c))
426             and np.all(np.isfinite(result.moisture))
427         ),
428     },
429 }
430 output_path = RESULTS_DIR / "P1_q1_smoke.json"
431 output_path.write_text(
432     json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
433 )
434 print(json.dumps(report, ensure_ascii=False, indent=2))

```

```

435
436
437 def run_ql() -> None:
438     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
439     authoritative = simulate_ql(
440         end_time_s=1800.0, time_step_s=0.25, intervals=160
441     )
442     time_refined = simulate_ql(
443         end_time_s=1800.0, time_step_s=0.125, intervals=160
444     )
445     space_refined = simulate_ql(
446         end_time_s=1800.0, time_step_s=0.25, intervals=320
447     )
448     sample_times = [100.0, 300.0, 600.0, 900.0, 1200.0, 1500.0, 1800.0]
449     sample_radii = [0.0, 0.5, 1.0, 1.5, 2.0]
450     authoritative_temperature = sample_field(
451         authoritative, authoritative.temperature_c, sample_times, sample_radii
452     )
453     authoritative_moisture = sample_field(
454         authoritative, authoritative.moisture, sample_times, sample_radii
455     )
456     time_refined_temperature = sample_field(
457         time_refined, time_refined.temperature_c, sample_times, sample_radii
458     )
459     time_refined_moisture = sample_field(
460         time_refined, time_refined.moisture, sample_times, sample_radii
461     )
462     space_refined_temperature = sample_field(
463         space_refined, space_refined.temperature_c, sample_times, sample_radii
464     )
465     space_refined_moisture = sample_field(
466         space_refined, space_refined.moisture, sample_times, sample_radii
467     )
468     time_temperature_difference = np.abs(
469         authoritative_temperature - time_refined_temperature
470     )
471     time_moisture_difference = np.abs(
472         authoritative_moisture - time_refined_moisture
473     )
474     space_temperature_difference = np.abs(
475         authoritative_temperature - space_refined_temperature
476     )
477     space_moisture_difference = np.abs(
478         authoritative_moisture - space_refined_moisture
479     )
480
481     result_workbook = RESULTS_DIR / "result1.xlsx"
482     write_ql_workbook(authoritative, result_workbook)

```

```

483 write_key_table_csv(
484     RESULTS_DIR / "问题1_关键结果.csv",
485     sample_times,
486     sample_radii,
487     authoritative_temperature,
488     authoritative_moisture,
489 )
490 convergence_lines = [
491     "检验, 变量, 最大绝对差, 平均绝对差, 权威空间步长_m, 权威时间步长_s, 加密空间步长_m, 加密时间步长_s",
492     f"时间步减半, 温度_C, {time_temperature_difference.max():.12g}, {time_temperature_difference.mean():.12g}, 0.000125, 0.25, 0.000125, 0.125",
493     f"时间步减半, 水分浓度_kg_per_kg, {time_moisture_difference.max():.12g}, {time_moisture_difference.mean():.12g}, 0.000125, 0.25, 0.000125, 0.125",
494     f"空间步减半, 温度_C, {space_temperature_difference.max():.12g}, {space_temperature_difference.mean():.12g}, 0.000125, 0.25, 0.0000625, 0.25",
495     f"空间步减半, 水分浓度_kg_per_kg, {space_moisture_difference.max():.12g}, {space_moisture_difference.mean():.12g}, 0.000125, 0.25, 0.0000625, 0.25",
496 ]
497 (RESULTS_DIR / "问题1_网格收敛.csv").write_text(
498     "\n".join(convergence_lines) + "\n", encoding="utf-8-sig"
499 )
500
501 summary = {
502     "mode": "q1_full",
503     "input_sha256": {str(AIR_PATH.relative_to(PROJECT_ROOT)): sha256_file(AIR_PATH)},
504     "parameters": {
505         "authoritative": {
506             "radial_spacing_m": 0.000125,
507             "time_step_s": 0.25,
508         },
509         "time_refined": {
510             "radial_spacing_m": 0.000125,
511             "time_step_s": 0.125,
512         },
513         "space_refined": {
514             "radial_spacing_m": 0.0000625,
515             "time_step_s": 0.25,
516         },
517         "diffusivity_formula": "7e-9*exp(-0.89/C)",
518         "picard_tolerance": 1.0e-10,
519     },
520     "requested_times_s": sample_times,
521     "requested_radii_cm": sample_radii,
522     "temperature_c": np.round(authoritative_temperature, 8).tolist(),
523     "moisture_kg_per_kg": np.round(authoritative_moisture, 8).tolist(),
524     "verification": {
525         "max_temperature_difference_time_refined_c": float(

```

```

526         time_temperature_difference.max()
527     ),
528     "max_moisture_difference_time_refined_kg_per_kg": float(
529         time_moisture_difference.max()
530     ),
531     "max_temperature_difference_space_refined_c": float(
532         space_temperature_difference.max()
533     ),
534     "max_moisture_difference_space_refined_kg_per_kg": float(
535         space_moisture_difference.max()
536     ),
537     "authoritative_moisture_balance_relative_error": authoritative.
moisture_balance_relative_error,
538     "time_refined_moisture_balance_relative_error": time_refined.
moisture_balance_relative_error,
539     "space_refined_moisture_balance_relative_error": space_refined.
moisture_balance_relative_error,
540     "authoritative_max_picard_iterations": authoritative.max_picard_iterations,
541     "time_refined_max_picard_iterations": time_refined.max_picard_iterations,
542     "space_refined_max_picard_iterations": space_refined.max_picard_iterations,
543     "authoritative_heat_matrix_condition_number": authoritative.heat_matrix_condition_number
544 ,
545     "authoritative_moisture_matrix_condition_number": authoritative.
moisture_matrix_condition_number,
546 },
547 "outputs": {
548     "workbook": str(result_workbook.relative_to(PROJECT_ROOT)),
549     "key_table": "results/问题1_关键结果.csv",
550     "convergence": "results/问题1_网格收敛.csv",
551 },
552 "environment": {
553     "python": platform.python_version(),
554     "numpy": np.__version__,
555     "openpyxl": openpyxl.__version__,
556     "platform": platform.platform(),
557 },
558 }
559 (RESULTS_DIR / "问题1_结果摘要.json").write_text(
560     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
561 )
562 print(json.dumps(summary, ensure_ascii=False, indent=2))
563
564 def main() -> None:
565     parser = argparse.ArgumentParser(description="2026 CUMCM A题药材烘干数值求解")
566     parser.add_argument(
567         "--mode", choices=("smoke", "ql"), default="smoke", help="运行模式"
568     )

```

```

569     arguments = parser.parse_args()
570     if arguments.mode == "smoke":
571         run_smoke()
572     else:
573         run_q1()
574
575
576 if __name__ == "__main__":
577     main()

```

2.2 问题二与问题三求解程序

```

1  from __future__ import annotations
2
3  import argparse
4  import csv
5  import hashlib
6  import json
7  import platform
8  import shutil
9  import time
10 from dataclasses import dataclass
11 from pathlib import Path
12
13 import numpy as np
14 import openpyxl
15 from scipy.linalg import solve_banded
16
17 import 药材烘干求解 as core
18
19
20 PROJECT_ROOT = Path(__file__).resolve().parent
21 AIR_PATH = PROJECT_ROOT / "problem A" / "附件" / "附件1.xlsx"
22 RADIUS_PATH = PROJECT_ROOT / "problem A" / "附件" / "附件2.xlsx"
23 TEMPLATE_DIR = PROJECT_ROOT / "problem A" / "附件" / "附件3"
24 RESULTS_DIR = PROJECT_ROOT / "results"
25
26 HEAT_TRANSFER_COEFFICIENT = 25.0
27 MASS_TRANSFER_COEFFICIENT = 8.0e-7
28 INITIAL_TEMPERATURE_C = 28.0
29 INITIAL_MOISTURE = 2.55
30 DRYING_THRESHOLD = 0.15
31 STABLE_AIR_START_S = 9000.0
32 STABLE_AIR_END_S = 14400.0
33
34
35 @dataclass(frozen=True)

```

```

36 class RadiusHistory:
37     time_s: np.ndarray
38     radius_m: np.ndarray
39
40     def at(self, time_s: float) -> float:
41         return float(
42             np.interp(
43                 time_s,
44                 self.time_s,
45                 self.radius_m,
46                 left=self.radius_m[0],
47                 right=self.radius_m[-1],
48             )
49         )
50
51
52 @dataclass
53 class VariableSimulation:
54     time_s: np.ndarray
55     radius_m: np.ndarray
56     node_coordinate: np.ndarray
57     temperature_c: np.ndarray
58     moisture: np.ndarray
59     max_coupling_iterations: int
60     moisture_balance_relative_error: float
61     dry_time_s: float | None
62     coupling_iterations: np.ndarray
63     max_temperature_linear_residual: float
64     max_moisture_linear_residual: float
65
66
67 def sha256_file(path: Path) -> str:
68     digest = hashlib.sha256()
69     with path.open("rb") as stream:
70         for block in iter(lambda: stream.read(1024 * 1024), b''):
71             digest.update(block)
72     return digest.hexdigest()
73
74
75 def commit_temporary_file(temporary: Path, output_path: Path) -> None:
76     try:
77         temporary.replace(output_path)
78     except PermissionError:
79         shutil.copyfile(temporary, output_path)
80         if sha256_file(temporary) != sha256_file(output_path):
81             raise IOError(f"文件复制校验失败: {output_path}")
82     temporary.unlink()
83

```

```

84
85 def read_radius_history(path: Path = RADIUS_PATH) -> RadiusHistory:
86     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
87     worksheet = workbook.active
88     rows = list(worksheet.iter_rows(values_only=True))
89     workbook.close()
90     if len(rows) != 146:
91         raise ValueError(f"附件2有效行数应为146，实际为{len(rows)}")
92     if tuple(rows[0]) != ("时间", "半径"):
93         raise ValueError(f"附件2表头异常: {tuple(rows[0])}")
94     data = np.asarray(rows[1:], dtype=float)
95     if data.shape != (145, 2):
96         raise ValueError(f"附件2数据形状异常: {data.shape}")
97     if data[0, 0] != 0 or data[-1, 0] != 259200:
98         raise ValueError("附件2时间范围不是0至259200秒")
99     if not np.allclose(np.diff(data[:, 0]), 1800.0):
100         raise ValueError("附件2时间间隔不是1800秒")
101     radius_m = data[:, 1] / 100.0
102     if np.any(np.diff(radius_m) > 1.0e-12):
103         raise ValueError("附件2半径存在增大记录")
104     if not np.isclose(radius_m[0], 0.02) or not np.isclose(
105         radius_m[-1], 0.01198
106     ):
107         raise ValueError("附件2首末半径异常")
108     return RadiusHistory(data[:, 0], radius_m)
109
110
111 def q23_properties(
112     moisture: np.ndarray, temperature_c: np.ndarray
113 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
114     safe_moisture = np.maximum(moisture, 1.0e-8)
115     temperature_k = temperature_c + 273.15
116     density = 650.0 + 128.0 * safe_moisture
117     heat_capacity = 1450.0 + 2736.0 * safe_moisture / (safe_moisture + 1.0)
118     conductivity = 0.21 + 0.38 * safe_moisture / (safe_moisture + 1.0)
119     diffusivity = (
120         2.4e-3
121         * np.exp(-0.45 / safe_moisture)
122         * np.exp(-3850.0 / temperature_k)
123     )
124     return density, heat_capacity, conductivity, diffusivity
125
126
127 def q4_properties(
128     moisture: np.ndarray, temperature_c: np.ndarray
129 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
130     safe_moisture = np.maximum(moisture, 1.0e-8)
131     temperature_k = temperature_c + 273.15

```

```

132     density = 760.0 + 90.0 * safe_moisture
133     heat_capacity = 1850.0 + 2150.0 * safe_moisture / (safe_moisture + 1.0)
134     conductivity = 0.12 + 0.20 * safe_moisture / (safe_moisture + 1.0)
135     diffusivity = (
136         4.2e-4
137         * np.exp(-0.30 / safe_moisture)
138         * np.exp(-3850.0 / temperature_k)
139     )
140     return density, heat_capacity, conductivity, diffusivity
141
142
143 def stable_air_statistics() -> dict[str, float]:
144     air = core.read_air_boundary()
145     mask = (air.time_s >= STABLE_AIR_START_S) & (
146         air.time_s <= STABLE_AIR_END_S
147     )
148     if int(np.sum(mask)) != 91:
149         raise ValueError("附件1的9000至14400秒稳定段应含91个观测点")
150     temperature = air.temperature_c[mask]
151     moisture = air.moisture[mask]
152     return {
153         "sample_count": int(temperature.size),
154         "temperature_mean_c": float(np.mean(temperature)),
155         "temperature_std_c": float(np.std(temperature, ddof=0)),
156         "temperature_min_c": float(np.min(temperature)),
157         "temperature_max_c": float(np.max(temperature)),
158         "moisture_mean_kg_per_kg": float(np.mean(moisture)),
159         "moisture_std_kg_per_kg": float(np.std(moisture, ddof=0)),
160         "moisture_min_kg_per_kg": float(np.min(moisture)),
161         "moisture_max_kg_per_kg": float(np.max(moisture)),
162         "last_temperature_c": float(air.temperature_c[-1]),
163         "last_moisture_kg_per_kg": float(air.moisture[-1]),
164     }
165
166
167 def solve_tridiagonal_fast(
168     lower: np.ndarray,
169     diagonal: np.ndarray,
170     upper: np.ndarray,
171     right_hand_side: np.ndarray,
172 ) -> np.ndarray:
173     banded = np.zeros((3, diagonal.size), dtype=float)
174     banded[0, 1:] = upper
175     banded[1] = diagonal
176     banded[2, :-1] = lower
177     return solve_banded(
178         (1, 1),
179         banded,

```

```

180     right_hand_side,
181     overwrite_ab=True,
182     overwrite_b=False,
183     check_finite=False,
184 )
185
186
187 def relative_tridiagonal_residual(
188     lower: np.ndarray,
189     diagonal: np.ndarray,
190     upper: np.ndarray,
191     solution: np.ndarray,
192     right_hand_side: np.ndarray,
193 ) -> float:
194     residual = diagonal * solution - right_hand_side
195     residual[1:] += lower * solution[:-1]
196     residual[:-1] += upper * solution[1:]
197     scale = max(float(np.max(np.abs(right_hand_side))), 1.0e-30)
198     return float(np.max(np.abs(residual)) / scale)
199
200
201 def bdf2_effective_history(
202     current_state: np.ndarray,
203     previous_state: np.ndarray,
204 ) -> np.ndarray:
205     """Return the history state that lets the BE assembler realize BDF2.
206
207     With an effective step of 2*dt/3, the standard three-level formula is
208     (3*u[n+1] - 4*u[n] + u[n-1])/(2*dt). Material properties are evaluated
209     at the current Picard iterate on time level n+1.
210     """
211     return (4.0 * current_state - previous_state) / 3.0
212
213
214 def simulate_variable_model(
215     *,
216     model: str,
217     end_time_s: float,
218     time_step_s: float,
219     intervals: int,
220     record_interval_s: float,
221     stop_at_dryness: bool = False,
222     coupling_temperature_tolerance: float = 1.0e-8,
223     coupling_moisture_tolerance: float = 1.0e-10,
224     coupling_max_iterations: int = 40,
225     temporal_scheme: str = "backward_euler",
226     heat_transfer_coefficient: float = HEAT_TRANSFER_COEFFICIENT,
227     mass_transfer_coefficient: float = MASS_TRANSFER_COEFFICIENT,

```

```

228     post_observation_temperature_c: float | None = None,
229     post_observation_moisture: float | None = None,
230 ) -> VariableSimulation:
231     if model not in {"q23", "q4"}:
232         raise ValueError("model必须为q23或q4")
233     if temporal_scheme not in {"backward_euler", "bdf2"}:
234         raise ValueError("temporal_scheme必须为backward_euler或bdf2")
235     if heat_transfer_coefficient <= 0.0 or mass_transfer_coefficient <= 0.0:
236         raise ValueError("表面对流系数必须为正数")
237     if (post_observation_temperature_c is None) != (
238         post_observation_moisture is None
239     ):
240         raise ValueError("长期空气温度和水分边界必须同时给出或同时省略")
241     if post_observation_moisture is not None and post_observation_moisture < 0.0:
242         raise ValueError("长期空气水分浓度不得为负")
243     step_count_float = end_time_s / time_step_s
244     step_count = int(round(step_count_float))
245     if not np.isclose(step_count, step_count_float):
246         raise ValueError("结束时刻必须是时间步长的整数倍")
247     record_stride_float = record_interval_s / time_step_s
248     record_stride = int(round(record_stride_float))
249     if not np.isclose(record_stride, record_stride_float):
250         raise ValueError("记录间隔必须是时间步长的整数倍")
251
252     air = core.read_air_boundary()
253     radius_history = read_radius_history() if model == "q4" else None
254     property_function = q23_properties if model == "q23" else q4_properties
255     node_coordinate = np.linspace(0.0, 1.0, intervals + 1)
256     temperature = np.full(intervals + 1, INITIAL_TEMPERATURE_C, dtype=float)
257     moisture = np.full(intervals + 1, INITIAL_MOISTURE, dtype=float)
258
259     recorded_time = [0.0]
260     initial_radius = 0.02
261     recorded_radius = [initial_radius]
262     recorded_temperature = [temperature.copy()]
263     recorded_moisture = [moisture.copy()]
264     recorded_iterations = [0]
265     normalized_weights = np.empty(intervals + 1, dtype=float)
266     normalized_faces = np.empty(intervals + 2, dtype=float)
267     normalized_faces[0] = 0.0
268     normalized_faces[-1] = 1.0
269     normalized_faces[1:-1] = 0.5 * (
270         node_coordinate[:-1] + node_coordinate[1:]
271     )
272     normalized_weights[:] = (
273         normalized_faces[1:] ** 2 - normalized_faces[:-1] ** 2
274     )
275     if not np.isclose(np.sum(normalized_weights), 1.0):

```

```

276     raise RuntimeError("归一化控制体权重之和不等于1")
277
278     previous_temperature = temperature.copy()
279     previous_moisture = moisture.copy()
280     max_iterations_used = 0
281     max_balance_error = 0.0
282     max_temperature_linear_residual = 0.0
283     max_moisture_linear_residual = 0.0
284     dry_time_s: float | None = None
285
286     for step in range(1, step_count + 1):
287         current_time = step * time_step_s
288         radius_m = (
289             0.02 if radius_history is None else radius_history.at(current_time)
290         )
291         grid = core.make_grid(radius_m=radius_m, intervals=intervals)
292         ambient_temperature, ambient_moisture = air.at(current_time)
293         if (
294             current_time > float(air.time_s[-1])
295             and post_observation_temperature_c is not None
296             and post_observation_moisture is not None
297         ):
298             ambient_temperature = post_observation_temperature_c
299             ambient_moisture = post_observation_moisture
300             temperature_guess = temperature.copy()
301             moisture_guess = moisture.copy()
302             converged = False
303
304         for iteration in range(1, coupling_max_iterations + 1):
305             use_bdf2 = temporal_scheme == "bdf2" and step > 1
306             effective_step_s = 2.0 * time_step_s / 3.0 if use_bdf2 else time_step_s
307             temperature_history_state = (
308                 bdf2_effective_history(temperature, previous_temperature)
309                 if use_bdf2
310                 else temperature
311             )
312             moisture_history_state = (
313                 bdf2_effective_history(moisture, previous_moisture)
314                 if use_bdf2
315                 else moisture
316             )
317             density, heat_capacity, conductivity, _ = property_function(
318                 moisture_guess, temperature_guess
319             )
320             lower_t, diagonal_t, upper_t, rhs_t = core.assemble_implicit_system(
321                 grid,
322                 temperature_history_state,
323                 density * heat_capacity,

```

```

324         conductivity,
325         effective_step_s,
326         heat_transfer_coefficient,
327         ambient_temperature,
328     )
329     temperature_new = solve_tridiagonal_fast(
330         lower_t, diagonal_t, upper_t, rhs_t
331     )
332     max_temperature_linear_residual = max(
333         max_temperature_linear_residual,
334         relative_tridiagonal_residual(
335             lower_t, diagonal_t, upper_t, temperature_new, rhs_t
336         ),
337     )
338     _, _, _, diffusivity = property_function(
339         moisture_guess, temperature_new
340     )
341     lower_c, diagonal_c, upper_c, rhs_c = core.assemble_implicit_system(
342         grid,
343         moisture_history_state,
344         np.ones(intervals + 1, dtype=float),
345         diffusivity,
346         effective_step_s,
347         mass_transfer_coefficient,
348         ambient_moisture,
349     )
350     moisture_new = solve_tridiagonal_fast(
351         lower_c, diagonal_c, upper_c, rhs_c
352     )
353     max_moisture_linear_residual = max(
354         max_moisture_linear_residual,
355         relative_tridiagonal_residual(
356             lower_c, diagonal_c, upper_c, moisture_new, rhs_c
357         ),
358     )
359     temperature_error = float(
360         np.max(np.abs(temperature_new - temperature_guess))
361     )
362     moisture_error = float(np.max(np.abs(moisture_new - moisture_guess)))
363     temperature_guess = temperature_new
364     moisture_guess = moisture_new
365     if (
366         temperature_error <= coupling_temperature_tolerance
367         and moisture_error <= coupling_moisture_tolerance
368     ):
369         converged = True
370         break
371 if not converged:

```

```

372         raise RuntimeError(
373             f"{model} 耦合迭代在t={current_time:g}秒未收敛，"
374             f"温度误差{temperature_error:.3e}，含水率误差{moisture_error:.3e}"
375         )
376
377     mean_new = float(np.dot(normalized_weights, moisture_guess))
378     mean_current = float(np.dot(normalized_weights, moisture))
379     if temporal_scheme == "bdf2" and step > 1:
380         mean_previous = float(np.dot(normalized_weights, previous_moisture))
381         inventory_change = 1.5 * mean_new - 2.0 * mean_current + 0.5 * mean_previous
382     else:
383         inventory_change = mean_new - mean_current
384     boundary_loss = (
385         time_step_s
386         * 2.0
387         * mass_transfer_coefficient
388         / radius_m
389         * (moisture_guess[-1] - ambient_moisture)
390     )
391     balance_scale = max(abs(inventory_change), abs(boundary_loss), 1.0e-30)
392     max_balance_error = max(
393         max_balance_error,
394         abs(inventory_change + boundary_loss) / balance_scale,
395     )
396
397     previous_temperature, temperature = temperature, temperature_guess
398     previous_moisture, moisture = moisture, moisture_guess
399     max_iterations_used = max(max_iterations_used, iteration)
400
401     if not np.all(np.isfinite(temperature)) or not np.all(np.isfinite(moisture)):
402         raise RuntimeError(f"{model} 在t={current_time:g}秒出现非有限状态")
403     if float(np.min(moisture)) < -1.0e-12:
404         raise RuntimeError(f"{model} 在t={current_time:g}秒出现负含水率")
405
406     if step % record_stride == 0:
407         recorded_time.append(current_time)
408         recorded_radius.append(radius_m)
409         recorded_temperature.append(temperature.copy())
410         recorded_moisture.append(moisture.copy())
411         recorded_iterations.append(iteration)
412
413     if stop_at_dryness and float(np.max(moisture)) < DRYING_THRESHOLD:
414         dry_time_s = current_time
415         if recorded_time[-1] != current_time:
416             recorded_time.append(current_time)
417             recorded_radius.append(radius_m)
418             recorded_temperature.append(temperature.copy())
419             recorded_moisture.append(moisture.copy())

```

```

420         recorded_iterations.append(iteration)
421         break
422
423     if stop_at_dryness and dry_time_s is None:
424         raise RuntimeError(f"{model} 在 {end_time_s / 3600:g} 小时内未达到干燥阈值")
425
426     return VariableSimulation(
427         time_s=np.asarray(recorded_time, dtype=float),
428         radius_m=np.asarray(recorded_radius, dtype=float),
429         node_coordinate=node_coordinate,
430         temperature_c=np.asarray(recorded_temperature, dtype=float),
431         moisture=np.asarray(recorded_moisture, dtype=float),
432         max_coupling_iterations=max_iterations_used,
433         moisture_balance_relative_error=float(max_balance_error),
434         dry_time_s=dry_time_s,
435         coupling_iterations=np.asarray(recorded_iterations, dtype=int),
436         max_temperature_linear_residual=float(max_temperature_linear_residual),
437         max_moisture_linear_residual=float(max_moisture_linear_residual),
438     )
439
440
441 def interpolate_at_physical_radii(
442     simulation: VariableSimulation,
443     field: np.ndarray,
444     row_index: int,
445     radii_cm: list[float],
446 ) -> np.ndarray:
447     current_radius_cm = 100.0 * simulation.radius_m[row_index]
448     physical_nodes_cm = simulation.node_coordinate * current_radius_cm
449     return np.interp(radii_cm, physical_nodes_cm, field[row_index])
450
451
452 def write_result2(simulation: VariableSimulation, output_path: Path) -> None:
453     if not np.allclose(np.diff(simulation.time_s), 1.0):
454         raise ValueError("result2要求每1秒记录一次")
455     template = openpyxl.load_workbook(TEMPLATE_DIR / "result2.xlsx")
456     radii_cm = [round(0.1 * index, 1) for index in range(21)]
457     for sheet_name, field in (
458         ("温度", simulation.temperature_c),
459         ("水分浓度", simulation.moisture),
460     ):
461         worksheet = template[sheet_name]
462         if worksheet.max_row > 1:
463             worksheet.delete_rows(2, worksheet.max_row - 1)
464         worksheet.cell(1, 1, "时间\\到药材中心的距离")
465         for column, radius_cm in enumerate(radii_cm, start=2):
466             worksheet.cell(1, column, radius_cm)
467         for row, time_s in enumerate(simulation.time_s[1:], start=2):

```

```

468         source_index = row - 1
469         values = interpolate_at_physical_radii(
470             simulation, field, source_index, radii_cm
471         )
472         worksheet.cell(row, 1, int(round(float(time_s))))
473         for column, value in enumerate(values, start=2):
474             cell = worksheet.cell(row, column, round(float(value), 4))
475             cell.number_format = "0.0000"
476         worksheet.freeze_panes = "B2"
477     output_path.parent.mkdir(parents=True, exist_ok=True)
478     temporary = output_path.with_suffix(".tmp.xlsx")
479     template.save(temporary)
480     template.close()
481     commit_temporary_file(temporary, output_path)
482
483
484 def write_result3(simulation: VariableSimulation, output_path: Path) -> None:
485     template = openpyxl.load_workbook(TEMPLATE_DIR / "result3.xlsx")
486     worksheet = template.active
487     if worksheet.max_row > 1:
488         worksheet.delete_rows(2, worksheet.max_row - 1)
489     radii_cm = [round(0.1 * index, 1) for index in range(21)]
490     worksheet.cell(1, 1, "时间\\到药材中心的距离")
491     for column, radius_cm in enumerate(radii_cm, start=2):
492         worksheet.cell(1, column, radius_cm)
493     for row, time_s in enumerate(simulation.time_s[1:], start=2):
494         source_index = row - 1
495         values = interpolate_at_physical_radii(
496             simulation, simulation.moisture, source_index, radii_cm
497         )
498         worksheet.cell(row, 1, int(round(float(time_s))))
499         for column, value in enumerate(values, start=2):
500             cell = worksheet.cell(row, column, round(float(value), 4))
501             cell.number_format = "0.0000"
502     worksheet.freeze_panes = "B2"
503     output_path.parent.mkdir(parents=True, exist_ok=True)
504     temporary = output_path.with_suffix(".tmp.xlsx")
505     template.save(temporary)
506     template.close()
507     commit_temporary_file(temporary, output_path)
508
509
510 def write_result4(simulation: VariableSimulation, output_path: Path) -> None:
511     template = openpyxl.load_workbook(TEMPLATE_DIR / "result4.xlsx")
512     worksheet = template.active
513     if worksheet.max_row > 1:
514         worksheet.delete_rows(2, worksheet.max_row - 1)
515     fixed_radii_cm = [round(0.1 * index, 1) for index in range(20)]

```

```

516     worksheet.cell(1, 1, "时间\\到药材中心的距离")
517     for column, radius_cm in enumerate(fixed_radii_cm, start=2):
518         worksheet.cell(1, column, radius_cm)
519     surface_column = len(fixed_radii_cm) + 2
520     worksheet.cell(1, surface_column, "药材表面")
521
522     for row, time_s in enumerate(simulation.time_s[1:], start=2):
523         source_index = row - 1
524         current_radius_cm = 100.0 * simulation.radius_m[source_index]
525         physical_nodes_cm = simulation.node_coordinate * current_radius_cm
526         worksheet.cell(row, 1, int(round(float(time_s))))
527         for column, radius_cm in enumerate(fixed_radii_cm, start=2):
528             if radius_cm < current_radius_cm - 1.0e-10:
529                 value = np.interp(
530                     radius_cm,
531                     physical_nodes_cm,
532                     simulation.moisture[source_index],
533                 )
534                 worksheet.cell(row, column, round(float(value), 4))
535             else:
536                 worksheet.cell(row, column, None)
537         worksheet.cell(
538             row,
539             surface_column,
540             round(float(simulation.moisture[source_index, -1]), 4),
541         )
542     worksheet.freeze_panes = "B2"
543     output_path.parent.mkdir(parents=True, exist_ok=True)
544     temporary = output_path.with_suffix(".tmp.xlsx")
545     template.save(temporary)
546     template.close()
547     commit_temporary_file(temporary, output_path)
548
549
550 def requested_table(
551     simulation: VariableSimulation,
552     field: np.ndarray,
553     times_s: list[float],
554     radii_cm: list[float],
555 ) -> list[list[float]]:
556     rows: list[list[float]] = []
557     for time_s in times_s:
558         index = int(np.argmin(np.abs(simulation.time_s - time_s)))
559         if not np.isclose(simulation.time_s[index], time_s):
560             raise ValueError(f"记录结果不含时刻{time_s}秒")
561         rows.append(
562             interpolate_at_physical_radii(
563                 simulation, field, index, radii_cm

```

```

564         ).tolist()
565     )
566     return rows
567
568
569 def write_table_csv(
570     output_path: Path,
571     times_h: list[float],
572     radii_cm: list[float],
573     values: np.ndarray,
574     value_format: str = ".8f",
575 ) -> None:
576     output_path.parent.mkdir(parents=True, exist_ok=True)
577     with output_path.open("w", encoding="utf-8-sig", newline="") as stream:
578         writer = csv.writer(stream)
579         writer.writerow(["时间_h", *[f"{radius:g}_cm" for radius in radii_cm]])
580         for time_h, row in zip(times_h, values, strict=True):
581             writer.writerow([f"{time_h:.1f}", *[format(float(value), value_format) for value in row
582                                     ]]])
583
584 def final_system_condition_numbers(
585     simulation: VariableSimulation,
586     time_step_s: float,
587     heat_transfer_coefficient: float,
588     mass_transfer_coefficient: float,
589     post_observation_temperature_c: float | None = None,
590     post_observation_moisture: float | None = None,
591 ) -> dict[str, float]:
592     intervals = simulation.node_coordinate.size - 1
593     radius_m = float(simulation.radius_m[-1])
594     grid = core.make_grid(radius_m=radius_m, intervals=intervals)
595     air = core.read_air_boundary()
596     ambient_temperature, ambient_moisture = air.at(float(simulation.time_s[-1]))
597     if (
598         float(simulation.time_s[-1]) > float(air.time_s[-1])
599         and post_observation_temperature_c is not None
600         and post_observation_moisture is not None
601     ):
602         ambient_temperature = post_observation_temperature_c
603         ambient_moisture = post_observation_moisture
604     density, heat_capacity, conductivity, diffusivity = q23_properties(
605         simulation.moisture[-1], simulation.temperature_c[-1]
606     )
607     effective_step_s = 2.0 * time_step_s / 3.0
608     lower_t, diagonal_t, upper_t, _ = core.assemble_implicit_system(
609         grid,
610         simulation.temperature_c[-1],

```

```

611     density * heat_capacity,
612     conductivity,
613     effective_step_s,
614     heat_transfer_coefficient,
615     ambient_temperature,
616 )
617 lower_c, diagonal_c, upper_c, _ = core.assemble_implicit_system(
618     grid,
619     simulation.moisture[-1],
620     np.ones(intervals + 1, dtype=float),
621     diffusivity,
622     effective_step_s,
623     mass_transfer_coefficient,
624     ambient_moisture,
625 )
626 return {
627     "temperature_final": float(
628         np.linalg.cond(core.dense_from_tridiagonal(lower_t, diagonal_t, upper_t))
629     ),
630     "moisture_final": float(
631         np.linalg.cond(core.dense_from_tridiagonal(lower_c, diagonal_c, upper_c))
632     ),
633 }
634
635
636 def run_smoke() -> None:
637     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
638     q2 = simulate_variable_model(
639         model="q23",
640         end_time_s=600.0,
641         time_step_s=1.0,
642         intervals=20,
643         record_interval_s=60.0,
644         temporal_scheme="bdf2",
645     )
646     q4 = simulate_variable_model(
647         model="q4",
648         end_time_s=3600.0,
649         time_step_s=10.0,
650         intervals=20,
651         record_interval_s=60.0,
652     )
653     q2_rho, q2_cp, q2_k, q2_d = q23_properties(
654         q2.moisture[-1], q2.temperature_c[-1]
655     )
656     q4_rho, q4_cp, q4_k, q4_d = q4_properties(
657         q4.moisture[-1], q4.temperature_c[-1]
658     )

```

```

659 report = {
660     "mode": "PI_q2_q4_smoke",
661     "inputs": {
662         str(AIR_PATH.relative_to(PROJECT_ROOT)): sha256_file(AIR_PATH),
663         str(RADIUS_PATH.relative_to(PROJECT_ROOT)): sha256_file(RADIUS_PATH),
664     },
665     "q2": {
666         "time_step_s": 1.0,
667         "radial_intervals": 20,
668         "end_time_s": 600.0,
669         "temperature_range_c": [
670             float(np.min(q2.temperature_c)),
671             float(np.max(q2.temperature_c)),
672         ],
673         "moisture_range_kg_per_kg": [
674             float(np.min(q2.moisture)),
675             float(np.max(q2.moisture)),
676         ],
677         "property_ranges": {
678             "density_kg_per_m3": [float(q2_rho.min()), float(q2_rho.max())],
679             "heat_capacity_j_per_kg_k": [float(q2_cp.min()), float(q2_cp.max())],
680             "conductivity_w_per_m_k": [float(q2_k.min()), float(q2_k.max())],
681             "diffusivity_m2_per_s": [float(q2_d.min()), float(q2_d.max())],
682         },
683         "max_coupling_iterations": q2.max_coupling_iterations,
684         "moisture_balance_relative_error": q2.moisture_balance_relative_error,
685         "max_temperature_linear_residual": q2.max_temperature_linear_residual,
686         "max_moisture_linear_residual": q2.max_moisture_linear_residual,
687     },
688     "q4": {
689         "time_step_s": 10.0,
690         "radial_intervals": 20,
691         "end_time_s": 3600.0,
692         "radius_start_m": float(q4.radius_m[0]),
693         "radius_end_m": float(q4.radius_m[-1]),
694         "temperature_range_c": [
695             float(np.min(q4.temperature_c)),
696             float(np.max(q4.temperature_c)),
697         ],
698         "moisture_range_kg_per_kg": [
699             float(np.min(q4.moisture)),
700             float(np.max(q4.moisture)),
701         ],
702         "property_ranges": {
703             "density_kg_per_m3": [float(q4_rho.min()), float(q4_rho.max())],
704             "heat_capacity_j_per_kg_k": [float(q4_cp.min()), float(q4_cp.max())],
705             "conductivity_w_per_m_k": [float(q4_k.min()), float(q4_k.max())],
706             "diffusivity_m2_per_s": [float(q4_d.min()), float(q4_d.max())],

```

```

707         },
708         "max_coupling_iterations": q4.max_coupling_iterations,
709         "moisture_balance_relative_error": q4.moisture_balance_relative_error,
710     },
711 }
712 output = RESULTS_DIR / "P1_q2_q4_smoke.json"
713 output.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
714 print(json.dumps(report, ensure_ascii=False, indent=2))
715
716
717 def run_q2() -> None:
718     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
719     authoritative = simulate_variable_model(
720         model="q23",
721         end_time_s=10800.0,
722         time_step_s=1.0,
723         intervals=160,
724         record_interval_s=1.0,
725         temporal_scheme="bdf2",
726     )
727     time_refined = simulate_variable_model(
728         model="q23",
729         end_time_s=10800.0,
730         time_step_s=0.5,
731         intervals=160,
732         record_interval_s=1800.0,
733         temporal_scheme="bdf2",
734     )
735     space_refined = simulate_variable_model(
736         model="q23",
737         end_time_s=10800.0,
738         time_step_s=1.0,
739         intervals=320,
740         record_interval_s=1800.0,
741         temporal_scheme="bdf2",
742     )
743     write_result2(authoritative, RESULTS_DIR / "result2.xlsx")
744
745     times_s = [1800.0 * index for index in range(1, 7)]
746     radii_cm = [0.0, 0.5, 1.0, 1.5, 2.0]
747     temperature = np.asarray(
748         requested_table(
749             authoritative, authoritative.temperature_c, times_s, radii_cm
750         )
751     )
752     moisture = np.asarray(
753         requested_table(authoritative, authoritative.moisture, times_s, radii_cm)
754     )

```

```

755     time_temperature = np.asarray(
756         requested_table(
757             time_refined, time_refined.temperature_c, times_s, radii_cm
758         )
759     )
760     time_moisture = np.asarray(
761         requested_table(time_refined, time_refined.moisture, times_s, radii_cm)
762     )
763     space_temperature = np.asarray(
764         requested_table(
765             space_refined, space_refined.temperature_c, times_s, radii_cm
766         )
767     )
768     space_moisture = np.asarray(
769         requested_table(space_refined, space_refined.moisture, times_s, radii_cm)
770     )
771     times_h = [time_s / 3600.0 for time_s in times_s]
772     write_table_csv(
773         RESULTS_DIR / "问题2_表3温度.csv", times_h, radii_cm, temperature
774     )
775     write_table_csv(
776         RESULTS_DIR / "问题2_表4含水率.csv", times_h, radii_cm, moisture
777     )
778
779     with (RESULTS_DIR / "问题2_迭代次数.csv").open(
780         "w", encoding="utf-8-sig", newline=""
781     ) as stream:
782         writer = csv.writer(stream)
783         writer.writerow(["时间_s", "Picard迭代次数"])
784         writer.writerows(
785             zip(
786                 authoritative.time_s.astype(int).tolist(),
787                 authoritative.coupling_iterations.tolist(),
788                 strict=True,
789             )
790         )
791
792     property_density, property_cp, property_k, property_d = q23_properties(
793         authoritative.moisture, authoritative.temperature_c
794     )
795     property_rows = []
796     for time_s in times_s:
797         row_index = int(np.argmin(np.abs(authoritative.time_s - time_s)))
798         property_rows.append(
799             [
800                 time_s / 3600.0,
801                 float(np.min(property_density[row_index])),
802                 float(np.max(property_density[row_index])),

```

```

803         float(np.min(property_cp[row_index])),
804         float(np.max(property_cp[row_index])),
805         float(np.min(property_k[row_index])),
806         float(np.max(property_k[row_index])),
807         float(np.min(property_d[row_index])),
808         float(np.max(property_d[row_index])),
809     ]
810 )
811 with (RESULTS_DIR / "问题2_物性范围.csv").open(
812     "w", encoding="utf-8-sig", newline=""
813 ) as stream:
814     writer = csv.writer(stream)
815     writer.writerow(
816         [
817             "时间_h",
818             "密度最小_kg_m3",
819             "密度最大_kg_m3",
820             "比热最小_J_kg_K",
821             "比热最大_J_kg_K",
822             "导热系数最小_W_m_K",
823             "导热系数最大_W_m_K",
824             "扩散系数最小_m2_s",
825             "扩散系数最大_m2_s",
826         ]
827     )
828     writer.writerows(property_rows)
829
830 sensitivity_cases = {
831     "换热系数_0.8倍": (0.8 * HEAT_TRANSFER_COEFFICIENT, MASS_TRANSFER_COEFFICIENT),
832     "换热系数_1.2倍": (1.2 * HEAT_TRANSFER_COEFFICIENT, MASS_TRANSFER_COEFFICIENT),
833     "传质系数_0.8倍": (HEAT_TRANSFER_COEFFICIENT, 0.8 * MASS_TRANSFER_COEFFICIENT),
834     "传质系数_1.2倍": (HEAT_TRANSFER_COEFFICIENT, 1.2 * MASS_TRANSFER_COEFFICIENT),
835 }
836 sensitivity = []
837 for case_name, (heat_coefficient, mass_coefficient) in sensitivity_cases.items():
838     case_simulation = simulate_variable_model(
839         model="q23",
840         end_time_s=10800.0,
841         time_step_s=1.0,
842         intervals=160,
843         record_interval_s=1800.0,
844         temporal_scheme="bdf2",
845         heat_transfer_coefficient=heat_coefficient,
846         mass_transfer_coefficient=mass_coefficient,
847     )
848     case_temperature = np.asarray(
849         requested_table(
850             case_simulation,

```

```

851         case_simulation.temperature_c,
852         times_s,
853         radii_cm,
854     )
855 )
856 case_moisture = np.asarray(
857     requested_table(
858         case_simulation,
859         case_simulation.moisture,
860         times_s,
861         radii_cm,
862     )
863 )
864 sensitivity.append(
865     {
866         "case": case_name,
867         "heat_transfer_coefficient_w_per_m2_k": heat_coefficient,
868         "mass_transfer_coefficient_m_per_s": mass_coefficient,
869         "max_temperature_change_c": float(
870             np.max(np.abs(case_temperature - temperature))
871         ),
872         "max_moisture_change_kg_per_kg": float(
873             np.max(np.abs(case_moisture - moisture))
874         ),
875         "temperature_center_3h_change_c": float(
876             case_temperature[-1, 0] - temperature[-1, 0]
877         ),
878         "temperature_surface_3h_change_c": float(
879             case_temperature[-1, -1] - temperature[-1, -1]
880         ),
881         "moisture_center_3h_change_kg_per_kg": float(
882             case_moisture[-1, 0] - moisture[-1, 0]
883         ),
884         "moisture_surface_3h_change_kg_per_kg": float(
885             case_moisture[-1, -1] - moisture[-1, -1]
886         ),
887     }
888 )
889 with (RESULTS_DIR / "问题2_敏感性.csv").open(
890     "w", encoding="utf-8-sig", newline=""
891 ) as stream:
892     fieldnames = list(sensitivity[0].keys())
893     writer = csv.DictWriter(stream, fieldnames=fieldnames)
894     writer.writeheader()
895     writer.writerows(sensitivity)
896
897 condition_numbers = final_system_condition_numbers(
898     authoritative,

```

```

899     time_step_s=1.0,
900     heat_transfer_coefficient=HEAT_TRANSFER_COEFFICIENT,
901     mass_transfer_coefficient=MASS_TRANSFER_COEFFICIENT,
902 )
903 summary = {
904     "mode": "q2_full",
905     "inputs": {
906         str(AIR_PATH.relative_to(PROJECT_ROOT)): sha256_file(AIR_PATH)
907     },
908     "parameters": {
909         "temporal_scheme": "backward_euler_start_then_bdf2",
910         "authoritative": {"intervals": 160, "time_step_s": 1.0},
911         "time_refined": {"intervals": 160, "time_step_s": 0.5},
912         "space_refined": {"intervals": 320, "time_step_s": 1.0},
913         "heat_transfer_coefficient_w_per_m2_k": HEAT_TRANSFER_COEFFICIENT,
914         "mass_transfer_coefficient_m_per_s": MASS_TRANSFER_COEFFICIENT,
915         "coupling_temperature_tolerance_c": 1.0e-8,
916         "coupling_moisture_tolerance_kg_per_kg": 1.0e-10,
917         "coupling_max_iterations": 40,
918     },
919     "requested_times_s": times_s,
920     "requested_radii_cm": radii_cm,
921     "temperature_c": np.round(temperature, 8).tolist(),
922     "moisture_kg_per_kg": np.round(moisture, 8).tolist(),
923     "property_ranges": {
924         "density_kg_per_m3": [
925             float(np.min(property_density)),
926             float(np.max(property_density)),
927         ],
928         "heat_capacity_j_per_kg_k": [
929             float(np.min(property_cp)),
930             float(np.max(property_cp)),
931         ],
932         "conductivity_w_per_m_k": [
933             float(np.min(property_k)),
934             float(np.max(property_k)),
935         ],
936         "diffusivity_m2_per_s": [
937             float(np.min(property_d)),
938             float(np.max(property_d)),
939         ],
940     },
941     "sensitivity": sensitivity,
942     "verification": {
943         "max_temperature_difference_time_refined_c": float(
944             np.max(np.abs(temperature - time_temperature))
945         ),
946         "max_moisture_difference_time_refined_kg_per_kg": float(

```

```

947         np.max(np.abs(moisture - time_moisture))
948     ),
949     "max_temperature_difference_space_refined_c": float(
950         np.max(np.abs(temperature - space_temperature))
951     ),
952     "max_moisture_difference_space_refined_kg_per_kg": float(
953         np.max(np.abs(moisture - space_moisture))
954     ),
955     "rounded_temperature_changes_time_refined": int(
956         np.sum(np.round(temperature, 4) != np.round(time_temperature, 4))
957     ),
958     "rounded_moisture_changes_time_refined": int(
959         np.sum(np.round(moisture, 4) != np.round(time_moisture, 4))
960     ),
961     "rounded_temperature_changes_space_refined": int(
962         np.sum(np.round(temperature, 4) != np.round(space_temperature, 4))
963     ),
964     "rounded_moisture_changes_space_refined": int(
965         np.sum(np.round(moisture, 4) != np.round(space_moisture, 4))
966     ),
967     "moisture_balance_relative_error": authoritative.moisture_balance_relative_error,
968     "max_coupling_iterations": authoritative.max_coupling_iterations,
969     "max_temperature_linear_residual": authoritative.max_temperature_linear_residual,
970     "max_moisture_linear_residual": authoritative.max_moisture_linear_residual,
971     "final_system_condition_numbers": condition_numbers,
972 },
973 "environment": {
974     "python": platform.python_version(),
975     "numpy": np.__version__,
976     "openpyxl": openpyxl.__version__,
977     "platform": platform.platform(),
978 },
979 }
980 (RESULTS_DIR / "问题2_结果摘要.json").write_text(
981     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
982 )
983 print(json.dumps(summary, ensure_ascii=False, indent=2))
984
985
986 def q3_periodic_table(
987     simulation: VariableSimulation,
988     radii_cm: list[float],
989 ) -> tuple[list[float], np.ndarray]:
990     if simulation.dry_time_s is None:
991         raise RuntimeError("问题三未返回烘干结束时刻")
992     periodic_times = list(
993         np.arange(6.0 * 3600.0, simulation.dry_time_s, 6.0 * 3600.0)
994     )

```

```

995     table_times = periodic_times + [simulation.dry_time_s]
996     table = np.asarray(
997         requested_table(simulation, simulation.moisture, table_times, radii_cm)
998     )
999     return table_times, table
1000
1001
1002 def q3_common_table_difference(
1003     first: VariableSimulation,
1004     second: VariableSimulation,
1005     radii_cm: list[float],
1006 ) -> float:
1007     if first.dry_time_s is None or second.dry_time_s is None:
1008         raise RuntimeError("网格比较缺少烘干结束时刻")
1009     common_end = min(first.dry_time_s, second.dry_time_s)
1010     periodic_times = list(np.arange(6.0 * 3600.0, common_end, 6.0 * 3600.0))
1011     first_table = np.asarray(
1012         requested_table(first, first.moisture, periodic_times, radii_cm)
1013     )
1014     second_table = np.asarray(
1015         requested_table(second, second.moisture, periodic_times, radii_cm)
1016     )
1017     periodic_difference = float(np.max(np.abs(first_table - second_table)))
1018     first_end = interpolate_at_physical_radii(
1019         first, first.moisture, -1, radii_cm
1020     )
1021     second_end = interpolate_at_physical_radii(
1022         second, second.moisture, -1, radii_cm
1023     )
1024     return max(periodic_difference, float(np.max(np.abs(first_end - second_end))))
1025
1026
1027 def write_q3_process_csv(
1028     simulation: VariableSimulation,
1029     output_path: Path,
1030 ) -> None:
1031     radii_cm = [0.0, 0.5, 1.0, 1.5, 2.0]
1032     with output_path.open("w", encoding="utf-8-sig", newline="") as stream:
1033         writer = csv.writer(stream)
1034         writer.writerow(
1035             [
1036                 "time_s",
1037                 "time_h",
1038                 *[f"moisture_{radius:g}_cm" for radius in radii_cm],
1039                 "maximum_moisture",
1040                 "maximum_radius_cm",
1041             ]
1042         )

```

```

1043     for index, time_s in enumerate(simulation.time_s):
1044         values = interpolate_at_physical_radii(
1045             simulation, simulation.moisture, index, radii_cm
1046         )
1047         maximum_index = int(np.argmax(simulation.moisture[index]))
1048         writer.writerow(
1049             [
1050                 f"{time_s:.6f}",
1051                 f"{time_s / 3600.0:.10f}",
1052                 *[f"{value:.10f}" for value in values],
1053                 f"{simulation.moisture[index, maximum_index]:.10f}",
1054                 f"{100.0 * simulation.radius_m[index] * simulation.node_coordinate[maximum_index]
1055                  :.8f}",
1056             ]
1057         )
1058
1059 def run_q3_smoke() -> None:
1060     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1061     stable = stable_air_statistics()
1062     simulation = simulate_variable_model(
1063         model="q23",
1064         end_time_s=18000.0,
1065         time_step_s=10.0,
1066         intervals=20,
1067         record_interval_s=60.0,
1068         temporal_scheme="bdf2",
1069         post_observation_temperature_c=stable["temperature_mean_c"],
1070         post_observation_moisture=stable["moisture_mean_kg_per_kg"],
1071     )
1072     maximum_series = np.max(simulation.moisture, axis=1)
1073     maximum_increase = float(np.max(np.diff(maximum_series)))
1074     if maximum_increase > 1.0e-8:
1075         raise RuntimeError("问题三冒烟测试的最大含水率出现异常回升")
1076     if simulation.moisture_balance_relative_error > 1.0e-8:
1077         raise RuntimeError("问题三冒烟测试的水分平衡残差超过合同阈值")
1078     if simulation.max_temperature_linear_residual > 1.0e-12:
1079         raise RuntimeError("问题三冒烟测试的温度代数残差超过合同阈值")
1080     if simulation.max_moisture_linear_residual > 1.0e-12:
1081         raise RuntimeError("问题三冒烟测试的含水率代数残差超过合同阈值")
1082     report = {
1083         "mode": "PI_q3_smoke",
1084         "inputs": {
1085             str(AIR_PATH.relative_to(PROJECT_ROOT)): sha256_file(AIR_PATH),
1086         },
1087         "model": "q23_fixed_radius_stable_mean_extension",
1088         "temporal_scheme": "backward_euler_start_then_bdf2",
1089         "time_step_s": 10.0,

```

```

1090     "radial_intervals": 20,
1091     "end_time_s": 18000.0,
1092     "stable_air": stable,
1093     "temperature_range_c": [
1094         float(np.min(simulation.temperature_c)),
1095         float(np.max(simulation.temperature_c)),
1096     ],
1097     "moisture_range_kg_per_kg": [
1098         float(np.min(simulation.moisture)),
1099         float(np.max(simulation.moisture)),
1100     ],
1101     "maximum_moisture_start": float(maximum_series[0]),
1102     "maximum_moisture_end": float(maximum_series[-1]),
1103     "maximum_recorded_increase": maximum_increase,
1104     "max_coupling_iterations": simulation.max_coupling_iterations,
1105     "moisture_balance_relative_error": simulation.moisture_balance_relative_error,
1106     "max_temperature_linear_residual": simulation.max_temperature_linear_residual,
1107     "max_moisture_linear_residual": simulation.max_moisture_linear_residual,
1108 }
1109 output = RESULTS_DIR / "P1_q3_smoke.json"
1110 output.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
1111 print(json.dumps(report, ensure_ascii=False, indent=2))
1112
1113
1114 def run_q3() -> None:
1115     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1116     run_started = time.perf_counter()
1117     maximum_time_s = 7.0 * 24.0 * 3600.0
1118     stable = stable_air_statistics()
1119     boundary_temperature = stable["temperature_mean_c"]
1120     boundary_moisture = stable["moisture_mean_kg_per_kg"]
1121     common_arguments = {
1122         "model": "q23",
1123         "end_time_s": maximum_time_s,
1124         "intervals": 160,
1125         "record_interval_s": 60.0,
1126         "stop_at_dryness": True,
1127         "temporal_scheme": "bdf2",
1128         "post_observation_temperature_c": boundary_temperature,
1129         "post_observation_moisture": boundary_moisture,
1130     }
1131
1132     base = simulate_variable_model(time_step_s=1.0, **common_arguments)
1133     time_half = simulate_variable_model(time_step_s=0.5, **common_arguments)
1134     time_quarter = simulate_variable_model(time_step_s=0.25, **common_arguments)
1135     if base.dry_time_s is None or time_half.dry_time_s is None or time_quarter.dry_time_s is None:
1136         raise RuntimeError("时间步验证缺少烘干结束时刻")
1137     time_base_difference = abs(base.dry_time_s - time_quarter.dry_time_s)

```

```

1138 time_half_difference = abs(time_half.dry_time_s - time_quarter.dry_time_s)
1139 adjacent_base_half = abs(base.dry_time_s - time_half.dry_time_s)
1140 adjacent_half_quarter = time_half_difference
1141 time_gate_passed = (
1142     time_base_difference <= 1.0
1143     and time_half_difference <= 0.5
1144     and adjacent_half_quarter <= adjacent_base_half + 1.0e-12
1145 )
1146 authoritative = base if time_gate_passed else time_quarter
1147 authoritative_time_step = 1.0 if time_gate_passed else 0.25
1148
1149 space_refined = simulate_variable_model(
1150     model="q23",
1151     end_time_s=maximum_time_s,
1152     time_step_s=authoritative_time_step,
1153     intervals=320,
1154     record_interval_s=60.0,
1155     stop_at_dryness=True,
1156     temporal_scheme="bdf2",
1157     post_observation_temperature_c=boundary_temperature,
1158     post_observation_moisture=boundary_moisture,
1159 )
1160 if authoritative.dry_time_s is None or space_refined.dry_time_s is None:
1161     raise RuntimeError("空间加密验证缺少烘干结束时刻")
1162 radii_cm = [0.0, 0.5, 1.0, 1.5, 2.0]
1163 space_table_difference = q3_common_table_difference(
1164     authoritative, space_refined, radii_cm
1165 )
1166 space_time_difference_s = abs(
1167     authoritative.dry_time_s - space_refined.dry_time_s
1168 )
1169 space_time_relative_difference = (
1170     space_time_difference_s / space_refined.dry_time_s
1171 )
1172 if space_time_relative_difference > 0.001:
1173     space_fine = simulate_variable_model(
1174         model="q23",
1175         end_time_s=maximum_time_s,
1176         time_step_s=authoritative_time_step,
1177         intervals=640,
1178         record_interval_s=60.0,
1179         stop_at_dryness=True,
1180         temporal_scheme="bdf2",
1181         post_observation_temperature_c=boundary_temperature,
1182         post_observation_moisture=boundary_moisture,
1183     )
1184     if space_fine.dry_time_s is None:
1185         raise RuntimeError("640区间空间复验缺少烘干结束时刻")

```

```

1186     fine_relative_difference = abs(
1187         space_refined.dry_time_s - space_fine.dry_time_s
1188     ) / space_fine.dry_time_s
1189     if fine_relative_difference > 0.001:
1190         raise RuntimeError("640区间复验后烘干时长仍未通过0.1%空间阈值")
1191     authoritative = space_fine
1192     authoritative_intervals = 640
1193 elif space_table_difference > 5.0e-5:
1194     authoritative = space_refined
1195     authoritative_intervals = 320
1196     fine_relative_difference = None
1197 else:
1198     authoritative_intervals = 160
1199     fine_relative_difference = None
1200
1201 if authoritative.dry_time_s is None:
1202     raise RuntimeError("问题三权威解缺少烘干时长")
1203 maximum_series = np.max(authoritative.moisture, axis=1)
1204 maximum_recorded_increase = float(np.max(np.diff(maximum_series)))
1205 if maximum_recorded_increase > 1.0e-8:
1206     raise RuntimeError("问题三权威解的最大含水率出现异常回升")
1207 if authoritative.moisture_balance_relative_error > 1.0e-8:
1208     raise RuntimeError("问题三权威解的水分平衡残差超过合同阈值")
1209 if authoritative.max_temperature_linear_residual > 1.0e-12:
1210     raise RuntimeError("问题三权威解的温度代数残差超过合同阈值")
1211 if authoritative.max_moisture_linear_residual > 1.0e-12:
1212     raise RuntimeError("问题三权威解的含水率代数残差超过合同阈值")
1213
1214 write_result3(authoritative, RESULTS_DIR / "result3.xlsx")
1215 table_times, table = q3_periodic_table(authoritative, radii_cm)
1216 with (RESULTS_DIR / "问题3_表5含水率.csv").open(
1217     "w", encoding="utf-8-sig", newline=""
1218 ) as stream:
1219     writer = csv.writer(stream)
1220     writer.writerow(["时间_h", *[f"{radius:g}_cm" for radius in radii_cm]])
1221     for time_s, row in zip(table_times, table, strict=True):
1222         writer.writerow(
1223             [
1224                 f"{time_s / 3600.0:.6f}",
1225                 *[f"{float(value):.8f}" for value in row],
1226             ]
1227         )
1228 write_q3_process_csv(authoritative, RESULTS_DIR / "问题3_关键过程.csv")
1229 with (RESULTS_DIR / "问题3_迭代次数.csv").open(
1230     "w", encoding="utf-8-sig", newline=""
1231 ) as stream:
1232     writer = csv.writer(stream)
1233     writer.writerow(["时间_s", "Picard迭代次数"])

```

```

1234     writer.writerows(
1235         zip(
1236             authoritative.time_s.tolist(),
1237             authoritative.coupling_iterations.tolist(),
1238             strict=True,
1239         )
1240     )
1241
1242     with (RESULTS_DIR / "问题3_时间步敏感性.csv").open(
1243         "w", encoding="utf-8-sig", newline=""
1244     ) as stream:
1245         writer = csv.writer(stream)
1246         writer.writerow(["时间步_s", "烘干时长_s", "烘干时长_h", "相对0.25s绝对差_s"])
1247         for step, simulation in ((1.0, base), (0.5, time_half), (0.25, time_quarter)):
1248             writer.writerow(
1249                 [
1250                     step,
1251                     f"{float(simulation.dry_time_s):.6f}",
1252                     f"{float(simulation.dry_time_s) / 3600.0:.10f}",
1253                     f"{abs(float(simulation.dry_time_s) - time_quarter.dry_time_s):.6f}",
1254                 ]
1255             )
1256     with (RESULTS_DIR / "问题3_空间网格敏感性.csv").open(
1257         "w", encoding="utf-8-sig", newline=""
1258     ) as stream:
1259         writer = csv.writer(stream)
1260         writer.writerow(["径向区间数", "时间步_s", "烘干时长_s", "烘干时长_h"])
1261         writer.writerow([160, authoritative_time_step, authoritative.dry_time_s if
1262             authoritative_intervals == 160 else (base.dry_time_s if authoritative_time_step == 1.0 else
1263             time_quarter.dry_time_s), (base.dry_time_s if authoritative_time_step == 1.0 else time_quarter.
1264             dry_time_s) / 3600.0])
1265         writer.writerow([320, authoritative_time_step, space_refined.dry_time_s, space_refined.
1266             dry_time_s / 3600.0])
1267         if authoritative_intervals == 640:
1268             writer.writerow([640, authoritative_time_step, authoritative.dry_time_s, authoritative.
1269             dry_time_s / 3600.0])
1270
1271     sensitivity_cases = {
1272         "温度_减1总体标准差": (
1273             boundary_temperature - stable["temperature_std_c"],
1274             boundary_moisture,
1275             HEAT_TRANSFER_COEFFICIENT,
1276             MASS_TRANSFER_COEFFICIENT,
1277         ),
1278         "温度_加1总体标准差": (
1279             boundary_temperature + stable["temperature_std_c"],
1280             boundary_moisture,
1281             HEAT_TRANSFER_COEFFICIENT,

```

```

1277     MASS_TRANSFER_COEFFICIENT,
1278 ),
1279     "空气水分_减1总体标准差": (
1280         boundary_temperature,
1281         boundary_moisture - stable["moisture_std_kg_per_kg"],
1282         HEAT_TRANSFER_COEFFICIENT,
1283         MASS_TRANSFER_COEFFICIENT,
1284     ),
1285     "空气水分_加1总体标准差": (
1286         boundary_temperature,
1287         boundary_moisture + stable["moisture_std_kg_per_kg"],
1288         HEAT_TRANSFER_COEFFICIENT,
1289         MASS_TRANSFER_COEFFICIENT,
1290     ),
1291     "附件末条记录": (
1292         stable["last_temperature_c"],
1293         stable["last_moisture_kg_per_kg"],
1294         HEAT_TRANSFER_COEFFICIENT,
1295         MASS_TRANSFER_COEFFICIENT,
1296     ),
1297     "端面通量上界": (
1298         boundary_temperature,
1299         boundary_moisture,
1300         1.08 * HEAT_TRANSFER_COEFFICIENT,
1301         1.08 * MASS_TRANSFER_COEFFICIENT,
1302     ),
1303 }
1304 sensitivity = []
1305 for case_name, (
1306     case_temperature,
1307     case_moisture,
1308     case_heat_coefficient,
1309     case_mass_coefficient,
1310 ) in sensitivity_cases.items():
1311     case = simulate_variable_model(
1312         model="q23",
1313         end_time_s=maximum_time_s,
1314         time_step_s=authoritative_time_step,
1315         intervals=authoritative_intervals,
1316         record_interval_s=3600.0,
1317         stop_at_dryness=True,
1318         temporal_scheme="bdf2",
1319         heat_transfer_coefficient=case_heat_coefficient,
1320         mass_transfer_coefficient=case_mass_coefficient,
1321         post_observation_temperature_c=case_temperature,
1322         post_observation_moisture=case_moisture,
1323     )
1324     if case.dry_time_s is None:

```

```

1325         raise RuntimeError(f"敏感性情景{case_name}缺少烘干时长")
1326     sensitivity.append(
1327         {
1328             "case": case_name,
1329             "temperature_c": case_temperature,
1330             "air_moisture_kg_per_kg": case_moisture,
1331             "heat_transfer_coefficient_w_per_m2_k": case_heat_coefficient,
1332             "mass_transfer_coefficient_m_per_s": case_mass_coefficient,
1333             "dry_time_s": case.dry_time_s,
1334             "dry_time_h": case.dry_time_s / 3600.0,
1335             "relative_change": (
1336                 case.dry_time_s - authoritative.dry_time_s
1337             )
1338             / authoritative.dry_time_s,
1339         }
1340     )
1341     with (RESULTS_DIR / "问题3_边界敏感性.csv").open(
1342         "w", encoding="utf-8-sig", newline=""
1343     ) as stream:
1344         fieldnames = list(sensitivity[0].keys())
1345         writer = csv.DictWriter(stream, fieldnames=fieldnames)
1346         writer.writeheader()
1347         writer.writerows(sensitivity)
1348
1349     end_face_case = next(item for item in sensitivity if item["case"] == "端面通量上界")
1350     if abs(float(end_face_case["relative_change"])) > 0.08:
1351         raise RuntimeError("端面通量上界情景超过8%，必须回退二维圆柱模型")
1352     condition_numbers = final_system_condition_numbers(
1353         authoritative,
1354         time_step_s=authoritative_time_step,
1355         heat_transfer_coefficient=HEAT_TRANSFER_COEFFICIENT,
1356         mass_transfer_coefficient=MASS_TRANSFER_COEFFICIENT,
1357         post_observation_temperature_c=boundary_temperature,
1358         post_observation_moisture=boundary_moisture,
1359     )
1360     if max(condition_numbers.values()) > 1.0e8:
1361         raise RuntimeError("问题三末时刻离散系统条件数超过合同阈值")
1362
1363     summary = {
1364         "mode": "q3_full",
1365         "inputs": {
1366             str(AIR_PATH.relative_to(PROJECT_ROOT)): sha256_file(AIR_PATH),
1367             str((TEMPLATE_DIR / "result3.xlsx").relative_to(PROJECT_ROOT)): sha256_file(TEMPLATE_DIR
1368 / "result3.xlsx"),
1369         },
1370         "stable_air": stable,
1371         "parameters": {
1372             "temporal_scheme": "backward_euler_start_then_bdf2",

```

```

1372     "authoritative_time_step_s": authoritative_time_step,
1373     "authoritative_radial_intervals": authoritative_intervals,
1374     "record_interval_s": 60.0,
1375     "drying_threshold_kg_per_kg": DRYING_THRESHOLD,
1376     "heat_transfer_coefficient_w_per_m2_k": HEAT_TRANSFER_COEFFICIENT,
1377     "mass_transfer_coefficient_m_per_s": MASS_TRANSFER_COEFFICIENT,
1378     "coupling_temperature_tolerance_c": 1.0e-8,
1379     "coupling_moisture_tolerance_kg_per_kg": 1.0e-10,
1380     "coupling_max_iterations": 40,
1381 },
1382 "dry_time_s": authoritative.dry_time_s,
1383 "dry_time_h": authoritative.dry_time_s / 3600.0,
1384 "table_times_s": table_times,
1385 "requested_radii_cm": radii_cm,
1386 "moisture_kg_per_kg": np.round(table, 8).tolist(),
1387 "sensitivity": sensitivity,
1388 "verification": {
1389     "time_step_gate_passed": time_gate_passed,
1390     "time_step_dry_times_s": {
1391         "1.0": base.dry_time_s,
1392         "0.5": time_half.dry_time_s,
1393         "0.25": time_quarter.dry_time_s,
1394     },
1395     "time_base_to_reference_difference_s": time_base_difference,
1396     "time_half_to_reference_difference_s": time_half_difference,
1397     "space_refined_dry_time_s": space_refined.dry_time_s,
1398     "space_time_difference_s": space_time_difference_s,
1399     "space_time_relative_difference": space_time_relative_difference,
1400     "space_table_max_absolute_difference_kg_per_kg": space_table_difference,
1401     "space_fine_relative_difference": fine_relative_difference,
1402     "maximum_recorded_moisture_increase_kg_per_kg": maximum_recorded_increase,
1403     "moisture_balance_relative_error": authoritative.moisture_balance_relative_error,
1404     "max_coupling_iterations": authoritative.max_coupling_iterations,
1405     "max_temperature_linear_residual": authoritative.max_temperature_linear_residual,
1406     "max_moisture_linear_residual": authoritative.max_moisture_linear_residual,
1407     "final_system_condition_numbers": condition_numbers,
1408 },
1409 "runtime_seconds": time.perf_counter() - run_started,
1410 "environment": {
1411     "python": platform.python_version(),
1412     "numpy": np.__version__,
1413     "scipy": __import__("scipy").__version__,
1414     "openpyxl": openpyxl.__version__,
1415     "platform": platform.platform(),
1416 },
1417 }
1418 (RESULTS_DIR / "问题3_结果摘要.json").write_text(
1419     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"

```

```

1420 )
1421 print(json.dumps(summary, ensure_ascii=False, indent=2))
1422
1423
1424 def run_q4() -> None:
1425     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1426     maximum_time_s = 7.0 * 24.0 * 3600.0
1427     authoritative = simulate_variable_model(
1428         model="q4",
1429         end_time_s=maximum_time_s,
1430         time_step_s=5.0,
1431         intervals=160,
1432         record_interval_s=60.0,
1433         stop_at_dryness=True,
1434     )
1435     time_coarse = simulate_variable_model(
1436         model="q4",
1437         end_time_s=maximum_time_s,
1438         time_step_s=10.0,
1439         intervals=160,
1440         record_interval_s=60.0,
1441         stop_at_dryness=True,
1442     )
1443     space_coarse = simulate_variable_model(
1444         model="q4",
1445         end_time_s=maximum_time_s,
1446         time_step_s=5.0,
1447         intervals=80,
1448         record_interval_s=60.0,
1449         stop_at_dryness=True,
1450     )
1451     write_result4(authoritative, RESULTS_DIR / "result4.xlsx")
1452     if authoritative.dry_time_s is None:
1453         raise RuntimeError("问题四未返回干燥时长")
1454     six_hour_times = list(
1455         np.arange(6.0 * 3600.0, authoritative.dry_time_s, 6.0 * 3600.0)
1456     )
1457     table_times = six_hour_times + [authoritative.dry_time_s]
1458     summary_rows = []
1459     for time_s in table_times:
1460         index = int(np.argmax(np.abs(authoritative.time_s - time_s)))
1461         current_radius_cm = 100.0 * authoritative.radius_m[index]
1462         requested = [
1463             radius for radius in (0.0, 0.5, 1.0, 1.5) if radius < current_radius_cm
1464         ]
1465         values = interpolate_at_physical_radai(
1466             authoritative, authoritative.moisture, index, requested
1467         ).tolist()

```

```

1468     summary_rows.append(
1469         {
1470             "time_s": time_s,
1471             "radius_cm": current_radius_cm,
1472             "fixed_radii_cm": requested,
1473             "fixed_values": values,
1474             "surface_value": float(authoritative.moisture[index, -1]),
1475         }
1476     )
1477     summary = {
1478         "mode": "q4_full",
1479         "inputs": {
1480             str(AIR_PATH.relative_to(PROJECT_ROOT)): sha256_file(AIR_PATH),
1481             str(RADIUS_PATH.relative_to(PROJECT_ROOT)): sha256_file(RADIUS_PATH),
1482         },
1483         "dry_time_s": authoritative.dry_time_s,
1484         "dry_time_h": authoritative.dry_time_s / 3600.0,
1485         "table": summary_rows,
1486         "verification": {
1487             "time_coarse_dry_time_s": time_coarse.dry_time_s,
1488             "space_coarse_dry_time_s": space_coarse.dry_time_s,
1489             "time_step_difference_s": float(
1490                 abs(authoritative.dry_time_s - float(time_coarse.dry_time_s))
1491             ),
1492             "space_grid_difference_s": float(
1493                 abs(authoritative.dry_time_s - float(space_coarse.dry_time_s))
1494             ),
1495             "moisture_balance_relative_error": authoritative.moisture_balance_relative_error,
1496             "max_coupling_iterations": authoritative.max_coupling_iterations,
1497         },
1498     }
1499     (RESULTS_DIR / "问题4_结果摘要.json").write_text(
1500         json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
1501     )
1502     print(json.dumps(summary, ensure_ascii=False, indent=2))
1503
1504
1505 def main() -> None:
1506     parser = argparse.ArgumentParser(description="2026 CUMCM A题全程干燥求解")
1507     parser.add_argument(
1508         "--mode",
1509         choices=("smoke", "q3-smoke", "q2", "q3", "q4", "all"),
1510         default="smoke",
1511     )
1512     arguments = parser.parse_args()
1513     if arguments.mode == "smoke":
1514         run_smoke()
1515     elif arguments.mode == "q3-smoke":

```

```

1516     run_q3_smoke()
1517     elif arguments.mode == "q2":
1518         run_q2()
1519     elif arguments.mode == "q3":
1520         run_q3()
1521     elif arguments.mode == "q4":
1522         run_q4()
1523     else:
1524         run_q2()
1525         run_q3()
1526         run_q4()
1527
1528
1529 if __name__ == "__main__":
1530     main()

```

2.3 问题四求解与复现程序

```

1 """Q4: material-coordinate effective diffusion; see 题目分析报告.md.
2
3 Reproduce: python -X utf8 问题4_复现.py
4 Inputs are read-only. Numba is optional (plain Python uses the same functions).
5 """
6 from __future__ import annotations
7 import sys, json, argparse, time, hashlib
8 from pathlib import Path
9
10 ROOT=Path(__file__).resolve().parent
11 sys.path.insert(0, str(ROOT/'build/q4_runtime'))
12
13 import numpy as np
14 import pandas as pd
15 import openpyxl
16 from scipy.linalg import eigvalsh_tridiagonal
17
18 try:
19     from numba import njit
20     ACCELERATED=True
21 except ImportError:
22     ACCELERATED=False
23     def njit(*args,**kwargs):
24         return lambda f:f
25 import 全程干燥求解 as old
26
27 @njit(cache=True)
28 def geometry(x, xf, R, a):
29     g=(1-R/.02)/(1-.01198/.02)

```

```

30     r=R*(x+a*g*x*(1-x))
31     rf=R*(xf+a*g*xf*(1-xf))
32     w=(rf[1:]*2-rf[:-1]**2)/2
33     geom=rf[1:-1]/(r[1:]-r[:-1])
34     return r, w, geom
35
36 @njit(cache=True)
37 def props(c, t):
38     rho=760+90*c
39     cp=1850+2150*c/(1+c)
40     k=.12+.20*c/(1+c)
41     d=4.2e-4*np.exp(-.30/c)*np.exp(-3850/(t+273.15))
42     return rho*cp, k, d
43
44 @njit(cache=True)
45 def linear_solve(history, b, gamma, w, geom, R, beta, ambient, dt):
46     st=b*w/dt
47     conduct=geom*(gamma[:-1]+gamma[1:])/2
48     diag=st.copy()
49     diag[:-1]+=conduct
50     diag[1:]+=conduct
51     diag[-1]+=R*beta
52     rhs=st*history
53     rhs[-1]+=R*beta*ambient
54     dd=diag.copy(); yy=rhs.copy()
55     for i in range(1, len(dd)):
56         factor=-conduct[i-1]/dd[i-1]
57         dd[i]+=factor*conduct[i-1]
58         yy[i]-=factor*yy[i-1]
59     u=yy.copy(); u[-1]/=dd[-1]
60     for i in range(len(u)-2, -1, -1):
61         u[i]=(yy[i]+conduct[i]*u[i+1])/dd[i]
62     residual=diag*u-rhs
63     residual[:-1]-=conduct*u[1:]
64     residual[1:]-=conduct*u[:-1]
65     relative=np.max(np.abs(residual))/max(np.max(np.abs(rhs)), 1e-30)
66     return u, relative
67
68 @njit(cache=True, nogil=True)
69 def kernel(N, dt, end, ta, ca, rt, rv, airtime, stableT, stableC, a, fixed, stop, h=25., hm=8e-7, initT=28., initC
    =2.55):
70     x=np.linspace(0., 1., N+1)
71     xf=np.empty(N+2); xf[0]=0.; xf[-1]=1.; xf[1:-1]=(x[:-1]+x[1:])/2
72     t=np.full(N+1, initT); c=np.full(N+1, initC); tp=t.copy(); cp=c.copy()
73     count=int(round(end/dt)); stride=int(round(60/dt))
74     capacity=count//stride+3
75     times=np.zeros(capacity); radii=np.zeros(capacity)
76     ts=np.zeros((capacity, N+1)); cs=np.zeros((capacity, N+1))

```

```

77 ts[0]=t;cs[0]=c;radii[0]=.02;rec=1
78 iters=np.zeros(count,dtype=np.int16)
79 # residualT,residualC,balanceC,balanceT_abs,balanceT_scaled,
80 # minC,minT,maxT,max radial C increase,max(Cmax-Ccenter),minJac,
81 # previous Cmax,final Cmax,drytime,max temporal Cmax increase
82 stats=np.array([0.,0.,0.,0.,0.,0.,initC,initT,initT,0.,0.,.02,initC,initC,-1.,0.])
83 lastmax=initC
84 for step in range(1,count+1):
85     now=step*dt
86     R=.02 if fixed else np.interp(now,rt,rv)
87     ambT=np.interp(now,airtime,ta) if now<=airtime[-1] else stableT
88     ambC=np.interp(now,airtime,ca) if now<=airtime[-1] else stableC
89     r,w,geom=geometry(x,xf,R,a)
90     g=(1-R/.02)/(1-.01198/.02)
91     stats[10]=min(stats[10],R*(1-abs(a*g)))
92     effective=dt if step==1 else 2*dt/3
93     th=t.copy() if step==1 else (4*t-tp)/3
94     ch=c.copy() if step==1 else (4*c-cp)/3
95     tg=t.copy();cg=c.copy();converged=False
96     for iteration in range(1,41):
97         b,k,_=props(cg,tg)
98         tn,errT=linear_solve(th,b,k,w,geom,R,h,ambT,effective)
99         _,_,d=props(cg,tn)
100         cn,errC=linear_solve(ch,np.ones(N+1),d,w,geom,R,hm,ambC,effective)
101         stats[0]=max(stats[0],errT);stats[1]=max(stats[1],errC)
102         et=np.max(np.abs(tn-tg));ec=np.max(np.abs(cn-cg))
103         tg=tn;cg=cn
104         if et<=1e-8 and ec<=1e-10:
105             converged=True;break
106     if not converged:raise RuntimeError('Picard did not converge')
107     if not np.all(np.isfinite(cg)) or not np.all(np.isfinite(tg)) or np.min(cg)<0:
108         raise RuntimeError('invalid field')
109     deltaC=(cg-ch)/effective
110     lhs=np.sum(w*deltaC);flux=R*hm*(cg[-1]-ambC)
111     # Roundoff floor only for synthetic zero-flux constant-state test.
112     if max(abs(lhs),abs(flux))>1e-20:
113         stats[2]=max(stats[2],abs(lhs+flux)/max(abs(lhs),abs(flux)))
114     deltaT=(tg-th)/effective
115     heatlhs=np.sum(b*w*deltaT);heatflux=R*h*(tg[-1]-ambT)
116     habs=abs(heatlhs+heatflux)
117     hscale=np.sum(b*w*(np.abs(tg)+np.abs(th))/effective)+R*h*(abs(tg[-1])+abs(ambT))
118     stats[3]=max(stats[3],habs);stats[4]=max(stats[4],habs/max(hscale,1e-30))
119     stats[5]=min(stats[5],np.min(cg));stats[6]=min(stats[6],np.min(tg));stats[7]=max(stats[7],np
120 .max(tg))
121     stats[8]=max(stats[8],np.max(cg[1:]-cg[:-1]))
122     cmax=np.max(cg);stats[9]=max(stats[9],cmax-cg[0])
123     stats[11]=lastmax;stats[12]=cmax;stats[14]=max(stats[14],cmax-lastmax)
    lastmax=cmax

```

```

124         tp=t;t=tg;cp=c;c=cg
125         iters[step-1]=iteration
126         dry=stop and cmax<.15
127         if step%stride==0 or dry or step==count:
128             times[rec]=now;radii[rec]=R;ts[rec]=t;cs[rec]=c;rec+=1
129         if dry:
130             stats[13]=now;break
131     if stop and stats[13]<0:raise RuntimeError('dryness not reached by horizon')
132     return times[:rec], radii[:rec], ts[:rec], cs[:rec], iters[:step], stats
133
134 def input_data():
135     air=old.core.read_air_boundary();rad=old.read_radius_history();ss=old.stable_air_statistics()
136     assert np.all(np.isfinite(rad.radius_m))
137     return air, rad, ss
138
139 def solve(*, N=320, dt=1., a=0., fixed=False, end=604800., stop=True, Tshift=0., Cshift=0., last_air=False, h
    =25., hm=8e-7):
140     air, rad, ss=input_data()
141     st=ss['last_temperature_c'] if last_air else ss['temperature_mean_c']+Tshift
142     sc=ss['last_moisture_kg_per_kg'] if last_air else ss['moisture_mean_kg_per_kg']+Cshift
143     start=time.perf_counter()
144     values=kernel(N, dt, end, air.temperature_c, air.moisture, rad.time_s, rad.radius_m, air.time_s, st, sc, a
        , fixed, stop, h, hm)
145     t, r, T, C, it, stats=values
146     keys=['heat_linear_residual', 'moisture_linear_residual', 'moisture_balance_relative', '
        heat_balance_absolute', 'heat_balance_scaled', 'min_moisture', 'min_temperature_c', '
        max_temperature_c', 'max_radial_moisture_increase', 'max_Cmax_minus_center', '
        min_mapping_Jacobian_m', 'previous_Cmax', 'final_Cmax', 'dry_time_s', 'max_step_Cmax_increase']
147     meta=dict(zip(keys, map(float, stats)))
148     meta.update(N=N, dt_s=dt, a=a, fixed=fixed, Tshift=Tshift, Cshift=Cshift, last_air=last_air,
149         h_W_m2_K=h, hm_m_s=hm, runtime_s=time.perf_counter()-start,
150         max_picard_iterations=int(it.max()), stable_air=ss)
151     meta['dry_time_h']=meta['dry_time_s']/3600
152     meta['radius_end_cm']=float(r[-1]*100)
153     meta['beyond_radius_observation']=bool(t[-1]>rad.time_s[-1])
154     meta['final_matrix_conditions']=conditions(N, dt, r[-1], a, T[-1], C[-1], h, hm)
155     return dict(time_s=t, radius_m=r, temperature_c=T, moisture=C, iterations=it, meta=meta)
156
157 def conditions(N, dt, R, a, T, C, h=25., hm=8e-7):
158     x=np.linspace(0, 1, N+1);xf=np.r_[0, (x[:-1]+x[1:])/2, 1]
159     _, w, geom=geometry(x, xf, R, a);b, k, d=props(C, T)
160     out={}
161     for name, B, G, beta in [('heat', b, k, h), ('moisture', np.ones(N+1), d, hm)]:
162         f=geom*(G[:-1]+G[1:])/2;diag=1.5*B*w/dt
163         diag[:-1]+=f;diag[1:]+=f;diag[-1]+=R*beta
164         ev=eigvalsh_tridiagonal(diag, -f)
165         out[name]=float(ev[-1]/ev[0])
166     return out

```

```

167
168 def sample(sol, spacing=.005):
169     fixed=np.arange(0,.02-1e-10, spacing)
170     c=sol['moisture'];R=sol['radius_m'];a=sol['meta']['a']
171     x=np.linspace(0,1,c.shape[1]);out=np.full((len(R),len(fixed)+1),np.nan)
172     for j,rad in enumerate(R):
173         g=(1-rad/.02)/(1-.01198/.02);r=rad*(x+a*g*x*(1-x))
174         mask=fixed<=rad+1e-12
175         out[j,:len(fixed)][mask]=np.interp(fixed[mask],r,c[j])
176         out[j,-1]=c[j,-1]
177     return out
178
179 def save_run(sol, path):
180     path.parent.mkdir(parents=True, exist_ok=True)
181     np.savez_compressed(path, **{k:v for k,v in sol.items() if k!='meta'})
182     path.with_suffix('.json').write_text(json.dumps(sol['meta'], indent=2, ensure_ascii=False),
183                                           encoding='utf-8')
184
185 def load_run(path):
186     with np.load(path) as z:out={k:z[k] for k in z.files}
187     out['meta']=json.loads(path.with_suffix('.json').read_text(encoding='utf-8'))
188     return out
189
190 def smoke(output):
191     s=solve(N=32, dt=1, end=600, stop=False)
192     ss=s['meta']['stable_air']
193     ref=old.simulate_variable_model(model='q4', end_time_s=600, time_step_s=1, intervals=32,
194     record_interval_s=60, temporal_scheme='bdf2', post_observation_temperature_c=ss['
195     temperature_mean_c'], post_observation_moisture=ss['moisture_mean_kg_per_kg'])
196     errT=float(np.max(np.abs(ref.temperature_c-s['temperature_c'])))
197     errC=float(np.max(np.abs(ref.moisture-s['moisture'])))
198     assert errT<1e-9 and errC<1e-10, (errT, errC)
199     pert=solve(N=32, dt=1, end=1800, a=.2, stop=False)
200     pert_negative=solve(N=32, dt=1, end=1800, a=-.2, stop=False)
201     h_low=solve(N=32, dt=1, end=1800, h=20., stop=False)
202     h_high=solve(N=32, dt=1, end=1800, h=30., stop=False)
203     hm_low=solve(N=32, dt=1, end=1800, hm=6.4e-7, stop=False)
204     hm_high=solve(N=32, dt=1, end=1800, hm=9.6e-7, stop=False)
205     air, rad, _=input_data()
206     const=kernel(16, 1., 600., np.full_like(air.temperature_c, 28.), np.full_like(air.moisture, 2.55), rad.
207     time_s, rad.radius_m, air.time_s, 28., 2.55, .2, False, False)
208     constant_error=max(float(np.max(np.abs(const[2]-28))), float(np.max(np.abs(const[3]-2.55))))
209     assert constant_error<1e-9
210     fixed=solve(N=32, dt=1, end=1800, fixed=True, stop=False)
211     assert np.all(fixed['radius_m']==.02)
212     for z in [s, pert, pert_negative, h_low, h_high, hm_low, hm_high, fixed]:
213         m=z['meta'];assert m['moisture_balance_relative']<1e-8 and m['min_mapping_Jacobian_m']>0
214         assert max(m['heat_linear_residual'], m['moisture_linear_residual'])<1e-12

```

```

211 # At this short horizon the centre value is almost unchanged; compare the
212 # volume-profile mean to verify that each boundary coefficient is active.
213 assert h_high['moisture'][-1].mean() < h_low['moisture'][-1].mean()
214 assert hm_high['moisture'][-1].mean() < hm_low['moisture'][-1].mean()
215 output.parent.mkdir(parents=True, exist_ok=True)
216 report={'status': 'PASS', 'uniform_regression_temperature_error': errT,
217         'uniform_regression_moisture_error': errC, 'moving_constant_state_error': constant_error,
218         'base': s['meta'], 'shape_02': pert['meta'], 'shape_minus_02': pert_negative['meta'],
219         'h_low': h_low['meta'], 'h_high': h_high['meta'],
220         'hm_low': hm_low['meta'], 'hm_high': hm_high['meta'], 'fixed': fixed['meta']}
221 output.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding='utf-8')
222 print(json.dumps(report, ensure_ascii=False, indent=2))
223
224 if __name__ == '__main__':
225     p = argparse.ArgumentParser(); p.add_argument('--smoke', action='store_true'); p.add_argument('--
226     output', default='results/P1_q4_smoke.json'); args = p.parse_args()
227     if args.smoke: smoke(ROOT/args.output)
228     else: p.error('Use 问题4_复现.py for full computation, or --smoke')

```

```

1 """Unique Q4 pipeline. --output-root isolates independent reproduction."""
2 from __future__ import annotations
3 import argparse, json, sys, time, shutil, subprocess, importlib.metadata
4 from concurrent.futures import ThreadPoolExecutor
5 from pathlib import Path
6 import numpy as np
7 import pandas as pd
8 import openpyxl
9 import 问题4_求解 as q
10
11 ROOT = q.ROOT
12 FIGURE_TOOLS = ROOT/'figure_tools'
13 REPRO_MANIFEST = ROOT/'repro_manifest.py'
14
15 def difference(s, t):
16     times = np.intersect1d(s['time_s'], t['time_s'])
17     times = times[(times > 0) & (times % 21600 == 0)]
18     sc = q.sample(s); tc = q.sample(t)
19     si = np.searchsorted(s['time_s'], times); ti = np.searchsorted(t['time_s'], times)
20     return float(np.nanmax(np.abs(sc[si] - tc[ti])))
21
22 def validate_run(s):
23     m = s['meta']
24     assert m['previous_Cmax'] >= .15 * m['final_Cmax']
25     assert m['moisture_balance_relative'] <= 1e-8, m
26     assert m['heat_balance_scaled'] <= 1e-12
27     assert max(m['heat_linear_residual'], m['moisture_linear_residual']) <= 1e-12
28     assert max(m['final_matrix_conditions'].values()) <= 1e8

```

```

29     assert m['min_moisture']>=0 and m['min_temperature_c']>=28-1e-8 and m['max_temperature_c']<=50.5
30     assert m['min_mapping_Jacobian_m']>0
31     assert m['max_radial_moisture_increase']<1e-8 and m['max_step_Cmax_increase']<1e-8
32
33 def write_result4_workbook(base, out):
34     res=out/'results';res.mkdir(parents=True,exist_ok=True)
35     t=base['time_s'];R=base['radius_m'];C=base['moisture'];T=base['temperature_c'];x=np.linspace
36     (0,1,C.shape[1])
37     arr=q.sample(base,.001)
38     wb=openpyxl.load_workbook(q.old.TEMPLATE_DIR/'result4.xlsx');ws=wb.active
39     # Template contains illustrative ellipses, expanded into actual data columns.
40     for row in ws:
41         for cell in row:cell.value=None
42     headers=['时间\\到药材中心的距离']+[round(j*.1,1) for j in range(20)]+['药材表面']
43     for j,v in enumerate(headers,1):ws.cell(1,j,v)
44     for i,tt in enumerate(t[1:],2):
45         ws.cell(i,1,float(tt))
46         for j,v in enumerate(arr[i-1],2):
47             cell=ws.cell(i,j,None if np.isnan(v) else round(float(v),4));cell.number_format='0.0000'
48     ws.freeze_panes='B2';wb.save(res/'result4.xlsx');wb.close()
49     # Re-open and verify every value, mask and numeric format.
50     wb=openpyxl.load_workbook(res/'result4.xlsx',data_only=True);ws=wb.active
51     assert ws.max_row==len(t) and ws.max_column==22
52     for i in range(2,len(t)+1):
53         assert ws.cell(i,1).value==t[i-1]
54         for j,expected in enumerate(arr[i-1],2):
55             val=ws.cell(i,j).value
56             rounded=None if np.isnan(expected) else round(float(expected),4)
57             assert (val is None) if rounded is None else (val==rounded and ws.cell(i,j).
58                 number_format=='0.0000')
59     wb.close()
60     return t,R,C,T,arr
61
62 def write_outputs(base,runs,out,verification):
63     res=out/'results';res.mkdir(parents=True,exist_ok=True)
64     t,R,C,T,arr=write_result4_workbook(base,out)
65     mask=(t>0)&((t%21600==0)|(t==t[-1]));table=q.sample(base)[mask]
66     df=pd.DataFrame(table,columns=['r0_cm','r0.5_cm','r1_cm','r1.5_cm','surface'])
67     df.insert(0,'radius_cm',R[mask]*100);df.insert(0,'time_h',t[mask]/3600);df.insert(0,'time_s',t[
68         mask])
69     df.to_csv(res/'问题4_表6含水率.csv',index=False,encoding='utf-8-sig')
70     process=pd.DataFrame({'time_s':t,'radius_cm':R*100,'Cmax':C.Cmax(axis=1),'Ccenter':C[:,0],
71         'Csurface':C[:,-1],'Tcenter_C':T[:,0],'Tsurface_C':T[:,-1]})
72     process.to_csv(res/'问题4_关键过程.csv',index=False)
73     rows=[]
74     for name,s in runs.items():
75         m=s['meta'];rows.append({'scenario':name,'N':m['N'],'dt_s':m['dt_s'],'a':m['a'],'fixed':m[
76             'fixed'],

```

```

72         'h_W_m2_K':m['h_W_m2_K'], 'hm_m_s':m['hm_m_s'],
73         'dry_time_s':m['dry_time_s'], 'dry_time_h':m['dry_time_h'],
74         'relative_to_main_percent':100*(m['dry_time_s']/t[-1]-1),
75         'radius_end_cm':m['radius_end_cm'], 'runtime_s':m['runtime_s']})
76 pd.DataFrame(rows).to_csv(res/'问题4_情景汇总.csv', index=False, encoding='utf-8-sig')
77 pd.DataFrame([{'scenario':k,**v} for k,v in verification.items()]).to_csv(res/'问题4_数值加密.
78 csv', index=False)
79 unique, counts=np.unique(base['iterations'], return_counts=True)
80 pd.DataFrame({'iterations':unique, 'step_count':counts}).to_csv(res/'问题4_迭代统计.csv', index=
81 False)
82 fixed=runs['fixed']['meta']['dry_time_s']
83 etaR=100*(t[-1]-fixed)/fixed
84 shape_values=[-.2, -.1, .1, .2]
85 etaA={str(a):100*(runs[f'shape_{a}']['meta']['dry_time_s']/t[-1]-1) for a in shape_values}
86 etaH={str(f):100*(runs[f'h_{f}']['meta']['dry_time_s']/t[-1]-1) for f in [.8, 1.2]}
87 etaHm={str(f):100*(runs[f'hm_{f}']['meta']['dry_time_s']/t[-1]-1) for f in [.8, 1.2]}
88 sensitivity=[]
89 for name, parameter, multiplier, value in [
90     ('main', '基准', 1., 0.),
91     ('shape_-0.2', 'a^*', -1., -.2), ('shape_-0.1', 'a^*', -.5, -.1),
92     ('shape_0.1', 'a^*', .5, .1), ('shape_0.2', 'a^*', 1., .2),
93     ('h_0.8', 'h', .8, 20.), ('h_1.2', 'h', 1.2, 30.),
94     ('hm_0.8', 'hm', .8, 6.4e-7), ('hm_1.2', 'hm', 1.2, 9.6e-7)]:
95     s=base if name=='main' else runs[name]
96     sensitivity.append({'scenario':name, 'parameter':parameter, 'multiplier':multiplier,
97         'parameter_value':value, 'dry_time_s':s['meta']['dry_time_s'],
98         'dry_time_h':s['meta']['dry_time_h'],
99         'relative_to_main_percent':100*(s['meta']['dry_time_s']/t[-1]-1)})
100 pd.DataFrame(sensitivity).to_csv(res/'问题4_边界参数敏感性.csv', index=False, encoding='utf-8-sig'
101 )
102 summary={**base['meta'], 'fixed_radius_dry_time_s':fixed, 'fixed_radius_dry_time_h':fixed/3600,
103     'delta_geometry_s':float(t[-1]-fixed), 'eta_geometry_percent':etaR,
104     'eta_shape_percent':etaA, 'E_shape_percent':max(map(abs, etaA.values())),
105     'eta_h_percent':etaH, 'E_h_percent':max(map(abs, etaH.values())),
106     'eta_hm_percent':etaHm, 'E_hm_percent':max(map(abs, etaHm.values())),
107     'boundary_sensitivity':sensitivity, 'verification':verification,
108     'excel_rows':len(t)-1, 'excel_columns':22,
109     'structural_empty_cells':int(np.isnan(arr[1:]).sum()),
110     'table6':df.to_dict(orient='records'),
111     'model_scope':'Q4 prescribed effective material-scalar diffusion, not identified solid-
112     mass conservation'}
113 # Serialize structural nulls as JSON null, never as invalid JSON NaN.
114 clean=json.loads(json.dumps(summary, ensure_ascii=False).replace('NaN', 'null'))
115 (res/'问题4_结果摘要.json').write_text(json.dumps(clean, indent=2, ensure_ascii=False, allow_nan=
116 False), encoding='utf-8')
117 robust=out/'robustness'/ 'Q4';robust.mkdir(parents=True, exist_ok=True)
118 robust_summary={
119     'question':'Q4', 'status':'PASS', 'baseline_dry_time_s':float(t[-1]),

```

```

115     'one_factor_at_a_time': True,
116     'shape_parameter_values': shape_values, 'h_factors': [.8, 1.2], 'hm_factors': [.8, 1.2],
117     'eta_shape_percent': etaA, 'eta_h_percent': etaH, 'eta_hm_percent': etaHm,
118     'E_shape_percent': max(map(abs, etaA.values())),
119     'E_h_percent': max(map(abs, etaH.values())),
120     'E_hm_percent': max(map(abs, etaHm.values())),
121     'interpretation': 'Each parameter is perturbed alone and the full Q4 PDE is re-solved to the
endpoint.' }
122 (robust/'q4_robustness_summary.json').write_text(
123     json.dumps(robust_summary, indent=2, ensure_ascii=False), encoding='utf-8')
124 report=(
125     '# 问题四稳健性与敏感性复算报告\n\n'
126     f'- 基准终点: {t[-1]/3600:.6f} h.\n'
127     f'- 结构参数:  $a^* = -0.2, -0.1, 0.1, 0.2$ , 终点最大相对变化 {max(map(abs, etaA.values())):.6f}%.\n'
128     f'- 换热系数:  $h$  单因素上下浮动 20%, 终点最大相对变化 {max(map(abs, etaH.values())):.6f}%.\n'
129     f'- 传质系数:  $h_m$  单因素上下浮动 20%, 终点最大相对变化 {max(map(abs, etaHm.values())):.6f}%.\n'
130     '- 以上各情景均重新求解完整问题四偏微分方程至终点, 并非只对基准结果作比例换算.\n')
131 (robust/'q4_robustness_report.md').write_text(report, encoding='utf-8')
132 return summary
133
134 def run_all(out, plots=True):
135     out=out.resolve(); res=out/'results'; runpath=res/'q4_runs'; runpath.mkdir(parents=True, exist_ok=
True)
136     total=time.perf_counter(); runs={}
137     def run(name, **kwargs):
138         print('Computing', name, kwargs, flush=True); s=q.solve(**kwargs); validate_run(s)
139         q.save_run(s, runpath/(name+'.npz'))
140         print('Finished', name, s['meta']['dry_time_s'], s['meta']['runtime_s'], flush=True)
141         return s
142     N=320; base=run('main_320', N=N)
143     fine=run('space_640', N=2*N)
144     spatial_error=difference(base, fine); event_error=100*abs(base['time_s'][-1]/fine['time_s'][-1]-1)
145     verification={'space_320_640': {'table_max_abs_kg_kg': spatial_error, 'event_difference_s': float(
base['time_s'][-1]-fine['time_s'][-1]), 'event_relative_percent': event_error}}
146     runs['main_320']=base; runs['space_640']=fine
147     if spatial_error>5e-5 or event_error>.1:
148         N=640; base=fine
149         finer=run('space_1280', N=1280); runs['space_1280']=finer
150         err=difference(base, finer); ev=100*abs(base['time_s'][-1]/finer['time_s'][-1]-1)
151         verification['space_640_1280']= {'table_max_abs_kg_kg': err, 'event_difference_s': float(base[
'time_s'][-1]-finer['time_s'][-1]), 'event_relative_percent': ev}
152         assert err<=5e-5 and ev<=.1, 'Spatial accuracy requires model review'
153     runs['main']=base; q.save_run(base, runpath/'main.npz')
154     ss=base['meta']['stable_air']
155     cases=[('half_step', {'dt': .5}), ('fixed', {'fixed': True})],

```

```

156         ('shape_0.2', {'a': -.2}), ('shape_0.1', {'a': -.1}),
157         ('shape_0.1', {'a': .1}), ('shape_0.2', {'a': .2}),
158         ('h_0.8', {'h': 20.}), ('h_1.2', {'h': 30.}),
159         ('hm_0.8', {'hm': 6.4e-7}), ('hm_1.2', {'hm': 9.6e-7}),
160         ('air_T_minus', {'Tshift': -ss['temperature_std_c']}), ('air_T_plus', {'Tshift': ss['
temperature_std_c']}),
161         ('air_C_minus', {'Cshift': -ss['moisture_std_kg_per_kg']}), ('air_C_plus', {'Cshift': ss['
moisture_std_kg_per_kg']}),
162         ('last_air', {'last_air': True}])
163 # Independent frozen scenarios; two workers bound peak memory and CPU use.
164 with ThreadPoolExecutor(max_workers=2) as pool:
165     futures=[(name, pool.submit(run, name, N=N, **kw)) for name, kw in cases]
166     for name, future in futures: runs[name]=future.result()
167 half=runs['half_step']; err=difference(base, half); ev=float(base['time_s'][-1]-half['time_s'][-1])
168 verification['time_1_0.5']={'table_max_abs_kg_kg': err, 'event_difference_s': ev, '
event_relative_percent': 100*abs(ev)/half['time_s'][-1]}
169 assert abs(ev)<=1 and err<=5e-5, 'Time refinement failed'
170 assert runs['air_T_plus']['time_s'][-1]<runs['air_T_minus']['time_s'][-1]
171 assert runs['air_C_plus']['time_s'][-1]>runs['air_C_minus']['time_s'][-1]
172 summary=write_outputs(base, runs, out, verification)
173 if plots:
174     import 问题4_绘图 as plotting
175     plotting.main(out)
176 import 问题4_素材整理 as materials
177 materials.main(out)
178 manifest(out, summary)
179 (res/'问题4_执行状态.json').write_text(json.dumps({'status': 'computational_checks_passed', '
elapsed_s': time.perf_counter()-total, 'figures_generated': plots}, indent=2), encoding='utf-8')
180 print(json.dumps({k: summary[k] for k in ['N', 'dry_time_s', 'dry_time_h', 'fixed_radius_dry_time_h'
,
181                                     'eta_geometry_percent', 'eta_shape_percent', '
E_shape_percent',
182                                     'eta_h_percent', 'E_h_percent', 'eta_hm_percent', '
E_hm_percent',
183                                     'verification']}, ensure_ascii=False, indent=2), flush=
True)
184
185 def manifest(out, summary):
186     if (out/'results/复现清单.json').exists() and not (out/'results/复现清单_Q3历史.json').exists():
187         shutil.copy2(out/'results/复现清单.json', out/'results/复现清单_Q3历史.json')
188     inputs=[ROOT/'problem A/A题.pdf', q.old.AIR_PATH, q.old.RADIUS_PATH, q.old.TEMPLATE_DIR/'result4.
xlsx', ROOT/'Q4/第四问建模思路与公式整理_最终推荐版.tex', ROOT/'题目分析报告.md', ROOT/'术语表格.md
', ROOT/'问题4_求解.py', ROOT/'问题4_复现.py', ROOT/'问题4_绘图.py', ROOT/'问题4_素材整理.py', ROOT/'
全程干燥求解.py', ROOT/'药材烘干求解.py', ROOT/'utils/plot_style.py']
189     inputs += list(FigureTools.glob('*.*py'))
190     inputs.append(REPRO_MANIFEST)
191     cmd=[sys.executable, '-X', 'utf8', str(REPRO_MANIFEST), '--project-root', str(out), '--seed', '0', '--
parameters', json.dumps({'question': 4, 'N': summary['N'], 'dt_s': 1., 'refinement_dt_s': .5, '

```

```

192     shape_values': [-.2, -.1, 0, .1, .2], 'stable_start_s': 9000, 'stable_end_s': 14400, 'threshold': .15, 'h'
193     : 25., 'h_factors': [.8, 1.2], 'hm': 8e-7, 'hm_factors': [.8, 1.2]}}), '--command', 'python -X utf8 问题4_复
194     现.py', '--dependencies', json.dumps({'numba': '0.67.0', 'llvmlite': '0.49.0'}), '--overwrite']
195
196     for f in inputs: cmd += ['--input', str(f)]
197
198     for package in ['numpy', 'scipy', 'pandas', 'openpyxl', 'matplotlib']: cmd += ['--package', package]
199     subprocess.run(cmd, check=True)
200
201 if __name__ == '__main__':
202     parser = argparse.ArgumentParser(); parser.add_argument('--output-root', type=Path, default=ROOT);
203     parser.add_argument('--no-plots', action='store_true')
204     args = parser.parse_args(); run_all(args.output_root, not args.no_plots)

```

2.4 结果整理与绘图程序

```

1 """Generate auditable paper materials from frozen numerical output, without writing paper prose."""
2 from pathlib import Path
3 import json, sys, re
4 import numpy as np
5 import pandas as pd
6 import 问题4_求解 as q
7
8 def main(out=q.ROOT):
9     out = Path(out); folder = out/'Q4'; folder.mkdir(exist_ok=True); res = out/'results'
10    s = json.loads((res/'问题4_结果摘要.json').read_text(encoding='utf-8'))
11    table = pd.read_csv(res/'问题4_表6含水率.csv'); cases = pd.read_csv(res/'问题4_情景汇总.csv')
12    a = cases.set_index('scenario')
13    latex = [r'% 自动读取高精度结果生成；片段内容已同步入正文，但正文未通过\input直接引用本文件；需要
14    booktabs.', r'\begin{table}[htbp]', r'\centering', r'\caption{考虑尺寸收缩的药材含水率 (kg/kg)}',
15    r'\label{tab:q4-moisture}', r'\begin{tabular}{rrrrrr}', r'\toprule', r'时间/h & 半径/cm & 0 cm &
16    0.5 cm & 1.0 cm & 1.5 cm & 药材表面\\', r'\midrule']
17    rows = []
18    for _, r in table.iterrows():
19        vals = ['---' if pd.isna(v) else f'{v:.4f}' for v in r[['time_h', 'radius_cm', 'r0_cm', 'r0.5_cm',
20        'r1_cm', 'r1.5_cm', 'surface']]]
21        latex.append(' & '.join(vals) + r'\\')
22        rows.append('| ' + ' | '.join(vals) + ' |')
23    latex += [r'\bottomrule', r'\end{tabular}', r'\par\smallskip{\footnotesize 横线表示固定位置已位于
24    药材域外；末列独立记录当前表面。终点中心四位显示为0.1500，未舍入值严格小于0.15。}', r'\end{table}']
25    (folder/'表6_可插入.tex').write_text('\n'.join(latex), encoding='utf-8')
26    macros = {'QFourDryHours': s['dry_time_h'], 'QFourFixedHours': s['fixed_radius_dry_time_h'],
27    'QFourGeometryPercent': s['eta_geometry_percent'], 'QFourShapePercent': s['E_shape_percent'],
28    'QFourHeatSensitivityPercent': s['E_h_percent'],
29    'QFourMassSensitivityPercent': s['E_hm_percent']}
30    bs = chr(92)
31    (folder/'关键数值_可引用.tex').write_text('% 自动生成；数值已同步入正文，但正文未通过\input直接

```

```

引用本文件\n'+'\n'.join(f' {bs}newcommand{{{bs} {k}}}{v:.4f}}' for k,v in macros.items()),
encoding='utf-8')
evidence=[
    ('附录4物性','题面第4页; 问题4_求解.py props','existing eq:q4-properties',' $\rho$ , cp, k, D系数一致, Arrhenius温度用K'),
    ('相似收缩与表面条件','题目分析报告.md Q4; geometry/kernel','existing eq:moving-heat / moving-moisture / moving-boundaries','保留既有正确公式'),
    ('一般非均匀映射','题目分析报告.md Q4; geometry','eq:q4-nonuniform-map',' $r \xi > 0$ ; 同外半径a  $\ast=0, \pm 0.1, \pm 0.2$ '),
    ('固定半径反事实','results/q4_runs/fixed.json','eq:q4-geometry-result','模型内对照; 必须同附录4, 不得替用Q3'),
    ('正式干燥时间','results/问题4_结果摘要.json dry_time_s/h','tab:q4-moisture','全部节点每1秒检验严格<0.15'),
    ('表6','results/问题4_表6含水率.csv; Q4/表6_可插入.tex','tab:q4-moisture','浮点结果生成, 结构性空值保留'),
    ('完整结果Excel','results/result4.xlsx','待下一轮附录清单','60s间隔加终点; 22列、四位显示'),
    (' $\eta R$ ','results/问题4_结果摘要.json eta_geometry_percent','eq:q4-geometry-result','分母为同物性固定半径时长; 不作真实因果解释'),
    (' $\eta a$ 及Eshape','results/问题4_结果摘要.json eta_shape_percent/E_shape_percent','eq:q4-shape-error','双侧确定性结构扰动, 不是置信区间'),
    ('h、hm终点敏感性','results/问题4_边界参数敏感性.csv','tab:q4-transfer-sensitivity','分别 $\pm 20\%$ 单因素完整复算'),
    ('数值误差','results/问题4_数值加密.csv','eq:q4-refinement-error','空间与时间分别检验, 同6h表格位置'),
    ('离散平衡','results/q4_runs/main.json','existing eq:global-moisture-balance','有效方程残差, 不是物理质量守恒证明'),
    ('12幅候选图','figures/q4/图表契约.json 与图表说明.md','已选图入正文, 其余保留为候选','各图PDF/SVG/PNG, 灰度在_qa'),
]
pd.DataFrame(evidence, columns=['claim', 'evidence', 'paper_location', 'check']).to_csv(res/'问题4_论文核对.csv', index=False, encoding='utf-8-sig')
md=['# 问题四主张—证据与论文核对', '', '本轮已把固定半径反事实、双侧结构扰动及边界传递系数敏感性并入论文源文件。', '', '| 主张 | 权威证据 | 论文落点/状态 | 核验口径 |', '|---|---|---|---|']
md += ['| '+ '| '.join(row)+' |' for row in evidence]
md += ['', '## 本轮论文修正', '', '入口为完整论文-LaTeX/example.tex, 根目录example.tex是模板。本轮把固定半径结果改写为模型内反事实, 不再表述为真实独立因果效应; 把a*扩展为 $\pm 0.1$ 、 $\pm 0.2$ 双侧结构扰动; 新增h、hm分别 $\pm 20\%$ 的终点时长敏感性。旧PDF和压缩包未重新编译或导出, 不能当作当前源码版本。', '', '## 口径限制', '', '1.  $\rho$ 只用于题设热容量, 不由 $\rho/(1+C)$ 反演真实干固体运动; 不声称完整多相质量守恒。', '2. 热平衡使用合并历史状态的绝对量作为归一分母, 比逐项历史绝对值和更严格; 记录中同时给出绝对残差。', '3. 结构情景a*= $\pm 0.1$ 、 $\pm 0.2$ 不是材料参数估计; h、hm的 $\pm 20\%$ 是确定性单因素情景, 不是置信区间。固定半径反事实是第四题物性下人为关闭收缩的解释实验, 不是正式答案, 也不作真实因果解释。', '4. N=320/640事件相差13s, 主步长1s只是事件分辨率, 不是物理或空间误差界。', '5. 长期空气边界延拓仍是假设; 有效模型未验证端面通量和显式蒸发潜热。', '', '## 文献素材', '', 'Adrover A, Brasiello A, Ponso G. A moving boundary model for food isothermal drying and shrinkage: General setting. Journal of Food Engineering, 2019, 244:178–191. DOI:10.1016/j.jfoodeng.2018.09.018。作者机构原始记录: https://iris.uniroma1.it/handle/11573/1156871。只用于方法背景, 不冒称本题映射已获实验验证。', 'Adrover A, Brasiello A. A moving boundary model for food isothermal drying and shrinkage: One-dimensional versus two-dimensional approaches. Journal of Food Process

```

Engineering, 2019, 42(6):e13178. DOI:10.1111/jfpe.13178. <https://onlinelibrary.wiley.com/doi/10.1111/jfpe.13178>。提示一维近似局限，不能把几何面积比当严格误差证明。’，’’，’双引擎原始日志位于build/q4_audit/literature_search.json。limit=5截取展示数组，stats保留截取前数量；General setting采用原始机构记录核验，未称双引擎交叉命中。’]

```
(folder/'问题四证据映射.md').write_text('\n'.join(md),encoding='utf-8')
```

```
v=s['verification']
```

```
guide=f'''# 问题四计算交付说明
```

本轮交付模型、可复现代码、真实数值结果及已更新的论文源文件。

可核验核心结果

```
| 项目 | 计算结果 |
```

```
|---|---|
```

```
| 主模型干燥时长 | {s['dry_time_s']:.0f} s = {s['dry_time_h']:.4f} h |
```

```
| 同附录4固定半径对照 | {s['fixed_radius_dry_time_s']:.0f} s = {s['fixed_radius_dry_time_h']:.4f} h |
```

```
| 净几何效应  $\eta_R$  | {s['eta_geometry_percent']:.6f}% |
```

```
| a*=-0.2时长 | {a.loc['shape_-0.2','dry_time_h']:.6f} h |
```

```
| a*=-0.1时长 | {a.loc['shape_-0.1','dry_time_h']:.6f} h |
```

```
| a*=0.1时长 | {a.loc['shape_0.1','dry_time_h']:.6f} h |
```

```
| a*=0.2时长 | {a.loc['shape_0.2','dry_time_h']:.6f} h |
```

```
| 最大结构偏差 | {s['E_shape_percent']:.6f}% |
```

```
| h的±20%最大时长变化 | {s['E_h_percent']:.6f}% |
```

```
| hm的±20%最大时长变化 | {s['E_hm_percent']:.6f}% |
```

```
| 结束时半径 | {s['radius_end_cm']:.4f} cm |
```

```
| 前一秒全域最大含水率 | {s['previous_Cmax']:.12f} kg/kg |
```

```
| 终点全域最大含水率 | {s['final_Cmax']:.12f} kg/kg |
```

主模型约51.08h达到阈值，早于附件2的72h观测终点，未用到半径未值延拓。四种双侧结构扰动仅检验当前映射族，不能推断所有内部收缩模型均不敏感。净几何差异须以同附录4对照为基准，Q3不能替代该对照；该比较是模型内反事实，不作真实独立因果解释。

数值验证

主网格{int(s['N'])}个径向区间、1s时间步，首步BE、后续BDF2。最终网格上0.5s复算终点差{v['time_1_0.5']['event_difference_s']:.1f}s，共同表6点最大差{v['time_1_0.5']['table_max_abs_kg_kg']:.8g} kg/kg。320/640网格终点差{abs(v['space_320_640']['event_difference_s']:.0f)s ({v['space_320_640']['event_relative_percent']:.8f}%)，表6最大差{v['space_320_640']['table_max_abs_kg_kg']:.8g} kg/kg，均通过事前门槛。

最大Picard次数{s['max_picard_iterations']}；水方程离散平衡相对残差{s['moisture_balance_relative']:.5g}；热方程归一残差{s['heat_balance_scaled']:.5g}；热、水线性残差分别{s['heat_linear_residual']:.5g}和{s['moisture_linear_residual']:.5g}。这些数字验证有效方程的计算，不证明真实干固体质量闭合。

表6素材

```

80 所有含水率单位为kg/kg；“---”表示该固定位置在当前药材外。表面单独给出。
81
82 | 时间/h | 半径/cm | 0 cm | 0.5 cm | 1.0 cm | 1.5 cm | 药材表面 |
83 |---|---|---|---|---|---|---|
84 ''' + '\n'.join(rows)+f'''
85
86 四位显示的终点中心0.1500是舍入，判断使用未舍入值。result4.xlsx有{s['excel_rows']}条数据、{s['
    excel_columns']}列，固定位置0-1.9cm每0.1cm加表面。起始60s，最后{s['dry_time_s']:.0f}s；域外结构
    空值{s['structural_empty_cells']}个，每个已逐项核对。
87
88 ## 使用与复现
89
90 从项目根目录执行：`python -X utf8 问题4_复现.py`。完整入口重新计算所有情景、表格、12幅图、素材和复现
    清单。隔离复算：`python -X utf8 问题4_复现.py --output-root build/q4_reproduction`。
91
92 本轮数值复算使用Python3.12、NumPy、SciPy、pandas、openpyxl；图表使用Matplotlib和Pillow；本机加速依赖
    Numba0.67.0/llvmlite0.49.0安装在build/q4_runtime，亦可用标准pip安装。无Numba时同算法退回Python，
    速度明显下降。绘图及清单调用本机math-modeling技能脚本，技能位置与输入哈希见复现清单。`全程干燥求
    解.py --mode q4`是历史未完成入口，本轮不要使用。
93
94 | 文件/目录 | 用途 |
95 |---|---|
96 | 题目分析报告.md、术语表格.md | 冻结模型、补齐公式、单位、假设、验证门槛 |
97 | 问题4_求解.py、问题4_复现.py、问题4_绘图.py、问题4_素材整理.py | 核心、完整入口、绘图、素材自动生
    成 |
98 | results/result4.xlsx | 题定完整输出 |
99 | results/问题4_表6含水率.csv | 高精度表6 |
100 | results/q4_runs | 各情景高精度场与诊断；main.npz为主模型 |
101 | results/问题4_情景汇总.csv、问题4_边界参数敏感性.csv、问题4_数值加密.csv | 几何、结构、边界和数值
    证据 |
102 | figures/q4 | 3张原始、4张过程、5张结果图，PDF/SVG/300DPI PNG |
103 | figures/q4/_qa | 12张灰度预览 |
104 | Q4/表6_可插入.tex、关键数值_可引用.tex | 内容已同步入正文的LaTeX备用片段；正文未通过input直接引用
    |
105 | Q4/问题四证据映射.md、results/问题4_论文核对.csv | 主张与证据、正文核对、图表落点 |
106 | results/复现清单.json | 输入SHA256、依赖、种子、参数、唯一命令 |
107 | build/q4_audit | M1/P1/P2与脚本门禁证据、构建记录 |
108
109 题面附件、Q4原始思路稿及Q1-Q3权威结果未改动。论文源文件已并入本轮Q4修改并通过静态校验；按用户要求没
    有调用LaTeX编译，因此旧PDF不是当前源码的渲染结果。
110
111 ## 数据剖析与写作边界
112
113 附件1的241条及附件2的145条均无缺失。半径分布IQR规则标记早期28点，这些是时间演化形成的真实大半径，不
    删除、不缩尾。空气稳定段91点只用于均值与总体标准差，图中不是置信区间。12幅图契约见figures/q4/图
    表契约.json。
114
115 主模型仍依赖一维径向有效扩散、表面对流参数沿用和长期空气均值假设；没有内部形变观测及完整多相质量闭

```

合。稿件最终需由队伍阅读、核对和改写，不应把素材原样视作已完成参赛论文。

'''

(folder/'问题四交付说明.md').write_text(guide,encoding='utf-8')

if __name__=='__main__':main()

1 from __future__ import annotations

2 import sys

3 from pathlib import Path

4 import matplotlib

5 matplotlib.use("Agg")

6 import matplotlib.pyplot as plt

7 import numpy as np

8 import pandas as pd

9 PROJECT_ROOT = Path(__file__).resolve().parent

10 FIGURE_TOOLS = PROJECT_ROOT / "figure_tools"

11 sys.path.insert(0, str(PROJECT_ROOT))

12 sys.path.insert(0, str(FIGURE_TOOLS))

13 from export_figure import export_figure

14 from setup_style import setup_style

15 from utils.plot_style import PALETTE, audit_design, audit_layout

16 FIGURES_DIR = PROJECT_ROOT / "figures"

17 AIR_PATH = PROJECT_ROOT / "problem A" / "附件" / "附件1.xlsx"

18 RESULT1_PATH = PROJECT_ROOT / "results" / "result1.xlsx"

19 CONVERGENCE_PATH = PROJECT_ROOT / "results" / "问题1_网格收敛.csv"

20 def ensure_figure_is_valid(figure: plt.Figure) -> None:

21 issues = audit_layout(figure) + audit_design(figure)

22 if issues:

23 raise ValueError("图形预检未通过: " + "; ".join(issues))

24 def export(figure: plt.Figure, basename: str, size: tuple[float, float]) -> None:

25 ensure_figure_is_valid(figure)

26 export_figure(

27 figure,

28 str(FIGURES_DIR / basename),

29 formats=("pdf", "svg", "png"),

```

42     size_inches=size,
43     dpi=300,
44     grayscale_preview=True,
45     pad_inches=0.04,
46 )
47 plt.close(figure)
48
49
50 def plot_raw_boundary() -> None:
51     air = pd.read_excel(AIR_PATH, sheet_name=0, header=0)
52     if tuple(air.columns) != ("时间", "温度", "水分浓度"):
53         raise ValueError(f"附件1表头异常: {tuple(air.columns)}")
54     air = air.loc[air["时间"] <= 1800].copy()
55     if len(air) != 31 or air.isna().any().any():
56         raise ValueError("问题一边界记录数量或缺失值异常")
57     time_min = air["时间"].to_numpy(dtype=float) / 60.0
58
59     figure, axes = plt.subplots(1, 2, figsize=(6.3, 2.45), layout="constrained")
60     axes[0].plot(
61         time_min,
62         air["温度"],
63         color=PALETTE["primary"],
64         marker="o",
65         markevery=5,
66         markerfacecolor="white",
67         markeredgewidth=0.7,
68     )
69     axes[0].set_xlabel("时间 / min")
70     axes[0].set_ylabel("温度 / °C")
71     axes[0].set_xlim(0, 30)
72
73     axes[1].plot(
74         time_min,
75         air["水分浓度"],
76         color=PALETTE["contrast"],
77         linestyle="--",
78         marker="s",
79         markevery=5,
80         markerfacecolor="white",
81         markeredgewidth=0.7,
82     )
83     axes[1].set_xlabel("时间 / min")
84     axes[1].set_ylabel("水分浓度 / kg · kg-1")
85     axes[1].set_xlim(0, 30)
86     export(figure, "raw_q1_air_boundary", (6.3, 2.45))
87
88
89 def plot_convergence() -> None:

```

```

90 data = pd.read_csv(CONVERGENCE_PATH)
91 expected = {"时间步减半", "空间步减半"}
92 if set(data["检验"]) != expected or len(data) != 4:
93     raise ValueError("问题一收敛结果结构异常")
94 order = ["时间步减半", "空间步减半"]
95 colors = [PALETTE["primary"], PALETTE["contrast"]]
96 figure, axes = plt.subplots(1, 2, figsize=(6.3, 2.4), layout="constrained")
97 for axis, variable, unit in (
98     (axes[0], "温度_C", "最大绝对差 / °C"),
99     (
100         axes[1],
101         "水分浓度_kg_per_kg",
102         "最大绝对差 / kg·kg-1"),
103 ):
104     subset = data.loc[data["变量"] == variable].set_index("检验").loc[order]
105     values = subset["最大绝对差"].to_numpy(dtype=float)
106     y = np.arange(2)
107     for index, value in enumerate(values):
108         axis.hlines(y[index], 0.0, value, color=colors[index], linewidth=1.2)
109         axis.plot(
110             value,
111             y[index],
112             marker="o" if index == 0 else "s",
113             color=colors[index],
114             markerfacecolor="white",
115             markeredgewidth=0.9,
116         )
117         axis.text(value, y[index] + 0.14, f"{value:.2g}", ha="center", va="bottom")
118     axis.set_yticks(y, order)
119     axis.set_ylim(-0.55, 1.55)
120     axis.set_xlim(left=0)
121     axis.set_xlabel(unit)
122 export(figure, "process_q1_convergence", (6.3, 2.4))
123
124
125
126 def plot_result_fields() -> None:
127     temperature = pd.read_excel(RESULT1_PATH, sheet_name="温度", header=0)
128     moisture = pd.read_excel(RESULT1_PATH, sheet_name="水分浓度", header=0)
129     if temperature.shape != (1800, 22) or moisture.shape != (1800, 22):
130         raise ValueError("result1工作表行列数异常")
131     if temperature.isna().any().any() or moisture.isna().any().any():
132         raise ValueError("result1含有缺失值")
133     time_min = temperature.iloc[:, 0].to_numpy(dtype=float) / 60.0
134     radius_cm = np.asarray(temperature.columns[1:], dtype=float)
135     temperature_field = temperature.iloc[:, 1:].to_numpy(dtype=float)
136     moisture_field = moisture.iloc[:, 1:].to_numpy(dtype=float)
137

```

```

138 figure, axes = plt.subplots(1, 2, figsize=(6.3, 2.55), layout="constrained")
139 meshes = []
140 for axis, field, color_map, color_label in (
141     (axes[0], temperature_field, "magma", "温度 / °C"),
142     (
143         axes[1],
144         moisture_field,
145         "viridis",
146         "含水率 / kg · kg-1",
147     ),
148 ):
149     mesh = axis.pcolormesh(
150         radius_cm,
151         time_min,
152         field,
153         shading="auto",
154         cmap=color_map,
155     )
156     meshes.append(mesh)
157     axis.set_xlabel("距轴线距离 / cm")
158     axis.set_ylabel("时间 / min")
159     axis.set_xlim(0, 2)
160     axis.set_ylim(0, 30)
161     colorbar = figure.colorbar(mesh, ax=axis, pad=0.02)
162     colorbar.set_label(color_label)
163 export.figure, "result_q1_fields", (6.3, 2.55))
164
165
166 def main() -> None:
167     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
168     style = setup_style(
169         journal="general",
170         lang="zh",
171         serif_for_zh=True,
172         use_sciplots=False,
173         constrained_layout=True,
174     )
175     print(f"使用字体: {style['cjk_font']}")
176     plot_raw_boundary()
177     plot_convergence()
178     plot_result_fields()
179
180
181 if __name__ == "__main__":
182     main()

```

```
1 #!/usr/bin/env python3
```

```

2 """绘制问题二的原始数据、求解过程和最终结果候选图。"""
3
4 from __future__ import annotations
5
6 import json
7 import sys
8 from pathlib import Path
9
10 import matplotlib
11
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 from matplotlib.colors import Normalize
15 from matplotlib.cm import ScalarMappable
16 import numpy as np
17 import pandas as pd
18
19
20 PROJECT_ROOT = Path(__file__).resolve().parent
21 FIGURE_SCRIPTS = PROJECT_ROOT / "figure_tools"
22 sys.path.insert(0, str(PROJECT_ROOT))
23 sys.path.insert(0, str(FIGURE_SCRIPTS))
24
25 from utils.plot_style import add_panel_labels, publication_subplots # noqa: E402
26 from export_figure import export_figure # noqa: E402
27 from setup_style import setup_style # noqa: E402
28 from visual_qa import audit_layout, print_report, render_preview # noqa: E402
29
30
31 FIGURES_DIR = PROJECT_ROOT / "figures"
32 QA_DIR = FIGURES_DIR / "_qa"
33 RESULTS_DIR = PROJECT_ROOT / "results"
34 FIGURE_SIZE = (6.3, 3.9)
35 COLORS = {
36     "blue": "#0072B2",
37     "orange": "#D55E00",
38     "green": "#009E73",
39     "sky": "#56B4E9",
40     "purple": "#CC79A7",
41     "gray": "#6B7280",
42     "black": "#111827",
43 }
44
45
46 def load_data() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict, pd.DataFrame]:
47     boundary = pd.read_excel(PROJECT_ROOT / "problem A" / "附件" / "附件1.xlsx")
48     boundary.columns = ["时间_s", "温度_C", "空气含湿量_kg_kg"]
49     boundary = boundary.loc[boundary["时间_s"] <= 10800].copy()

```

```

50     boundary["时间_h"] = boundary["时间_s"] / 3600.0
51     boundary.to_csv(RESULTS_DIR / "问题2_空气边界.csv", index=False, encoding="utf-8-sig")
52
53     temperature = pd.read_excel(RESULTS_DIR / "result2.xlsx", sheet_name="温度")
54     moisture = pd.read_excel(RESULTS_DIR / "result2.xlsx", sheet_name="水分浓度")
55     summary = json.loads((RESULTS_DIR / "问题2_结果摘要.json").read_text(encoding="utf-8"))
56     iterations = pd.read_csv(RESULTS_DIR / "问题2_迭代次数.csv")
57     return boundary, temperature, moisture, summary, iterations
58
59
60 def field_arrays(frame: pd.DataFrame) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
61     times = frame.iloc[:, 0].to_numpy(dtype=float)
62     radii = np.asarray([float(value) for value in frame.columns[1:]], dtype=float)
63     values = frame.iloc[:, 1:].to_numpy(dtype=float)
64     return times / 3600.0, radii, values
65
66
67 def finish(fig, basename: str, size: tuple[float, float] = FIGURE_SIZE) -> None:
68     QA_DIR.mkdir(parents=True, exist_ok=True)
69     preview = QA_DIR / f"{basename}_preview.png"
70     render_preview(fig, str(preview), dpi=150)
71     issues = audit_layout(fig)
72     report = print_report(issues)
73     print(f"[{basename}]\n{report}")
74     failures = [message for severity, message in issues if severity == "FAIL"]
75     if failures:
76         raise RuntimeError(f"{basename} 版面检查未通过: {'; '.join(failures)}")
77     export_figure(
78         fig,
79         str(FIGURES_DIR / basename),
80         formats=("pdf", "svg", "png"),
81         dpi=300,
82         size_inches=size,
83         grayscale_preview=True,
84     )
85     grayscale_path = FIGURES_DIR / f"{basename}_grayscale.png"
86     if grayscale_path.exists():
87         from PIL import Image
88
89         with Image.open(grayscale_path) as source:
90             grayscale_pixels = source.copy()
91             grayscale_pixels.save(grayscale_path, dpi=(300, 300))
92     plt.close(fig)
93
94
95 def plot_boundary_series(boundary: pd.DataFrame) -> None:
96     fig, axes = publication_subplots(2, 1, width="report", aspect=0.66, squeeze=False)
97     axes = axes.ravel()

```

```

98 axes[0].plot(boundary["时间_h"], boundary["温度_C"], color=COLORS["orange"])
99 axes[0].set_ylabel("烘房温度 摄氏度")
100 axes[0].set_xlim(0, 3)
101 axes[0].set_xticklabels([])
102 axes[1].plot(boundary["时间_h"], boundary["空气含湿量_kg_kg"], color=COLORS["blue"], linestyle="--")
103 axes[1].set_xlabel("时间 小时")
104 axes[1].set_ylabel("空气水分浓度 千克每千克")
105 axes[1].set_xlim(0, 3)
106 axes[1].ticklabel_format(axis="y", style="plain", useOffset=False)
107 finish(fig, "raw_q2_boundary_series", (6.3, 4.2))
108
109
110 def plot_boundary_relation(boundary: pd.DataFrame) -> None:
111     fig, ax = publication_subplots(width="report", aspect=0.66)
112     ax.plot(
113         boundary["温度_C"],
114         boundary["空气含湿量_kg_kg"],
115         color="#B8BDC7",
116         linewidth=0.8,
117         zorder=1,
118     )
119     scatter = ax.scatter(
120         boundary["温度_C"],
121         boundary["空气含湿量_kg_kg"],
122         c=boundary["时间_h"],
123         cmap="viridis",
124         s=18,
125         edgecolors="white",
126         linewidths=0.25,
127         zorder=2,
128     )
129     correlation = boundary[["温度_C", "空气含湿量_kg_kg"]].corr().iloc[0, 1]
130     ax.text(0.04, 0.95, f"Pearson r = {correlation:.3f}", transform=ax.transAxes, va="top")
131     ax.set_xlabel("烘房温度 摄氏度")
132     ax.set_ylabel("空气含湿量 千克每千克")
133     colorbar = fig.colorbar(scatter, ax=ax, pad=0.03)
134     colorbar.set_label("时间 小时")
135     finish(fig, "raw_q2_boundary_relation")
136
137
138 def plot_boundary_rates(boundary: pd.DataFrame) -> None:
139     changes = boundary.diff().iloc[1:].copy()
140     changes["时间_h"] = boundary["时间_h"].iloc[1:].to_numpy()
141     fig, axes = publication_subplots(2, 1, width="report", aspect=0.66, squeeze=False)
142     axes = axes.ravel()
143     axes[0].plot(changes["时间_h"], changes["温度_C"], color=COLORS["orange"])
144     axes[0].axhline(0, color=COLORS["gray"], linewidth=0.6, linestyle=":")

```

```

145 axes[0].set_ylabel("每分钟温升 摄氏度")
146 axes[0].set_xlim(0, 3)
147 axes[0].set_xticklabels([])
148 axes[1].plot(changes["时间_h"], changes["空气含湿量_kg_kg"], color=COLORS["blue"], linestyle="--")
149 axes[1].axhline(0, color=COLORS["gray"], linewidth=0.6, linestyle=":")
150 axes[1].set_xlabel("时间 小时")
151 axes[1].set_ylabel("每分钟含湿量增量")
152 axes[1].set_xlim(0, 3)
153 axes[1].ticklabel_format(axis="y", style="sci", scilimits=(-3, 3))
154 add_panel_labels(axes)
155 finish(fig, "raw_q2_boundary_rates", (6.3, 4.2))
156
157
158 def plot_picard_iterations(iterations: pd.DataFrame) -> None:
159     fig, axes = publication_subplots(1, 2, width="report", aspect=0.57, width_ratios=(2.15, 1.0),
160     squeeze=False)
161     axes = axes.ravel()
162     data = iterations.loc[iterations["时间_s"] > 0].copy()
163     axes[0].step(data["时间_s"] / 3600.0, data["Picard迭代次数"], where="post", color=COLORS["blue"]
164     ])
165     axes[0].set_xlabel("时间 小时")
166     axes[0].set_ylabel("Picard 迭代次数")
167     axes[0].set_xlim(0, 3)
168     axes[0].set_ylim(2.35, 4.3)
169     axes[0].set_yticks([3, 4])
170     axes[0].text(
171         0.02,
172         0.04,
173         r"阈值  $10^{-8}$  摄氏度和  $10^{-10}$  千克每千克, 迭代上限 40 次",
174         transform=axes[0].transAxes,
175         fontsize=6.5,
176     )
177
178     counts = data["Picard迭代次数"].value_counts().sort_index()
179     axes[1].plot(counts.to_numpy(), counts.index.to_numpy(), "o", color=COLORS["orange"], markersize
180     =5)
181     for y_value, x_value in zip(counts.index, counts.to_numpy()):
182         axes[1].hlines(y_value, 0, x_value, color=COLORS["orange"], linewidth=1.0)
183         axes[1].text(x_value, y_value + 0.08, f"{x_value}", ha="right", va="bottom", fontsize=7)
184     axes[1].set_xlabel("时间层数")
185     axes[1].set_ylabel("迭代次数")
186     axes[1].set_yticks([3, 4])
187     axes[1].set_xlim(0, max(counts) * 1.08)
188     add_panel_labels(axes)
189     finish(fig, "process_q2_picard_iterations", (6.3, 3.7))

```

```

189 def plot_refinement(summary: dict) -> None:
190     verification = summary["verification"]
191     labels = ["时间步减半", "空间网格减半"]
192     temperature_error = [
193         verification["max_temperature_difference_time_refined_c"],
194         verification["max_temperature_difference_space_refined_c"],
195     ]
196     moisture_error = [
197         verification["max_moisture_difference_time_refined_kg_per_kg"],
198         verification["max_moisture_difference_space_refined_kg_per_kg"],
199     ]
200     fig, axes = publication_subplots(1, 2, width="report", aspect=0.57, squeeze=False)
201     axes = axes.ravel()
202     for axis, values, ylabel, color in (
203         (axes[0], temperature_error, "温度最大绝对差 摄氏度", COLORS["orange"]),
204         (axes[1], moisture_error, "含水率最大绝对差 千克每千克", COLORS["blue"]),
205     ):
206         x = np.arange(2)
207         axis.plot(x, values, "o", color=color, markersize=5)
208         axis.vlines(x, 1e-9, values, color=color, linewidth=1.0)
209         axis.axhline(5e-5, color=COLORS["gray"], linestyle="--", linewidth=0.8)
210         axis.text(
211             0.97,
212             0.92,
213             "四位小数半单位",
214             transform=axis.transAxes,
215             ha="right",
216             va="top",
217             color=COLORS["gray"],
218             fontsize=6.5,
219         )
220         axis.set_yscale("log")
221         axis.set_xticks(x, labels)
222         axis.tick_params(axis="x", rotation=12)
223         axis.set_ylabel(ylabel)
224         axis.set_ylim(1e-9, 1e-4)
225     add_panel_labels(axes)
226     finish(fig, "process_q2_refinement", (6.3, 3.7))
227
228
229 def plot_sensitivity(summary: dict) -> None:
230     frame = pd.DataFrame(summary["sensitivity"])
231     labels = [r"$h$ 0.8 倍", r"$h$ 1.2 倍", r"$h_m$ 0.8 倍", r"$h_m$ 1.2 倍"]
232     y = np.arange(len(labels))
233     colors = [COLORS["orange"], COLORS["orange"], COLORS["blue"], COLORS["blue"]]
234     markers = ["o", "s", "o", "s"]
235     fig, axes = publication_subplots(1, 2, width="report", aspect=0.57, squeeze=False)
236     axes = axes.ravel()

```

```

237 columns = ["max_temperature_change_c", "max_moisture_change_kg_per_kg"]
238 xlabels = ["温度最大变化 摄氏度", "含水率最大变化 千克每千克"]
239 for axis, column, xlabel in zip(axes, columns, xlabel):
240     values = frame[column].to_numpy()
241     for index, value in enumerate(values):
242         axis.hlines(y[index], 0, value, color=colors[index], linewidth=1.0)
243         axis.plot(value, y[index], marker=markers[index], color=colors[index], markersize=5)
244     axis.set_yticks(y, labels)
245     axis.set_xlabel(xlabel)
246     axis.set_xlim(left=0)
247     axis.invert_yaxis()
248 finish(fig, "process_q2_sensitivity", (6.3, 3.7))
249
250
251 def plot_field(time_h: np.ndarray, radii: np.ndarray, values: np.ndarray, basename: str, label: str,
252               cmap: str) -> None:
253     fig, ax = publication_subplots(width="report", aspect=0.60)
254     display_stride = 30
255     display_time = time_h[::display_stride]
256     display_values = values[::display_stride]
257     if display_time[-1] != time_h[-1]:
258         display_time = np.append(display_time, time_h[-1])
259         display_values = np.vstack([display_values, values[-1]])
260     image = ax.pcolormesh(display_time, radii, display_values.T, shading="auto", cmap=cmap)
261     ax.set_xlabel("时间 小时")
262     ax.set_ylabel("到药材中心的距离 厘米")
263     ax.set_xlim(0, 3)
264     ax.set_ylim(0, 2)
265     colorbar = fig.colorbar(image, ax=ax, pad=0.03)
266     colorbar.set_label(label)
267     finish(fig, basename, (6.3, 3.8))
268
269 def plot_center_surface_gap(
270     time_h: np.ndarray,
271     temperature_values: np.ndarray,
272     moisture_values: np.ndarray,
273 ) -> None:
274     temperature_gap = temperature_values[:, -1] - temperature_values[:, 0]
275     moisture_gap = moisture_values[:, 0] - moisture_values[:, -1]
276     report_times = np.asarray([1800, 3600, 5400, 7200, 9000, 10800], dtype=float)
277     report_indices = np.searchsorted(time_h * 3600.0, report_times)
278     fig, axes = publication_subplots(2, 1, width="report", aspect=0.66, squeeze=False)
279     axes = axes.ravel()
280     axes[0].plot(time_h, temperature_gap, color=COLORS["orange"])
281     axes[0].plot(time_h[report_indices], temperature_gap[report_indices], "o", color=COLORS["black"],
282                 markersize=3)
283     axes[0].set_ylabel("表面减中心温差 摄氏度")

```

```

283 axes[0].set_xlim(0, 3)
284 axes[0].set_ylim(bottom=0)
285 axes[0].set_xticklabels([])
286 axes[1].plot(time_h, moisture_gap, color=COLORS["blue"], linestyle="--")
287 axes[1].plot(time_h[report_indices], moisture_gap[report_indices], "s", color=COLORS["black"],
288             markersize=3)
289 axes[1].set_xlabel("时间 小时")
290 axes[1].set_ylabel("中心减表面含水率")
291 axes[1].set_xlim(0, 3)
292 axes[1].set_ylim(bottom=0)
293 add_panel_labels(axes)
294 finish(fig, "result_q2_center_surface_gap", (6.3, 4.2))
295
296 def main() -> None:
297     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
298     setup_style(journal="nature", lang="zh", use_sciplots=True, serif_for_zh=False)
299     boundary, temperature, moisture, summary, iterations = load_data()
300     time_h, radii, temperature_values = field_arrays(temperature)
301     moisture_time_h, moisture_radii, moisture_values = field_arrays(moisture)
302     if not np.array_equal(time_h, moisture_time_h) or not np.array_equal(radii, moisture_radii):
303         raise ValueError("温度表和水分浓度表的网格不一致")
304
305     plot_boundary_series(boundary)
306     plot_boundary_relation(boundary)
307     plot_boundary_rates(boundary)
308     plot_picard_iterations(iterations)
309     plot_refinement(summary)
310     plot_sensitivity(summary)
311     plot_field(time_h, radii, temperature_values, "result_q2_temperature_field", "温度 摄氏度", "
312             inferno")
313     plot_field(time_h, radii, moisture_values, "result_q2_moisture_field", "含水率 千克每千克", "
314             cividis_r")
315     plot_center_surface_gap(time_h, temperature_values, moisture_values)
316     print("问题二九张候选图已完成")
317
318 if __name__ == "__main__":
319     main()

```

```

1 #!/usr/bin/env python3
2 """绘制问题三的原始数据、求解过程和最终结果候选图。"""
3
4 from __future__ import annotations
5
6 import json
7 import sys

```

```

8 from pathlib import Path
9
10 import matplotlib
11
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 import numpy as np
15 import pandas as pd
16
17
18 PROJECT_ROOT = Path(__file__).resolve().parent
19 FIGURE_SCRIPTS = PROJECT_ROOT / "figure_tools"
20 sys.path.insert(0, str(PROJECT_ROOT))
21 sys.path.insert(0, str(FIGURE_SCRIPTS))
22
23 from utils.plot_style import add_panel_labels, publication_subplots # noqa: E402
24 from export_figure import export_figure # noqa: E402
25 from setup_style import setup_style # noqa: E402
26 from visual_qa import audit_layout, print_report, render_preview # noqa: E402
27
28
29 FIGURES_DIR = PROJECT_ROOT / "figures"
30 QA_DIR = FIGURES_DIR / "_qa"
31 RESULTS_DIR = PROJECT_ROOT / "results"
32 COLORS = {
33     "blue": "#0072B2",
34     "orange": "#D55E00",
35     "green": "#009E73",
36     "sky": "#56B4E9",
37     "purple": "#CC79A7",
38     "gray": "#6B7280",
39     "black": "#111827",
40 }
41
42
43 def load_data():
44     boundary = pd.read_excel(PROJECT_ROOT / "problem A" / "附件" / "附件1.xlsx")
45     boundary.columns = ["时间_s", "温度_C", "空气含湿量_kg_kg"]
46     boundary["时间_h"] = boundary["时间_s"] / 3600.0
47     stable = boundary.loc[boundary["时间_s"] >= 9000].copy()
48
49     process = pd.read_csv(REULTS_DIR / "问题3_关键过程.csv")
50     process.columns = [
51         "时间_s", "时间_h", "含水率_0_cm", "含水率_0.5_cm", "含水率_1_cm",
52         "含水率_1.5_cm", "含水率_2_cm", "最大含水率", "最大值半径_cm",
53     ]
54     time_refinement = pd.read_csv(REULTS_DIR / "问题3_时间步敏感性.csv")
55     time_refinement.columns = ["时间步_s", "烘干时长_s", "烘干时长_h", "相对参考差_s"]

```

```

56 space_refinement = pd.read_csv(RESULTS_DIR / "问题3_空间网格敏感性.csv")
57 space_refinement.columns = ["径向区间数", "时间步_s", "烘干时长_s", "烘干时长_h"]
58 sensitivity = pd.read_csv(RESULTS_DIR / "问题3_边界敏感性.csv")
59 sensitivity.columns = [
60     "工况", "温度_C", "空气含湿量_kg_kg", "换热系数", "传质系数",
61     "烘干时长_s", "烘干时长_h", "相对变化",
62 ]
63 iterations = pd.read_csv(RESULTS_DIR / "问题3_迭代次数.csv")
64 iterations.columns = ["时间_s", "迭代次数"]
65 summary = json.loads((RESULTS_DIR / "问题3_结果摘要.json").read_text(encoding="utf-8"))
66
67 field = pd.read_excel(RESULTS_DIR / "result3.xlsx")
68 field_time = field.iloc[:, 0].to_numpy(dtype=float)
69 field_radii = np.asarray([float(item) for item in field.columns[1:]], dtype=float)
70 field_values = field.iloc[:, 1:].to_numpy(dtype=float)
71 field_time = np.insert(field_time, 0, 0.0)
72 field_values = np.vstack([np.full(field_radii.size, 2.55), field_values])
73 return (
74     boundary, stable, process, time_refinement, space_refinement,
75     sensitivity, iterations, summary, field_time, field_radii, field_values,
76 )
77
78
79 def finish(fig, basename: str, size=(6.3, 3.9)) -> None:
80     QA_DIR.mkdir(parents=True, exist_ok=True)
81     render_preview(fig, str(QA_DIR / f"{basename}_preview.png"), dpi=150)
82     issues = audit_layout(fig)
83     print(f"[{basename}]\n{print_report(issues)}")
84     failures = [message for severity, message in issues if severity == "FAIL"]
85     if failures:
86         raise RuntimeError(f"{basename} 版面检查未通过: {'; '.join(failures)}")
87     export_figure(
88         fig,
89         str(FIGURES_DIR / basename),
90         formats=("pdf", "svg", "png"),
91         dpi=300,
92         size_inches=size,
93         grayscale_preview=True,
94     )
95     grayscale_path = FIGURES_DIR / f"{basename}_grayscale.png"
96     if grayscale_path.exists():
97         from PIL import Image
98
99         with Image.open(grayscale_path) as source:
100             gray = source.copy()
101             gray.save(grayscale_path, dpi=(300, 300))
102     plt.close(fig)
103

```

```

104
105 def plot_air_full(boundary: pd.DataFrame) -> None:
106     fig, axes = publication_subplots(2, 1, width="report", aspect=0.66, squeeze=False)
107     axes = axes.ravel()
108     for axis in axes:
109         axis.axvspan(2.5, 4.0, color="#D9DEE7", alpha=0.55, linewidth=0)
110         axis.axvline(2.5, color=COLORS["gray"], linestyle=":", linewidth=0.8)
111         axis.set_xlim(0, 4)
112     axes[0].plot(boundary["时间_h"], boundary["温度_C"], color=COLORS["orange"])
113     axes[0].set_ylabel("烘房温度 摄氏度")
114     axes[0].tick_params(labelbottom=False)
115     axes[0].text(2.55, 29.3, "稳定段", color=COLORS["gray"], fontsize=7)
116     axes[1].plot(
117         boundary["时间_h"], boundary["空气含湿量_kg_kg"],
118         color=COLORS["blue"], linestyle="--",
119     )
120     axes[1].set_xlabel("时间 小时")
121     axes[1].set_ylabel("空气水分浓度 千克每千克")
122     axes[1].ticklabel_format(axis="y", style="plain", useOffset=False)
123     add_panel_labels(axes)
124     finish(fig, "raw_q3_air_full", (6.3, 4.2))
125
126
127 def plot_air_stable(stable: pd.DataFrame, summary: dict) -> None:
128     stats = summary["stable_air"]
129     fig, axes = publication_subplots(1, 2, width="report", aspect=0.56, squeeze=False)
130     axes = axes.ravel()
131     specs = (
132         ("温度_C", stats["temperature_mean_c"], stats["temperature_std_c"],
133          COLORS["orange"], "烘房温度 摄氏度"),
134         ("空气含湿量_kg_kg", stats["moisture_mean_kg_per_kg"],
135          stats["moisture_std_kg_per_kg"], COLORS["blue"], "空气水分浓度 千克每千克"),
136     )
137     for axis, (column, mean, sd, color, ylabel) in zip(axes, specs):
138         axis.plot(stable["时间_h"], stable[column], color=color, linewidth=0.9)
139         axis.axhline(mean, color=COLORS["black"], linestyle="--", linewidth=0.8, label="稳定段均值")
140         axis.fill_between(
141             stable["时间_h"], mean - sd, mean + sd,
142             color=color, alpha=0.15, linewidth=0, label="总体标准差范围",
143         )
144         axis.set_xlabel("时间 小时")
145         axis.set_ylabel(ylabel)
146         axis.set_xlim(2.5, 4.0)
147         axis.ticklabel_format(axis="y", style="plain", useOffset=False)
148     axes[1].legend(loc="lower right", frameon=False)
149     add_panel_labels(axes)
150     finish(fig, "raw_q3_air_stable", (6.3, 3.55))
151

```

```

152
153 def plot_air_distribution(stable: pd.DataFrame) -> None:
154     fig, axes = publication_subplots(
155         1, 3, width="report", aspect=0.48, width_ratios=(1.0, 1.0, 1.3), squeeze=False,
156     )
157     axes = axes.ravel()
158     axes[0].hist(stable["温度_C"], bins=11, color=COLORS["orange"], alpha=0.72,
159                 edgecolor="white", linewidth=0.4)
160     axes[0].axvline(stable["温度_C"].mean(), color=COLORS["black"], linestyle="--", linewidth=0.8)
161     axes[0].set_xlabel("烘房温度 摄氏度")
162     axes[0].set_ylabel("记录数")
163     axes[1].hist(stable["空气含湿量_kg_kg"], bins=11, color=COLORS["blue"], alpha=0.72,
164                 edgecolor="white", linewidth=0.4)
165     axes[1].axvline(stable["空气含湿量_kg_kg"].mean(), color=COLORS["black"], linestyle="--",
166                    linewidth=0.8)
167     axes[1].set_xlabel("空气水分浓度 千克每千克")
168     axes[1].set_ylabel("记录数")
169     axes[1].ticklabel_format(axis="x", style="plain", useOffset=False)
170     scatter = axes[2].scatter(
171         stable["温度_C"], stable["空气含湿量_kg_kg"], c=stable["时间_h"],
172         cmap="viridis", s=18, edgecolors="white", linewidths=0.25,
173     )
174     correlation = stable[["温度_C", "空气含湿量_kg_kg"]].corr().iloc[0, 1]
175     axes[2].set_title(f"Pearson r = {correlation:.3f}", loc="left", pad=4)
176     axes[2].set_xlabel("烘房温度 摄氏度")
177     axes[2].set_ylabel("空气水分浓度 千克每千克")
178     axes[2].ticklabel_format(axis="y", style="plain", useOffset=False)
179     colorbar = fig.colorbar(scatter, ax=axes[2], pad=0.03)
180     colorbar.set_label("时间 小时")
181     add_panel_labels(axes)
182     finish(fig, "raw_q3_air_distribution", (6.3, 3.2))
183
184 def plot_time_convergence(frame: pd.DataFrame) -> None:
185     frame = frame.sort_values("时间步_s")
186     fig, axes = publication_subplots(1, 2, width="report", aspect=0.56, squeeze=False)
187     axes = axes.ravel()
188     axes[0].plot(frame["时间步_s"], frame["烘干时长_s"], "o-", color=COLORS["blue"], markersize=4)
189     axes[0].set_xlabel("时间步 秒")
190     axes[0].set_ylabel("烘干时长 秒")
191     axes[0].ticklabel_format(axis="y", style="plain", useOffset=False)
192     axes[0].set_xticks([0.25, 0.5, 1.0])
193     axes[1].plot(frame["时间步_s"], frame["相对参考差_s"], "o", color=COLORS["orange"], markersize=5)
194     axes[1].hlines(1.0, 0.2, 1.05, color=COLORS["gray"], linestyle="--", linewidth=0.8)
195     axes[1].text(1.02, 1.02, "1 秒门槛", ha="right", va="bottom", color=COLORS["gray"], fontsize=6.5)
196     axes[1].set_xlabel("时间步 秒")

```

```

197 axes[1].set_ylabel("相对 0.25 秒结果的绝对差 秒")
198 axes[1].set_xlim(0.2, 1.05)
199 axes[1].set_ylim(-0.05, 1.18)
200 axes[1].set_xticks([0.25, 0.5, 1.0])
201 add_panel_labels(axes)
202 finish(fig, "process_q3_time_convergence", (6.3, 3.55))
203
204
205 def plot_space_convergence(frame: pd.DataFrame, summary: dict) -> None:
206     verification = summary["verification"]
207     fig, axes = publication_subplots(
208         1, 2, width="report", aspect=0.56, width_ratios=(1.0, 1.35), squeeze=False,
209     )
210     axes = axes.ravel()
211     axes[0].plot(frame["径向区间数"], frame["烘干时长_h"], "o-", color=COLORS["blue"], markersize=5)
212     axes[0].set_xlabel("径向区间数")
213     axes[0].set_ylabel("烘干时长 小时")
214     axes[0].set_xticks([160, 320])
215     ratios = np.asarray([
216         verification["space_time_relative_difference"] / 0.001,
217         verification["space_table_max_absolute_difference_kg_per_kg"] / 5e-5,
218     ])
219     labels = ["时长误差", "表值误差"]
220     y = np.arange(2)
221     axes[1].axvline(1.0, color=COLORS["gray"], linestyle="--", linewidth=0.8)
222     axes[1].plot(ratios, y, "o", color=COLORS["orange"], markersize=5)
223     axes[1].hlines(y, 0, ratios, color=COLORS["orange"], linewidth=1.0)
224     axes[1].set_yticks(y, labels)
225     axes[1].set_xlabel("误差与门槛之比")
226     axes[1].set_xlim(0, max(2.25, ratios.max() * 1.12))
227     axes[1].invert_yaxis()
228     axes[1].text(1.02, 0.96, "门槛", transform=axes[1].get_xaxis_transform(), color=COLORS["gray"],
229         fontsize=6.5)
230     add_panel_labels(axes)
231     finish(fig, "process_q3_space_convergence", (6.3, 3.55))
232
233 def plot_iterations(frame: pd.DataFrame) -> None:
234     data = frame.loc[frame["时间_s"] > 0].copy()
235     fig, axes = publication_subplots(
236         1, 2, width="report", aspect=0.56, width_ratios=(2.2, 1.0), squeeze=False,
237     )
238     axes = axes.ravel()
239     axes[0].step(data["时间_s"] / 3600.0, data["迭代次数"], where="post", color=COLORS["blue"])
240     axes[0].set_xlabel("时间 小时")
241     axes[0].set_ylabel("Picard 迭代次数")
242     axes[0].set_xlim(0, data["时间_s"].max() / 3600.0)
243     axes[0].set_ylim(2.35, 4.3)

```

```

244 axes[0].set_yticks([3, 4])
245 axes[0].text(0.02, 0.05, "温度阈值 1e-8 含水率阈值 1e-10 上限 40 次",
246             transform=axes[0].transAxes, fontsize=6.5)
247 counts = data["迭代次数"].value_counts().sort_index()
248 axes[1].hlines(counts.index, 0, counts.to_numpy(), color=COLORS["orange"], linewidth=1.0)
249 axes[1].plot(counts.to_numpy(), counts.index, "o", color=COLORS["orange"], markersize=5)
250 axes[1].set_xlabel("时间层数")
251 axes[1].set_ylabel("迭代次数")
252 axes[1].set_yticks([3, 4])
253 axes[1].set_xlim(0, counts.max() * 1.08)
254 add_panel_labels(axes)
255 finish(fig, "process_q3_picard_iterations", (6.3, 3.55))
256
257
258 def plot_sensitivity(frame: pd.DataFrame) -> None:
259     labels = ["稳定温度减一倍标准差", "稳定温度加一倍标准差", "空气水分减一倍标准差",
260             "空气水分加一倍标准差", "附件末条记录", "端面通量上界"]
261     values = 100.0 * frame["相对变化"].to_numpy()
262     y = np.arange(len(labels))
263     fig, ax = publication_subplots(width="report", aspect=0.60)
264     ax.axvline(0, color=COLORS["black"], linewidth=0.7)
265     ax.axvline(-1, color=COLORS["gray"], linestyle=":", linewidth=0.7)
266     ax.axvline(1, color=COLORS["gray"], linestyle=":", linewidth=0.7)
267     colors = [COLORS["orange"]] * 2 + [COLORS["blue"]] * 2 + [COLORS["purple"], COLORS["green"]]
268     markers = ["o", "s", "o", "s", "D", "^"]
269     for index, value in enumerate(values):
270         ax.hlines(y[index], 0, value, color=colors[index], linewidth=1.0)
271         ax.plot(value, y[index], marker=markers[index], color=colors[index], markersize=5)
272     ax.set_yticks(y, labels)
273     ax.set_xlabel("烘干时长相对基准变化 百分比")
274     ax.set_xlim(-1.05, 1.05)
275     ax.invert_yaxis()
276     finish(fig, "process_q3_boundary_sensitivity", (6.3, 3.8))
277
278
279 def plot_threshold(process: pd.DataFrame, summary: dict) -> None:
280     dry_h = summary["dry_time_h"]
281     fig, axes = publication_subplots(
282         1, 2, width="report", aspect=0.56, width_ratios=(1.55, 1.0), squeeze=False,
283     )
284     axes = axes.ravel()
285     axes[0].plot(process["时间_h"], process["最大含水率"], color=COLORS["blue"])
286     axes[0].axhline(0.15, color=COLORS["orange"], linestyle="--", linewidth=0.9)
287     axes[0].axvline(dry_h, color=COLORS["gray"], linestyle=":", linewidth=0.8)
288     axes[0].set_xlabel("时间 小时")
289     axes[0].set_ylabel("最大含水率 千克每千克")
290     axes[0].set_xlim(0, dry_h + 0.5)
291     axes[0].set_ylim(0, 2.65)

```

```

292 local = process.loc[process["时间_h"] >= dry_h - 6.0]
293 axes[1].plot(local["时间_h"], local["最大含水率"], color=COLORS["blue"])
294 axes[1].axhline(0.15, color=COLORS["orange"], linestyle="--", linewidth=0.9, label="判定阈值")
295 axes[1].plot(dry_h, summary["moisture_kg_per_kg"][-1][0], "o", color=COLORS["black"], markersize
=4)
296 axes[1].text(dry_h - 0.2, 0.15055, f"终点 {dry_h:.4f} 小时", ha="right", va="bottom", fontsize
=6.8)
297 axes[1].set_xlabel("时间 小时")
298 axes[1].set_ylabel("最大含水率 千克每千克")
299 axes[1].set_xlim(dry_h - 6.0, dry_h + 0.15)
300 axes[1].set_ylim(0.1485, 0.166)
301 axes[1].legend(loc="upper right", frameon=False)
302 add_panel_labels(axes)
303 finish(fig, "result_q3_threshold", (6.3, 3.55))
304
305
306 def plot_profiles(process: pd.DataFrame) -> None:
307     columns = ["含水率_0_cm", "含水率_0.5_cm", "含水率_1_cm", "含水率_1.5_cm", "含水率_2_cm"]
308     labels = ["0 厘米", "0.5 厘米", "1.0 厘米", "1.5 厘米", "2.0 厘米"]
309     colors = [COLORS["blue"], COLORS["sky"], COLORS["green"], COLORS["orange"], COLORS["purple"]]
310     styles = ["-", "--", "-.", ":", (0, (5, 2))]
311     markers = ["o", "s", "~", "D", "v"]
312     fig, ax = publication_subplots(width="report", aspect=0.60)
313     for column, label, color, style, marker in zip(columns, labels, colors, styles, markers):
314         ax.plot(process["时间_h"], process[column], color=color, linestyle=style,
315                 marker=marker, markevery=430, markersize=3, label=label)
316     ax.axhline(0.15, color=COLORS["black"], linestyle="--", linewidth=0.8, label="判定阈值")
317     ax.set_xlabel("时间 小时")
318     ax.set_ylabel("含水率 千克每千克")
319     ax.set_xlim(0, process["时间_h"].max())
320     ax.set_ylim(0, 2.65)
321     ax.legend(ncol=3, loc="upper right", frameon=False)
322     finish(fig, "result_q3_profiles", (6.3, 3.8))
323
324
325 def plot_field(time_s: np.ndarray, radii: np.ndarray, values: np.ndarray) -> None:
326     stride = 10
327     display_time = time_s[::stride]
328     display_values = values[::stride]
329     if display_time[-1] != time_s[-1]:
330         display_time = np.append(display_time, time_s[-1])
331         display_values = np.vstack([display_values, values[-1]])
332     fig, ax = publication_subplots(width="report", aspect=0.60)
333     image = ax.pcolormesh(display_time / 3600.0, radii, display_values.T, shading="auto", cmap="
cividis_r")
334     contour = ax.contour(display_time / 3600.0, radii, display_values.T, levels=[0.15],
335                          colors=[COLORS["orange"]], linewidths=0.9)
336     if contour.allsegs[0]:

```

```

337     ax.clabel(contour, fmt={0.15: "0.15"}, inline=True, fontsize=6.5)
338     ax.set_xlabel("时间 小时")
339     ax.set_ylabel("到药材中心的距离 厘米")
340     ax.set_xlim(0, time_s[-1] / 3600.0)
341     ax.set_ylim(0, 2)
342     colorbar = fig.colorbar(image, ax=ax, pad=0.03)
343     colorbar.set_label("含水率 千克每千克")
344     finish(fig, "result_q3_moisture_field", (6.3, 3.8))
345
346
347 def main() -> None:
348     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
349     setup_style(journal="nature", lang="zh", use_sciplots=True, serif_for_zh=False)
350     data = load_data()
351     (
352         boundary, stable, process, time_refinement, space_refinement,
353         sensitivity, iterations, summary, field_time, field_radII, field_values,
354     ) = data
355     plot_air_full(boundary)
356     plot_air_stable(stable, summary)
357     plot_air_distribution(stable)
358     plot_time_convergence(time_refinement)
359     plot_space_convergence(space_refinement, summary)
360     plot_iterations(iterations)
361     plot_sensitivity(sensitivity)
362     plot_threshold(process, summary)
363     plot_profiles(process)
364     plot_field(field_time, field_radII, field_values)
365     print("问题三十张候选图已完成")
366
367
368 if __name__ == "__main__":
369     main()

```

```

1 """Evidence plots for Q4. All values read from actual solver outputs."""
2 from pathlib import Path
3 import sys, json
4 import numpy as np
5 import pandas as pd
6 import matplotlib
7 matplotlib.use('Agg')
8 import matplotlib.pyplot as plt
9 from PIL import Image
10 import 问题4_求解 as q
11
12 FIGURE_TOOLS=q.ROOT/'figure_tools'
13 sys.path.insert(0, str(FIGURE_TOOLS))

```

```

14 from export_figure import export_figure
15 from setup_style import setup_style
16 from utils.plot_style import publication_subplots
17
18 COLORS=['#0072B2', '#D55E00', '#009E73', '#CC79A7', '#666666']
19 STYLES=['-', '--', '-.', ':', '-']
20
21 def refinement_figure(verification):
22     fig, ax=publication_subplots(1, 1, width='report', aspect=.60)
23     ys=np.arange(len(verification))
24     ax.scatter(verification.table_max_abs_kg_kg, ys, s=38, color=COLORS[0], marker='D')
25     for y, value in zip(ys, verification.table_max_abs_kg_kg):
26         coefficient, exponent=f'{value:.4e}'.split('e')
27         ax.annotate(rf'${coefficient}\times 10^{{{int(exponent)}}}$ kg/kg',
28                     (value, y), xytext=(10, 0), textcoords='offset points',
29                     va='center', ha='left', fontsize=9)
30     ax.axvline(5e-5, color=COLORS[1], ls='--', lw=1, label='四位小数半单位')
31     labels=[v.replace('space_', '网格').replace('time_', '时间步').replace('_', '/') for v in
32             verification.scenario]
33     ax.set_yticks(ys, labels); ax.set_xlabel('共同表6位置的最大绝对差 / (kg/kg)'); ax.set_ylim(-.6, len(
34     ys)-.4)
35     ax.ticklabel_format(axis='x', style='sci', scilimits=(0, 0)); ax.legend(loc='lower right', frameon=
36     False)
37     return fig
38
39 def figure16_only():
40     setup_style(journal='general', lang='zh', serif_for_zh=True)
41     plt.rcParams.update({'font.size': 9, 'axes.labelsize': 9, 'xtick.labelsize': 8, 'ytick.labelsize': 8,
42                          'legend.fontsize': 8, 'svg.fonttype': 'none', 'pdf.fonttype': 42})
43     verification=pd.read_csv(q.ROOT/'results/问题4_数值加密.csv')
44     fig=refinement_figure(verification)
45     stem=q.ROOT/'figures/q4/process_q4_refinement'
46     export_figure(fig, str(stem), formats=['pdf', 'svg', 'png'], dpi=300, size_inches=tuple(fig.
47     get_size_inches()), grayscale_preview=True)
48     gp=stem.with_name(stem.name+'_grayscale.png')
49     with Image.open(gp) as im: im.copy().save(stem.parent/'_qa'/gp.name, dpi=(300, 300))
50     gp.unlink()
51     plt.close(fig)
52
53 def main(out=q.ROOT):
54     out=Path(out); res=out/'results'; figdir=out/'figures/q4'; figdir.mkdir(parents=True, exist_ok=True)
55     data=res/'q4_plot_data'; data.mkdir(parents=True, exist_ok=True)
56     setup_style(journal='general', lang='zh', serif_for_zh=True)
57     plt.rcParams.update({'font.size': 9, 'axes.labelsize': 9, 'xtick.labelsize': 8, 'ytick.labelsize': 8,
58                          'legend.fontsize': 8, 'svg.fonttype': 'none', 'pdf.fonttype': 42})
59     air, rad, ss=q.input_data(); base=q.load_run(res/'q4_runs/main.npz'); fixed=q.load_run(res/'q4_runs/
60     fixed.npz')
61     summary=json.loads((res/'问题4_结果摘要.json').read_text(encoding='utf-8'))

```

```

55 scenarios=pd.read_csv(res/'问题4_情景汇总.csv');verification=pd.read_csv(res/'问题4_数值加密.csv')
56 contracts=[]
57 def figax():
58     return publication_subplots(1,1,width='report',aspect=.60)
59 def finish(fig,name,claim,source,kind='折线',caption=''):
60     size=tuple(fig.get_size_inches())
61     contracts.append({'name':name,'question':'q4','claim':claim,'source':source,'chart':kind,'
        layout':'single primary quantitative panel; shared-variable panels only when needed','backend':'
        Python','size_inches':size,'dpi':300,'caption':caption or claim,'statistical_scope':'
        deterministic model scenarios; no statistical CI or significance claim'})
62     export_figure(fig,figdir/name,formats=['pdf','svg','png'],dpi=300,size_inches=size,
        grayscale_preview=True)
63     gp=figdir/(name+'_grayscale.png')
64     qa=figdir/'_qa';qa.mkdir(exist_ok=True)
65     with Image.open(gp) as im:im.copy().save(qa/gp.name,dpi=(300,300))
66     gp.unlink()
67     plt.close(fig)
68
69 # Raw 1: measured radius and the exact interpolation.
70 fig,ax=figax();hours=rad.time_s/3600
71 ax.plot(hours,rad.radius_m*100,color=COLORS[0],lw=1.5)
72 ax.scatter(hours[:4],rad.radius_m[:4]*100,s=12,facecolors='white',edgecolors=COLORS[0],zorder
    =3)
73 ax.set(xlabel='时间 / h',ylabel='实测外半径 / cm',xlim=(0,72),ylim=(1.1,2.06))
74 pd.DataFrame({'time_h':hours,'radius_cm':rad.radius_m*100}).to_csv(data/'radius.csv',index=False)
75 finish(fig,'raw_q4_radius','附件2外半径早期快速减小,后期趋于平台。','problem A/附件/附件2.xlsx',
    caption='全部145点线性连接;空心标记每4点展示一次,未删去计算输入。')
76
77 # Raw 2: actual stable-period fluctuations; separate units.
78 mask=(air.time_s>=9000)&(air.time_s<=14400)
79 fig,axes=plt.subplots(1,2,figsize=(6.3,3.5),layout='constrained')
80 axes[0].hist(air.temperature_c[mask],bins=8,color=COLORS[0],edgecolor='white')
81 axes[1].hist(air.moisture[mask],bins=8,color=COLORS[1],edgecolor='white')
82 for ax in axes:ax.set_ylabel('观测点数');ax.spines[['top','right']].set_visible(False)
83 axes[0].set_xlabel('稳定段空气温度 / °C');axes[1].set_xlabel('稳定段空气水分浓度 / (kg/kg)')
84 axes[1].ticklabel_format(axis='x',style='sci',scilimits=(0,0),useOffset=False)
85 pd.DataFrame({'temperature_C':air.temperature_c[mask],'air_C':air.moisture[mask]}).to_csv(data/'
    stable_air.csv',index=False)
86 finish(fig,'raw_q4_air_distribution','91个稳定段样本在平台值附近波动。','problem A/附件/附件1.
    xls','直方图','9000-14400 s的91点;图示观测分布,不是参数置信区间。')
87
88 # Raw 3: radius-loss rate calculated directly from adjacent measurements.
89 speed=-np.diff(rad.radius_m*100)/np.diff(hours)
90 midR=(rad.radius_m[1:]+rad.radius_m[:-1])*50
91 fig,ax=figax();ax.scatter(midR,speed,s=14,color=COLORS[0],alpha=.8)
92 ax.set(xlabel='相邻时段平均半径 / cm',ylabel='分段收缩速率 / (cm/h)',ylim=(-.01,max(speed)*1.08)

```

```

)
93 pd.DataFrame({'mean_radius_cm':midR,'shrink_rate_cm_h':speed}).to_csv(data/'shrink_rate.csv',
    index=False)
94 finish(fig,'raw_q4_shrink_rate','原始半径记录对应的收缩速率随尺寸减小趋近零。','results/
    q4_plot_data/shrink_rate.csv','散点图','144个相邻观测区间的差商;零速率保留,不拟合相关性。')
95
96 # Process 1: admissible material mappings, final g=1; not measured displacements.
97 fig,ax=figax();xi=np.linspace(0,1,101)
98 mapping={'xi':xi}
99 for j,a in enumerate([0,.1,.2]):
100     y=xi+a*xi*(1-xi);mapping[str(a)]=y
101     ax.plot(xi,y,color=COLORS[j],ls=STYLES[j],label=f'a* = {a:g}')
102 ax.set(xlabel='初始材料坐标  $\xi$ ',ylabel='当前位置 r/R',xlim=(0,1),ylim=(0,1));ax.legend(loc='
    upper left',frameon=False)
103 pd.DataFrame(mapping).to_csv(data/'mapping.csv',index=False)
104 finish(fig,'process_q4_mapping','三种映射共享中心和表面端点,改变内部位置分配。','题目分析报告.
    md Q4映射','折线','显示g=1情形;映射是确定性假设,未被内部位移观测标定。')
105
106 # Process 2: counts represent actual iteration frequencies, not means.
107 hist=pd.read_csv(res/'问题4_迭代统计.csv');fig,ax=figax()
108 ax.bar(hist.iterations.astype(str),hist.step_count,color=COLORS[0],width=.55)
109 ax.set(xlabel='每个时间步的 Picard 迭代次数',ylabel='时间步数',ylim=(0,hist.step_count.max()
    *1.12))
110 finish(fig,'process_q4_iterations','每秒耦合迭代在规定的40次上限内收敛。','results/问题4_迭代统计.
    csv','频数柱状图','频数统计覆盖主模型全部时间步,非采样频数。')
111
112 # Process 3: numerical errors compared on common six-hour table locations.
113 fig=refinement_figure(verification)
114 finish(fig,'process_q4_refinement','时空加密误差按预先冻结的表值门槛核验。','results/问题4_数值
    加密.csv','点图','共同6h时刻与固定实际距离/表面值比较;终点时间差另外列在CSV中。')
115
116 # Result 1: full threshold trajectories, semilog makes the endpoint visible.
117 fig,ax=figax()
118 for j,(label,s) in enumerate([('移动域主模型',base),('同物性固定半径',fixed)]):
119     ax.plot(s['time_s']/3600,s['moisture'].max(axis=1),color=COLORS[j],ls=STYLES[j],label=label)
120 ax.axhline(.15,color=COLORS[4],ls=':',lw=1,label='干燥阈值 0.15')
121 ax.set(xlabel='时间 / h',ylabel='全域最大含水率 / (kg/kg)',yscale='log',ylim=(.12,3.0))
122 ax.legend(frameon=False,loc='upper right')
123 finish(fig,'result_q4_threshold','同附录4物性下,主模型与固定半径对照在不同时刻达到阈值。','
    results/q4_runs/main.npz; fixed.npz','半对数折线','最大值在全部计算节点上取值;精确首次跨越按1s
    步检测,图中为每60s记录及终点。')
124
125 # Result 2: actual physical coordinates; gaps outside R are left white.
126 times=base['time_s'];Tidx=np.unique(np.r_[np.linspace(0,len(times)-1,241).astype(int),len(times)
    -1])
127 phys=np.linspace(0,.02,101);z=np.full((len(Tidx),len(phys)),np.nan)
128 for ii,j in enumerate(Tidx):
129     R=base['radius_m'][j];valid=phys<=R

```

```

130     z[ii, valid]=np. interp(phys[valid], np. linspace(0, R, base['moisture']. shape[1]), base['moisture']
131     ][j])
132 fig, ax=figax(); mesh=ax. pcolormesh(times[Tidx]/3600, phys*100, z. T, cmap=' viridis', shading=' nearest'
133     , rasterized=False, vmin=0, vmax=2. 55)
134 mesh. set_edgecolor(' face')
135 ax. plot(times[Tidx]/3600, base[' radius_m' ][Tidx]*100, color=' black', lw=. 9)
136 ax. set(xlabel=' 时间 / h', ylabel=' 到轴线的实际距离 / cm', ylim=(0, 2))
137 cb=fig. colorbar(mesh, ax=ax, pad=. 03); cb. set_label(' 含水率 / (kg/kg)'); cb. solids. set_rasterized(
138     False)
139 np. savez_compressed(data/' physical_field. npz', time_h=times[Tidx]/3600, radius_cm=phys*100,
140     moisture=z)
141 finish(fig, 'result_q4_moving_field', '药材内部区域随表面收缩而减小, 内部含水率逐渐降低.', '
142     results/q4_plot_data/physical_field. npz', '热力图', '显示241个时间索引(含终点)、101个物理距离;
143     白色区域为药材域外, 非计算缺失。')
144
145 # Result 3: paired counterfactual estimates (no fabricated uncertainty bars).
146 fig, ax=figax(); vals=[summary[' dry_time_h' ], summary[' fixed_radius_dry_time_h' ]]
147 for j, v in enumerate(vals):
148     ax. plot([0, v], [j, j], color=COLORS[j], lw=1. 4); ax. scatter([v], [j], color=COLORS[j], marker=[' o', '
149     s' ][j], s=38)
150     ax. annotate(f' {v:. 4f} h', (v, j), xytext=(6, 0), textcoords=' offset points', va=' center', fontsize
151     =9)
152 ax. set_yticks([0, 1], [' 移动域主模型', ' 同物性固定半径']); ax. set(xlabel=' 干燥时间 / h', xlim=(0, max(
153     vals)*1. 22), ylim=(-. 6, 1. 6))
154 finish(fig, 'result_q4_geometry_effect', '仅更改是否收缩, 分离附录4体系内的净几何效应.', 'results/
155     问题4_结果摘要. json', '配对点图', '两点均来自确定性计算, 所有物性、初边值与数值设置相同。')
156
157 # Result 4: mapping uncertainty is distinct from geometric effect.
158 fig, ax=figax(); es=[0, summary[' eta_shape_percent' ][' 0. 1' ], summary[' eta_shape_percent' ][' 0. 2' ]]
159 ax. scatter(es, [0, 1, 2], s=40, color=COLORS[0], marker=' D'); ax. axvline(0, color=COLORS[4], lw=. 8)
160 ax. set_yticks([0, 1, 2], [' a* = 0', ' a* = 0. 1', ' a* = 0. 2']); ax. set_xlabel(' 相对主模型的干燥时长变化
161     / %'); ax. set_ylim(-. 5, 2. 5)
162 finish(fig, 'result_q4_shape_sensitivity', '相同实测外半径下, 内部映射假设引起可量化的时长变化.', '
163     results/问题4_结果摘要. json', '点图', '三个未校准的确定性映射情景; 点间差异不是置信区间。')
164
165 # Result 5: genuine profiles at selected recorded instants.
166 fig, ax=figax(); target=[6*3600, 24*3600, 48*3600, times[-1]]
167 for j, tt in enumerate(sorted(set(v for v in target if v<=times[-1]))):
168     i=int(np. searchsorted(times, tt)); R=base[' radius_m' ][i]
169     ax. plot(np. linspace(0, R*100, base[' moisture' ]. shape[1]), base[' moisture' ][i], color=COLORS[j],
170     ls=STYLES[j], label=f' {tt/3600:. 2f} h')
171 ax. set(xlabel=' 到轴线的实际距离 / cm', ylabel=' 含水率 / (kg/kg)', xlim=(0, 2), ylim=(0, 2. 7)); ax.
172     legend(frameon=False)
173 finish(fig, 'result_q4_profiles', '径向梯度与表面内移共同决定固定距离处的水分变化.', 'results/
174     q4_runs/main. npz', '折线', '各条曲线止于当时真实表面, 不延长到初始2cm。')
175
176 # Additional boundary sensitivity, with signed perturbations.
177 names=[' air_T_minus', ' air_T_plus', ' air_C_minus', ' air_C_plus', ' last_air']; labels=[' 温度 -1 SD', '

```

```

    温度 +1 SD', '空气水分 -1 SD', '空气水分 +1 SD', '末条边界替代']
163 sub=scenarios.set_index('scenario').loc[names];fig,ax=figax()
164 ax.scatter(sub.relative_to_main_percent,np.arange(5),color=COLORS[0],s=35)
165 ax.axvline(0,color=COLORS[4],ls=':',lw=1);ax.set_yticks(np.arange(5),labels);ax.set_xlabel='相对
    主模型的时长变化 / %',ylim=(-.5,4.5))
166 finish(fig,'process_q4_air_sensitivity','长期空气延拓不确定性与内部收缩假设分别评估。','results/
    问题4_情景汇总.csv','点图','单因素扰动,SD为91点总体标准差;不是随机置信区间。')
167
168 (figdir/'图表契约.json').write_text(json.dumps(contracts,ensure_ascii=False,indent=2),encoding='
    utf-8')
169 text=['# 问题四图表索引','', '仅q4范围: 3张原始图、4张过程图、5张结果图。每幅均有PDF、SVG、
    300DPI PNG和灰度预览。','']
170 for c in contracts:text += [f"## {c['name']}",c['claim'],f"图注: {c['caption']}",f"证据: {c['
    source']}",'']
171 (figdir/'图表说明.md').write_text('\n'.join(text),encoding='utf-8')
172
173 if __name__=='__main__':
174     if '--figurel6-only' in sys.argv:figurel6_only()
175     else:main()

```

```

1 """Reproduce four focused paper figures from existing, read-only solver outputs."""
2 from pathlib import Path
3 import hashlib
4 import json
5 import sys
6 import numpy as np
7 import pandas as pd
8 import matplotlib
9 matplotlib.use('Agg')
10 import matplotlib.pyplot as plt
11 from PIL import Image
12
13 ROOT = Path(__file__).resolve().parent
14 sys.path.insert(0, str(ROOT / 'figure_tools'))
15 from setup_style import setup_style
16 from export_figure import export_figure
17 from visual_qa import audit_layout, render_preview
18
19 OUT = ROOT / 'figures/paper_review'
20 QA = ROOT / 'build/figure_review'
21 OUT.mkdir(exist_ok=True)
22 QA.mkdir(exist_ok=True)
23 setup_style(journal='general', lang='zh', serif_for_zh=True)
24 plt.rcParams.update({'font.size':9, 'axes.labelsize':9, 'axes.titlesize':9,
25                     'xtick.labelsize':8, 'ytick.labelsize':8, 'legend.fontsize':8,
26                     'pdf.fonttype':42, 'svg.fonttype':'none', 'axes.unicode_minus':False})
27 BLUE, ORANGE, GREEN = '#0072B2', '#D55E00', '#009E73'

```

```

28 contracts=[]
29
30 def finish(fig, name, claim, sources, caption):
31     for ax in fig.axes:
32         ax.spines[['top', 'right']].set_visible(False)
33     render_preview(fig, str(QA / (name+'.png')), dpi=170)
34     fig.canvas.draw() # Restore renderer at figure DPI after the preview save.
35     issues=audit_layout(fig)
36     if any(level=='FAIL' for level, _ in issues):
37         raise RuntimeError(issues)
38     print(name, issues)
39     export_figure(fig, str(OUT/name), formats=['pdf', 'svg', 'png'], dpi=300,
40                 size_inches=tuple(fig.get_size_inches()), grayscale_preview=True)
41     gp=OUT/(name+'_grayscale.png')
42     with Image.open(gp) as im:
43         gray=im.copy()
44         gray.save(gp, dpi=(300, 300))
45     contracts.append({'name':name, 'claim':claim, 'sources':[
46         {'path':s, 'sha256':hashlib.sha256((ROOT/s).read_bytes()).hexdigest()} for s in sources],
47         'caption':caption, 'backend':'Python', 'layout':'quantitative panels',
48         'size_inches':list(fig.get_size_inches()), 'dpi':300, 'qa':issues,
49         'scope':'deterministic simulation outputs; no inferential confidence interval'})
50     plt.close(fig)
51
52 # Q2: the two differences have different units and therefore separate axes.
53 t=pd.read_excel(ROOT/'results/result2.xlsx', sheet_name='温度', header=0)
54 c=pd.read_excel(ROOT/'results/result2.xlsx', sheet_name='水分浓度', header=0)
55 assert t.shape==c.shape==(10800, 22)
56 assert np.array_equal(t.iloc[:, 0], c.iloc[:, 0])
57 assert t.iloc[0, 0]==1 and t.iloc[-1, 0]==10800
58 assert float(t.columns[1])==0 and float(t.columns[-1])==2
59 assert np.isfinite(t.to_numpy(float)).all() and np.isfinite(c.to_numpy(float)).all()
60 hours=t.iloc[:, 0].to_numpy(float)/3600
61 dt=t.iloc[:, -1].to_numpy(float)-t.iloc[:, 1].to_numpy(float)
62 dc=c.iloc[:, 1].to_numpy(float)-c.iloc[:, -1].to_numpy(float)
63 assert abs(dt[-1]-.1169)<1e-9 and abs(dc[-1]-.7581)<1e-9
64 pd.DataFrame({'time_h':hours, 'surface_minus_center_T_C':dt, 'center_minus_surface_C_kg_kg':dc}).
65     to_csv(QA/'q2_gap_data.csv', index=False)
66 fig, ax=plt.subplots(2, 1, figsize=(5.8, 3.3), sharex=True, layout='constrained')
67 for ax, v, color, style, marker, label in [
68     (axs[0], dt, ORANGE, '-', 'o', '表面-中心温差 / °C'),
69     (axs[1], dc, BLUE, '--', 's', '中心-表面含水率差 / (kg/kg)')]:
70     ax.plot(hours, v, color=color, ls=style, lw=1.15)
71     ii=np.searchsorted(hours, np.arange(.5, 3.01, .5))
72     ax.plot(hours[ii], v[ii], ls='none', marker=marker, ms=3.5, color=color)
73     ax.set(ylabel=label, xlim=(0, 3.1), ylim=(0, float(v.max())*1.22))
74     unit='°C' if ax is axs[0] else 'kg/kg'
75     ax.annotate(f'{v[-1]:.4f} {unit}', (3, v[-1]), xytext=(-6, 14), textcoords='offset points', ha='right'

```

```

    , fontsize=9)
75 ax[1].set_xlabel('时间 / h')
76 finish(fig, 'result_q2_gap_review', '温度场先趋于均匀, 水分梯度持续存在。',
77         ['results/result2.xlsx'], '全部10800条秒级输出; 符号标注每0.5h的报告时刻; 差值取自四位小数结果
      表。')
78
79 # Q2: preserve the original fields, colors and display sampling, but remove PDF mesh seams.
80 for frame, name, label, cmap in [
81     (t, 'result_q2_temperature_review', '温度 / °C', 'inferno'),
82     (c, 'result_q2_moisture_review', '含水率 / (kg/kg)', 'cividis_r')]:
83     idx=np.unique(np.r_[np.arange(0, len(frame), 30), len(frame)-1])
84     radii=np.array([float(x) for x in frame.columns[1:]])
85     fig, ax=plt.subplots(figsize=(5.8, 3.0), layout='constrained')
86     mesh=ax.pcolormesh(hours[idx], radii, frame.iloc[idx, 1:].to_numpy(float).T,
87                       shading='auto', cmap=cmap, rasterized=False)
88     mesh.set_edgecolor('face')
89     ax.set_xlabel='时间 / h', ylabel='到轴线的实际距离 / cm', xlim=(0, 3), ylim=(0, 2))
90     bar=fig.colorbar(mesh, ax=ax, pad=.025)
91     bar.set_label(label)
92     bar.solids.set_edgecolor('face')
93     bar.solids.set_rasterized(False)
94     finish(fig, name, '保留原径向场信息, 修复矢量网格在PDF渲染时的白色细缝。',
95            ['results/result2.xlsx'], '沿用原图每30条秒级输出显示一条并追加末条的规则; 21个输出半径均
      保留; 网格边缘填同色, 不改变数值。')
96
97 # Q3: preserve the full trajectory and show actual saved records without invented one-second points.
98 p=pd.read_csv(ROOT/'results/问题3_关键过程.csv')
99 assert len(p)==3447 and p.time_s.is_monotonic_increasing
100 assert not p.isna().any().any()
101 last=p.tail(2); end=float(last.time_s.iloc[-1]); excess=(last.maximum_moisture-.15)*1e6
102 assert list(last.time_s)==[206700, 206756] and excess.iloc[0]>0>excess.iloc[-1]
103 fig, ax=plt.subplots(1, 2, figsize=(5.8, 2.8), layout='constrained', width_ratios=[1.15, 1])
104 ax=axs[0]
105 ax.plot(p.time_h, p.maximum_moisture, color=BLUE, lw=1.25)
106 ax.axhline(.15, color=ORANGE, ls='--', lw=.9)
107 ax.plot(end/3600, p.maximum_moisture.iloc[-1], 'o', ms=4, color=BLUE)
108 ax.annotate(f' {end/3600:.4f} h', (end/3600, .15), xytext=(-5, 32), textcoords='offset points', ha='right',
      arrowprops={'arrowstyle':'-', 'lw':.6})
109 ax.text(.97, .89, '阈值 0.15 kg/kg', ha='right', transform=ax.transAxes, color=ORANGE)
110 ax.set_xlabel='时间 / h', ylabel='全域最大含水率 / (kg/kg)', xlim=(0, 60), ylim=(0, 2.7))
111 ax=axs[1]
112 for x, y, marker, color in zip(last.time_s-end, excess, ['s', 'o'], [ORANGE, BLUE]):
113     ax.scatter(x, y, s=26, marker=marker, color=color, zorder=3)
114 ax.axhline(0, color='#555555', ls='--', lw=.8)
115 ax.set_xlabel='相对终点时间 / s', ylabel='r 阈值偏差 / ($10^{-6}$ kg/kg)', xlim=(-70, 12), ylim=(-4, 21),
      xticks=[-60, -40, -20, 0])
116 ax.annotate(f'前一常规记录{n+{excess.iloc[0]:.4f}}', (-56, excess.iloc[0]), xytext=(7, -3), textcoords='
      offset points', va='top', fontsize=8)

```

```

117 ax.annotate(f' 终点记录\n{excess.iloc[-1]:.4f}', (0, excess.iloc[-1]), xytext=(-10, 24), textcoords='
    offset points', ha='right', fontsize=8, arrowprops={'arrowstyle': '-', 'lw': .6})
118 finish(fig, 'result_q3_threshold_review', '终点已保存值严格低于阈值, 首次时刻由原求解器逐秒检测.',
119         ['results/问题3_关键过程.csv', 'results/问题3_结果摘要.json'],
120         '左图全程; 右图仅前一60s常规记录及追加终点, 没有逐秒补点、插值连线或重新估计交点。')
121
122 # Q4: explain the same deterministic mapping family alongside its endpoint effect.
123 s=json.loads((ROOT/'results/问题4_结果摘要.json').read_text(encoding='utf-8'))
124 r=pd.read_csv(ROOT/'results/q4_plot_data/radius.csv')
125 g=(1-s['radius_end_cm']/r.radius_cm.iloc[0])/(1-r.radius_cm.iloc[-1]/r.radius_cm.iloc[0])
126 xi=np.linspace(0, 1, 201)
127 fig, axs=plt.subplots(1, 2, figsize=(5.8, 2.65), layout='constrained')
128 for a, col, ls in [(0, BLUE, '-'), (.1, ORANGE, '--'), (.2, GREEN, '-.')]:
129     axs[0].plot(xi, a*g*xi*(1-xi), color=col, ls=ls, lw=1.2, label=f'r' '$a_{a}$')
130 axs[0].set(xlabel=r' 初始材料坐标 $xi$', ylabel=r' 映射偏离 $r/R-xi$', xlim=(0, 1), ylim=(-.004, .075))
131 axs[0].legend(frameon=False, loc='upper center', ncol=3, fontsize=7, handlelength=1.6, columnspacing=.8)
132 values=[0, s['eta_shape_percent'] ['0.1'], s['eta_shape_percent'] ['0.2']]
133 for j, (v, col, marker) in enumerate(zip(values, [BLUE, ORANGE, GREEN], ['o', 's', 'D'])):
134     axs[1].scatter(v, j, color=col, marker=marker, s=26, zorder=3)
135     axs[1].annotate(f' {v:.4f}%', (v, j), xytext=(6, 0), textcoords='offset points', va='center', fontsize
        =8)
136 axs[1].axvline(0, color=' #777777', lw=.7)
137 axs[1].set(yticks=[0, 1, 2], yticklabels=[r' '$a_{a}=0$', r' '$a_{a}=0.1$', r' '$a_{a}=0.2$'],
138           xlabel=' 相对主模型的时长变化 / %', xlim=(-.03, .43), ylim=(-.45, 2.45))
139 pd.DataFrame({'xi':xi, **{f'a_{a}':a*g*xi*(1-xi) for a in [0, .1, .2]}}).to_csv(QA/'q4_mapping_data.csv',
    index=False)
140 finish(fig, 'result_q4_shape_review', '相同外半径下的内部映射扰动对应所检验情景中至多0.3143%的时长变
    化。',
141         ['results/问题4_结果摘要.json', 'results/q4_plot_data/radius.csv', 'results/问题4_情景汇总.csv'],
142         f' 左幅按主模型终点t=51.0775h计算, g={g:.10f}, 绘制的是假设映射而非实测位移; 右幅为确定性情景, 无置信区间。')
143
144 # Q4: preserve the physically matched counterfactual and label actual endpoint times.
145 fig, ax=plt.subplots(figsize=(5.8, 2.85), layout='constrained')
146 for key, label, col, ls, offset in [
147     ('main', '收缩主模型', BLUE, '-', (-14, 27)),
148     ('fixed', '同物性固定半径', ORANGE, '--', (-7, 45))]:
149     with np.load(ROOT/f'results/q4_runs/{key}.npz') as z:
150         tt=z['time_s']/3600; mm=z['moisture'].max(axis=1)
151         ax.plot(tt, mm, color=col, ls=ls, lw=1.25, label=label)
152         ax.plot(tt[-1], mm[-1], marker='o' if key=='main' else 's', color=col, ms=4)
153         ax.annotate(f' {tt[-1]:.4f} h', (tt[-1], mm[-1]), xytext=offset, textcoords='offset points', ha='right',
            color=col, fontsize=9, arrowprops={'arrowstyle': '-', 'lw': .7, 'color': col})
154 ax.axhline(.15, color=' #777777', ls=':', lw=1, label=' 干燥阈值 0.15 kg/kg')
155 ax.set(xlabel=' 时间 / h', ylabel=' 全域最大含水率 / (kg/kg)', yscale='log', ylim=(.12, 3), xlim=(0, 140))
156 ax.legend(frameon=False, loc='upper right')
157 finish(fig, 'result_q4_threshold_review', '同附录四物性的固定半径对照分离净几何效应。',

```

```

158     ['results/q4_runs/main.npz', 'results/q4_runs/fixed.npz'],
159     '全部60s记录及追加终点；终点时间直接读实际求解记录，半对数纵轴。')
160
161 (QA/'figure_contracts.json').write_text(json.dumps(contracts, ensure_ascii=False, indent=2), encoding='
    utf-8')
162 print('Six focused figures generated; original figures and solver outputs unchanged.')
```

2.5 公共辅助程序

```

1 #!/usr/bin/env python3
2 """数学建模论文图的统一样式工具（向后兼容层）。
3
4 本文件保留原有 API 以兼容 ``from utils.plot_style import ...`` 的外部代码。
5 实际常量和工具函数已迁移到 tools/figure/scripts/style_constants.py。
6 布局审计和导出功能由 tools/figure/scripts/visual_qa.py 和
7 tools/figure/scripts/export_figure.py 替代。
8 """
9
10 from __future__ import annotations
11
12 import logging
13 import unicodedata
14 import warnings
15 from pathlib import Path
16 from typing import Sequence
17
18
19 # -----
20 # SKILL_ROOT 定位
21 # -----
22
23 def _is_math_modeling_skill_root(path: Path) -> bool:
24     return (
25         (path / "SKILL.md").is_file()
26         and (path / "VERSION").is_file()
27         and (
28             path
29             / "references"
30             / "roles"
31             / "编程手"
32             / "scripts"
33             / "plot_style.py"
34         ).is_file()
35     )
36
37
38 def _find_skill_root(path: Path) -> Path | None:
```

```

39     directory = path.resolve().parent
40     return next(
41         (candidate for candidate in (directory, *directory.parents)
42         if _is_math_modeling_skill_root(candidate)),
43         None,
44     )
45
46
47 SKILL_ROOT = _find_skill_root(Path(__file__))
48
49 # -----
50 # 色盲安全配色
51 # -----
52
53 PALETTE = {
54     "primary": "#0072B2",
55     "secondary": "#E69F00",
56     "positive": "#009E73",
57     "contrast": "#D55E00",
58     "accent": "#CC79A7",
59     "sky": "#56B4E9",
60     "neutral": "#6B7280",
61     "dark": "#222222",
62 }
63
64 COLOR_SEQUENCE = tuple(PALETTE[name] for name in (
65     "primary",
66     "secondary",
67     "positive",
68     "contrast",
69     "accent",
70     "sky",
71     "neutral",
72 ))
73
74 # -----
75 # 论文栏宽 (英寸)
76 # -----
77
78 WIDTHS_IN = {
79     "single": 3.5,
80     "double": 7.2,
81     "report": 6.3,
82 }
83
84 # -----
85 # 字体选择
86 # -----

```

```

87
88 _CJK_SANS = (
89     "Noto Sans CJK SC",
90     "Source Han Sans SC",
91     "Microsoft YaHei",
92     "SimHei",
93     "PingFang SC",
94 )
95
96
97 def _available_fonts() -> set[str]:
98     from matplotlib import font_manager
99     return {item.name for item in font_manager.fontManager.ttflist}
100
101
102 def choose_font(language: str = "zh") -> str:
103     """选择可用字体；中文字体缺失时给出警告并安全回退。"""
104     if language not in {"zh", "en"}:
105         raise ValueError("language 只能是 'zh' 或 'en'")
106     available = _available_fonts()
107     if language == "zh":
108         for name in _CJK_SANS:
109             if name in available:
110                 return name
111         warnings.warn(
112             "未检测到常用中文字体，已回退到 DejaVu Sans；导出前必须检查中文和特殊符号是否缺字。",
113             RuntimeWarning,
114             stacklevel=2,
115         )
116     for name in ("Arial", "Helvetica", "DejaVu Sans"):
117         if name in available:
118             return name
119     return "DejaVu Sans"
120
121
122 # -----
123 # 尺寸计算
124 # -----
125
126 def figure_size(width: str = "report", aspect: float = 0.62) -> tuple[float, float]:
127     """按最终使用宽度返回英寸尺寸，避免在论文中二次大幅缩放。"""
128     if width not in WIDTHS_IN:
129         raise ValueError(f"未知宽度方案: {width}")
130     if aspect <= 0:
131         raise ValueError("aspect 必须大于 0")
132     width_in = WIDTHS_IN[width]
133     return width_in, width_in * aspect
134

```

```

135
136 # -----
137 # 子图创建
138 # -----
139
140 def publication_subplots(
141     nrows: int = 1,
142     ncols: int = 1,
143     *,
144     width: str = "report",
145     aspect: float = 0.62,
146     width_ratios: Sequence[float] | None = None,
147     height_ratios: Sequence[float] | None = None,
148     squeeze: bool = True,
149 ):
150     """按最终尺寸创建子图，并允许显式声明主次面板比例。"""
151     import matplotlib.pyplot as plt
152
153     if nrows < 1 or ncols < 1:
154         raise ValueError("nrows 和 ncols 必须大于 0")
155     if width_ratios is not None and len(width_ratios) != ncols:
156         raise ValueError("width_ratios 数量必须与 ncols 一致")
157     if height_ratios is not None and len(height_ratios) != nrows:
158         raise ValueError("height_ratios 数量必须与 nrows 一致")
159     gridspec_kw = {}
160     if width_ratios is not None:
161         gridspec_kw["width_ratios"] = list(width_ratios)
162     if height_ratios is not None:
163         gridspec_kw["height_ratios"] = list(height_ratios)
164     return plt.subplots(
165         nrows,
166         ncols,
167         figsize=figure_size(width, aspect),
168         layout="constrained",
169         gridspec_kw=gridspec_kw or None,
170         squeeze=squeeze,
171     )
172
173 # -----
174 # 面板标签
175 # -----
176
177
178 from collections.abc import Iterable as _Iterable
179
180
181 def add_panel_labels(
182     axes: _Iterable,

```

```

183     labels: Sequence[str] | None = None,
184     *,
185     x_offset_pt: float = -8.0,
186     y_offset_pt: float = 1.0,
187 ) -> None:
188     """在各面板左上外侧添加小写粗体编号。"""
189     axes_list = list(axes)
190     panel_labels = list(labels) if labels is not None else [chr(97 + i) for i in range(len(axes_list))]
191     if len(panel_labels) != len(axes_list):
192         raise ValueError("labels 数量必须与 axes 数量一致")
193     for axis, label in zip(axes_list, panel_labels):
194         axis.annotate(
195             label,
196             xy=(0, 1),
197             xycoords="axes fraction",
198             xytext=(x_offset_pt, y_offset_pt),
199             textcoords="offset points",
200             ha="right",
201             va="bottom",
202             fontsize=8,
203             fontweight="bold",
204             annotation_clip=False,
205         )
206
207
208 # -----
209 # 路径安全
210 # -----
211
212 def resolve_output_stem(output_stem: str | Path) -> Path:
213     """解析导出路径，并禁止把任务产物写回 Skill 目录。"""
214     stem = Path(output_stem).expanduser().resolve()
215     if stem.suffix.lower() in {".svg", ".png", ".pdf"}:
216         stem = stem.with_suffix("")
217     if any(
218         _is_math_modeling_skill_root(candidate)
219         for candidate in (stem.parent, *stem.parent.parents)
220     ):
221         raise ValueError("图形产物必须写入 PROJECT_ROOT，不能写入 SKILL_ROOT")
222     return stem
223
224
225 # -----
226 # 向后兼容：已被 tools/figure/ 新工具替代的函数
227 # 以下函数保留以兼容现有测试和外部代码。新代码请使用
228 # tools/figure/scripts/ 下的 setup_style、visual_qa、export_figure。
229 # -----

```

```

230
231 def apply_publication_style(language: str = "zh", width: str = "report") -> dict:
232     """应用出版样式基线（向后兼容）。新代码请用 setup_style()。"""
233     import matplotlib as mpl
234     from cycler import cycler
235
236     font_name = choose_font(language)
237     size = figure_size(width)
238     mpl.rcParams.update({
239         "figure.figsize": size,
240         "figure.constrained_layout.use": True,
241         "figure.facecolor": "white",
242         "axes.facecolor": "white",
243         "font.family": "sans-serif",
244         "font.sans-serif": [font_name, "Arial", "Helvetica", "DejaVu Sans"],
245         "font.size": 7.5,
246         "axes.titlesize": 8.0,
247         "axes.titleweight": "normal",
248         "axes.titlelocation": "left",
249         "axes.titlepad": 4.0,
250         "axes.labelsize": 8.0,
251         "xtick.labelsize": 7.0,
252         "ytick.labelsize": 7.0,
253         "legend.fontsize": 7.0,
254         "axes.linewidth": 0.7,
255         "lines.linewidth": 1.1,
256         "lines.markersize": 3.5,
257         "patch.linewidth": 0.6,
258         "hatch.linewidth": 0.4,
259         "xtick.major.size": 3.0,
260         "ytick.major.size": 3.0,
261         "xtick.major.width": 0.6,
262         "ytick.major.width": 0.6,
263         "axes.spines.top": False,
264         "axes.spines.right": False,
265         "axes.grid": False,
266         "legend.frameon": False,
267         "legend.handlelength": 1.6,
268         "legend.borderaxespad": 0.4,
269         "axes.unicode_minus": False,
270         "axes.prop_cycle": cycler(color=COLOR_SEQUENCE),
271         "svg.fonttype": "none",
272         "pdf.fonttype": 42,
273         "savefig.facecolor": "white",
274         "savefig.dpi": 300,
275     })
276     return {"font": font_name, "size_inches": size, "colors": COLOR_SEQUENCE}
277

```

```

278
279 class _GlyphHandler(logging.Handler):
280     def __init__(self) -> None:
281         super().__init__()
282         self.messages: list[str] = []
283
284     def emit(self, record) -> None:
285         message = record.getMessage()
286         if "Glyph" in message and "missing from font" in message:
287             self.messages.append(message)
288
289
290 def _labels_overlap(labels, renderer, axis: str) -> bool:
291     boxes = [
292         label.get_window_extent(renderer)
293         for label in labels
294         if label.get_visible() and label.get_text().strip()
295     ]
296     boxes.sort(key=(lambda box: box.x0) if axis == "x" else (lambda box: box.y0))
297     if axis == "x":
298         return any(left.x1 > right.x0 + 1 for left, right in zip(boxes, boxes[1:]))
299     return any(lower.y1 > upper.y0 + 1 for lower, upper in zip(boxes, boxes[1:]))
300
301
302 def audit_layout(fig) -> list[str]:
303     """在导出前检查缺字、画布外文字和相邻刻度重叠（向后兼容）。"""
304     import matplotlib.text as mtext
305
306     handler = _GlyphHandler()
307     logger = logging.getLogger("matplotlib")
308     logger.addHandler(handler)
309     try:
310         with warnings.catch_warnings(record=True) as caught:
311             warnings.simplefilter("always")
312             fig.canvas.draw()
313     finally:
314         logger.removeHandler(handler)
315
316     messages = handler.messages + [
317         str(item.message)
318         for item in caught
319         if "Glyph" in str(item.message) and "missing from font" in str(item.message)
320     ]
321     issues = [f"缺字: {message}" for message in dict.fromkeys(messages)]
322     renderer = fig.canvas.get_renderer()
323     width, height = fig.bbox.width, fig.bbox.height
324     tick_ids = {
325         id(label)

```

```

326     for axis in fig.axes
327         for label in (*axis.get_xticklabels(), *axis.get_yticklabels())
328     }
329     clipped = []
330     for text in fig.findobj(mtext.Text):
331         if id(text) in tick_ids or not text.get_visible() or not text.get_text().strip():
332             continue
333         box = text.get_window_extent(renderer)
334         if box.x0 < -1 or box.y0 < -1 or box.x1 > width + 1 or box.y1 > height + 1:
335             clipped.append(text.get_text().replace("\n", " ")[:24])
336     if clipped:
337         issues.append("文字可能超出画布: " + ", ".join(dict.fromkeys(clipped)))
338     for index, axis in enumerate(fig.axes, start=1):
339         if _labels_overlap(axis.get_xticklabels(), renderer, "x"):
340             issues.append(f"第 {index} 个坐标轴的 x 刻度标签重叠")
341         if _labels_overlap(axis.get_yticklabels(), renderer, "y"):
342             issues.append(f"第 {index} 个坐标轴的 y 刻度标签重叠")
343     return issues
344
345
346 def _display_width(text: str) -> int:
347     return sum(
348         2 if unicodedata.east_asian_width(character) in {"W", "F"} else 1
349         for character in text
350     )
351
352
353 def _is_colorbar_axis(axis) -> bool:
354     return axis.get_label() == "<colorbar>" or hasattr(axis, "_colorbar")
355
356
357 def audit_design(fig) -> list[str]:
358     """检查可由对象结构确定的高风险设计（向后兼容）。"""
359     from matplotlib.container import BarContainer
360
361     issues: list[str] = []
362     data_axes = [axis for axis in fig.axes if not _is_colorbar_axis(axis)]
363     colorbar_axes = [axis for axis in fig.axes if _is_colorbar_axis(axis)]
364     colorbar_mappables = {
365         id(axis._colorbar.mappable)
366         for axis in colorbar_axes
367         if hasattr(axis, "_colorbar") and getattr(axis._colorbar, "mappable", None) is not None
368     }
369     for index, axis in enumerate(data_axes, start=1):
370         for location in ("left", "center", "right"):
371             title = axis.get_title(loc=location).strip()
372             title_lines = title.splitlines()
373             if title and (

```

```

374         len(title_lines) > 1
375         or max(_display_width(line) for line in title_lines) > 28
376     ):
377         issues.append(
378             f"第 {index} 个坐标轴标题过长；完整论述应移到图注，面板内只保留短标题"
379         )
380         break
381
382 legend = axis.get_legend()
383 if legend is not None and len(legend.get_texts()) > 5:
384     issues.append(
385         f"第 {index} 个坐标轴图例超过 5 项；应直接标注、改为共享图例或拆分证据"
386     )
387
388 for line in axis.lines:
389     marker = line.get_marker()
390     if marker in {None, "", " ", "None", "none"}:
391         continue
392     point_count = len(line.get_xdata(orig=False))
393     if point_count > 25 and line.get_markevery() is None:
394         issues.append(
395             f"第 {index} 个坐标轴对 {point_count} 个点逐点绘制标记；"
396             "应取消标记或设置 markevery"
397         )
398         break
399
400 for container in axis.containers:
401     if not isinstance(container, BarContainer) or not container.patches:
402         continue
403     if container.orientation == "vertical":
404         lower, upper = axis.get_ylim()
405     else:
406         lower, upper = axis.get_xlim()
407     tolerance = max(abs(upper - lower), 1.0) * 1e-9
408     if lower > tolerance or upper < -tolerance:
409         issues.append(
410             f"第 {index} 个坐标轴的柱状图未从零开始；"
411             "若必须截断，应改用点图/区间图并显式说明"
412         )
413         break
414
415 for image in axis.images:
416     values = image.get_array()
417     if (
418         getattr(values, "size", 0) <= 4
419         and len(axis.texts) >= getattr(values, "size", 0)
420         and id(image) in colorbar_mappables
421     ):

```

```

422         issues.append(
423             f"第 {index} 个坐标轴是已标数值的 2×2 小矩阵，"
424             "不应再使用冗余 colorbar"
425         )
426         break
427     for legend in fig.legends:
428         if len(legend.get_texts()) > 5:
429             issues.append("整图共享图例超过 5 项；应直接标注、拆分证据或突出核心系列")
430             break
431     return issues
432
433
434 def _save_grayscale_preview(png_path: Path, dpi: int) -> Path:
435     import matplotlib.image as image_io
436     import numpy as np
437
438     pixels = image_io.imread(png_path)
439     rgb = pixels[..., :3]
440     if pixels.shape[-1] == 4:
441         alpha = pixels[..., 3:4]
442         rgb = rgb * alpha + (1 - alpha)
443     grayscale = np.dot(rgb, (0.2126, 0.7152, 0.0722))
444     output = png_path.parent / "_qa" / f"{png_path.stem}_grayscale.png"
445     output.parent.mkdir(parents=True, exist_ok=True)
446     image_io.imsave(output, grayscale, cmap="gray", vmin=0, vmax=1, dpi=dpi)
447     return output
448
449
450 def export_figure(
451     fig,
452     output_stem: str | Path,
453     *,
454     dpi: int = 300,
455     grayscale_preview: bool = True,
456     strict_layout: bool = True,
457     strict_design: bool = True,
458 ) -> dict[str, str]:
459     """按固定物理尺寸导出（向后兼容）。新代码请用 tools/figure/scripts/export_figure.py。"""
460     if dpi < 300:
461         raise ValueError("论文图 PNG 的 dpi 不能低于 300")
462     layout_issues = audit_layout(fig)
463     if strict_layout and layout_issues:
464         raise ValueError("版面预检未通过：" + "; ".join(layout_issues))
465     design_issues = audit_design(fig)
466     if strict_design and design_issues:
467         raise ValueError("出版设计预检未通过：" + "; ".join(design_issues))
468     stem = resolve_output_stem(output_stem)
469     stem.parent.mkdir(parents=True, exist_ok=True)

```

```

470     svg_path = stem.with_suffix(".svg")
471     png_path = stem.with_suffix(".png")
472     fig.savefig(svg_path)
473     fig.savefig(png_path, dpi=dpi)
474     outputs = {"svg": str(svg_path), "png": str(png_path)}
475     if grayscale_preview:
476         outputs["grayscale"] = str(_save_grayscale_preview(png_path, dpi))
477     return outputs
478
479
480 __all__ = [
481     "COLOR_SEQUENCE",
482     "PALETTE",
483     "WIDTHS_IN",
484     "SKILL_ROOT",
485     "add_panel_labels",
486     "apply_publication_style",
487     "audit_design",
488     "audit_layout",
489     "choose_font",
490     "export_figure",
491     "figure_size",
492     "publication_subplots",
493     "resolve_output_stem",
494 ]

```

```

1 """
2 scipilot-figure-skill :: setup_style.py
3 =====
4 Publication-grade matplotlib / seaborn style configuration.
5
6 应用出版级样式预设。支持 nature / ieee / science / general 四种期刊预设，
7 支持中英文（lang='zh'/'en'），中文模式按优先级自动查找
8 Noto Sans CJK SC > Source Han Sans SC > SimHei > Microsoft YaHei
9 并修正负号渲染。SciencePlots 可选——装了就用，没装回退到内置等效预设。
10
11 Usage
12 -----
13     from setup_style import setup_style
14
15     # Nature 单栏英文图
16     setup_style(journal='nature', lang='en')
17
18     # 中文期刊通用
19     setup_style(journal='general', lang='zh')
20
21     # 关闭 SciencePlots 强制用内置预设

```

```

22     setup_style(journal='iee', use_sciplots=False)
23
24 CLI: ``python setup_style.py --list-fonts`` 列出可用 CJK 字体。
25 """
26 from __future__ import annotations
27
28 import argparse
29 import sys
30 import warnings
31
32 import matplotlib
33 import matplotlib.font_manager as fm
34 import matplotlib.pyplot as plt
35
36
37 # 期刊预设: figsize 单位为英寸, 对应该期刊单栏标称宽度
38 # Nature: 89mm = 3.5 in 双栏: 183mm = 7.2 in
39 # Science: 与 Nature 接近
40 # IEEE: 单栏 3.5 in, 双栏 7.16 in
41 JOURNAL_PRESETS = {
42     "nature": {
43         "figure.figsize": (3.5, 2.625), # 4:3 ratio @ 89mm
44         "figure.dpi": 150,
45         "savefig.dpi": 300,
46         "font.family": "sans-serif",
47         "font.sans-serif": ["Helvetica", "Arial", "DejaVu Sans"],
48         "font.size": 7,
49         "axes.labelsize": 8,
50         "axes.titlesize": 8,
51         "xtick.labelsize": 7,
52         "ytick.labelsize": 7,
53         "legend.fontsize": 7,
54         "lines.linewidth": 1.0,
55         "lines.markersize": 4,
56         "axes.linewidth": 0.6,
57         "xtick.major.width": 0.6,
58         "ytick.major.width": 0.6,
59         "xtick.minor.width": 0.4,
60         "ytick.minor.width": 0.4,
61         "axes.spines.top": False,
62         "axes.spines.right": False,
63         "pdf.fonttype": 42,
64         "ps.fonttype": 42,
65         "svg.fonttype": "none",
66     },
67     "science": {
68         "figure.figsize": (3.5, 2.625),
69         "figure.dpi": 150,

```

```

70     "savefig.dpi": 300,
71     "font.family": "sans-serif",
72     "font.sans-serif": ["Helvetica", "Arial", "DejaVu Sans"],
73     "font.size": 7,
74     "axes.labelsize": 7,
75     "axes.titlesize": 8,
76     "xtick.labelsize": 7,
77     "ytick.labelsize": 7,
78     "legend.fontsize": 6,
79     "lines.linewidth": 1.0,
80     "lines.markersize": 4,
81     "axes.linewidth": 0.6,
82     "axes.spines.top": False,
83     "axes.spines.right": False,
84     "pdf.fonttype": 42,
85     "ps.fonttype": 42,
86     "svg.fonttype": "none",
87 },
88 "ieee": {
89     "figure.figsize": (3.5, 2.5),
90     "figure.dpi": 150,
91     "savefig.dpi": 600,
92     "font.family": "serif",
93     "font.serif": ["Times New Roman", "Times", "DejaVu Serif"],
94     "font.size": 8,
95     "axes.labelsize": 8,
96     "axes.titlesize": 9,
97     "xtick.labelsize": 7,
98     "ytick.labelsize": 7,
99     "legend.fontsize": 7,
100    "lines.linewidth": 1.0,
101    "lines.markersize": 4,
102    "axes.linewidth": 0.7,
103    "axes.grid": False,
104    "axes.spines.top": True,
105    "axes.spines.right": True,
106    "pdf.fonttype": 42,
107    "ps.fonttype": 42,
108    "svg.fonttype": "none",
109 },
110 "general": {
111     "figure.figsize": (5.0, 3.5),
112     "figure.dpi": 150,
113     "savefig.dpi": 300,
114     "font.family": "sans-serif",
115     "font.sans-serif": ["Arial", "Helvetica", "DejaVu Sans"],
116     "font.size": 9,
117     "axes.labelsize": 10,

```

```

118     "axes.titlesize": 11,
119     "xtick.labelsize": 9,
120     "ytick.labelsize": 9,
121     "legend.fontsize": 9,
122     "lines.linewidth": 1.2,
123     "lines.markersize": 5,
124     "axes.linewidth": 0.8,
125     "axes.spines.top": False,
126     "axes.spines.right": False,
127     "pdf.fonttype": 42,
128     "ps.fonttype": 42,
129     "svg.fonttype": "none",
130 },
131 }
132
133 # 中文字体优先级列表（按可用性 + 期刊接受度排序）
134 CJK_FONT_PRIORITY = [
135     "Noto Sans CJK SC",
136     "Noto Sans SC",
137     "Source Han Sans SC",
138     "Source Han Sans CN",
139     "SimHei",
140     "Microsoft YaHei",
141     "PingFang SC",
142     "Heiti SC",
143     "WenQuanYi Zen Hei",
144     "Arial Unicode MS",
145 ]
146
147 CJK_SERIF_PRIORITY = [
148     "Noto Serif CJK SC",
149     "Noto Serif SC",
150     "Source Han Serif SC",
151     "Source Han Serif CN",
152     "SimSun",
153     "STSong",
154     "Songti SC",
155 ]
156
157 CJK_INSTALL_HINT = """\
158 找不到任何中文字体。请安装 Noto CJK 字体之一：
159
160 Linux:    sudo apt install fonts-noto-cjk    # Debian/Ubuntu
161           sudo dnf install google-noto-sans-cjk-fonts # Fedora/RHEL
162 macOS:    brew install --cask font-noto-sans-cjk-sc
163           或下载: https://github.com/notofonts/noto-cjk/releases
164 Windows:  下载 https://github.com/notofonts/noto-cjk/releases
165           解压后右键 .ttf/.otf 文件 -> "为所有用户安装"

```

```

166
167 或者列出当前已安装的中文字体：
168     python setup_style.py --list-fonts
169     """
170
171
172 def _available_fonts() -> set[str]:
173     """返回 matplotlib 已索引的全部字体名集合。"""
174     return {f.name for f in fm.fontManager.ttflist}
175
176
177 def list_cjk_fonts() -> list[str]:
178     """返回系统上可用的 CJK 字体（按优先级排序）。"""
179     available = _available_fonts()
180     hits = []
181     for f in CJK_FONT_PRIORITY + CJK_SERIF_PRIORITY:
182         if f in available and f not in hits:
183             hits.append(f)
184     # 额外做一次包含中文关键词的扫描，捕获非标准命名的中文字体
185     for f in available:
186         lower = f.lower()
187         if any(k in lower for k in ("cjk", "han", "songti", "yahei", "simhei", "simsun")):
188             if f not in hits:
189                 hits.append(f)
190     return hits
191
192
193 def configure_chinese_fonts(serif_for_zh: bool = False) -> str:
194     """
195     自动检测并配置中文字体；同时修正负号渲染。
196
197     Args:
198         serif_for_zh: True 时优先选用衬线中文字体（宋体类），用于中文期刊
199             “宋体正文 + Times New Roman 数字”的混排约定。
200     Returns:
201         实际选用的中文字体名。
202     Raises:
203         RuntimeError: 系统未安装任何识别到的中文字体。
204     """
205     available = _available_fonts()
206     priority = CJK_SERIF_PRIORITY + CJK_FONT_PRIORITY if serif_for_zh else CJK_FONT_PRIORITY
207
208     chosen = None
209     for f in priority:
210         if f in available:
211             chosen = f
212             break
213 
```

```

214     if chosen is None:
215         # 兜底：扫包含 cjk/han 等关键字的字体
216         for f in available:
217             lower = f.lower()
218             if any(k in lower for k in ("cjk", "han", "song", "hei", "yahei", "kaiti")):
219                 chosen = f
220                 break
221
222     if chosen is None:
223         raise RuntimeError(CJK_INSTALL_HINT)
224
225     # 中文期刊常要求中文 + Times New Roman 混排：中文走中文字体，西文走 Times
226     plt.rcParams["font.family"] = ["sans-serif"] if not serif_for_zh else ["serif"]
227     if serif_for_zh:
228         plt.rcParams["font.serif"] = [chosen, "Times New Roman", "Times", "DejaVu Serif"]
229     else:
230         plt.rcParams["font.sans-serif"] = [chosen, "Arial", "Helvetica", "DejaVu Sans"]
231     # 修正负号 unicode minus 在某些中文字体里渲染成方框的问题
232     plt.rcParams["axes.unicode_minus"] = False
233     return chosen
234
235
236 def _try_sciencplots(journal: str) -> bool:
237     """If SciencePlots is installed, apply its style stack; otherwise return False."""
238     try:
239         import scienceplots # noqa: F401
240     except ImportError:
241         return False
242
243     # SciencePlots 风格栈：基础 + 期刊变体
244     stack = ["science"]
245     if journal == "nature":
246         stack.append("nature")
247     elif journal == "ieee":
248         stack.append("ieee")
249     # 关掉 LaTeX 渲染避免环境缺 LaTeX 时崩溃；中文模式必须关
250     stack.append("no-latex")
251     try:
252         plt.style.use(stack)
253         return True
254     except OSError as e:
255         warnings.warn(f"SciencePlots style stack failed: {e}; fallback to builtin.")
256         return False
257
258
259 def setup_style(
260     journal: str = "general",
261     lang: str = "en",

```

```

262     use_sciplots: bool = True,
263     serif_for_zh: bool = False,
264     constrained_layout: bool = True,
265 ) -> dict:
266     """
267     应用出版级样式预设。
268
269     Args:
270         journal: 'nature' | 'science' | 'ieee' | 'general'
271         lang: 'en' | 'zh' — 中文模式自动配置中文字体并修正负号
272         use_sciplots: 优先尝试 SciencePlots; 不可用则回退到内置预设
273         serif_for_zh: 中文模式下使用宋体类衬线字体（中文期刊常约定）
274         constrained_layout: 默认 True——全局开启 constrained_layout 自适应排版，
275             从源头减少标题/轴标签被裁、图例压数据、子图互相重叠。需要手动
276             subplots_adjust 或某些 colorbar 写法时可传 False 关闭。
277     Returns:
278         dict 包含 keys: journal / lang / sciplots_used / cjk_font / constrained_layout
279     """
280     if journal not in JOURNAL_PRESETS:
281         raise ValueError(f"Unknown journal preset: {journal}. "
282                          f"Choose from {sorted(JOURNAL_PRESETS)}")
283
284     sciplots_used = False
285     if use_sciplots:
286         sciplots_used = _try_sciencplots(journal)
287
288     # 内置预设始终在 SciencePlots 之上覆盖一遍，确保关键参数（fonttype、字号）落实
289     plt.rcParams.update(JOURNAL_PRESETS[journal])
290
291     # 默认开启自适应排版：从源头减少文字遮盖 / 裁切 / 子图重叠
292     plt.rcParams["figure.constrained_layout.use"] = constrained_layout
293
294     # 全模式默认修正负号：避免所选字体缺 U+2212 时负号渲染成方框（一种乱码）。
295     # 用 ASCII hyphen-minus 代替真减号，几乎所有字体都含，最稳妥。
296     plt.rcParams["axes.unicode_minus"] = False
297
298     cjk_font = None
299     if lang == "zh":
300         cjk_font = configure_chinese_fonts(serif_for_zh=serif_for_zh)
301     elif lang != "en":
302         raise ValueError(f"lang must be 'en' or 'zh', got {lang!r}")
303
304     return {
305         "journal": journal,
306         "lang": lang,
307         "sciplots_used": sciplots_used,
308         "cjk_font": cjk_font,
309         "constrained_layout": constrained_layout,

```

```

310     }
311
312
313 def _cli() -> int:
314     p = argparse.ArgumentParser(description="scipilot-figure-skill style setup")
315     p.add_argument("--list-fonts", action="store_true",
316                   help="列出当前系统上可用的 CJK 字体")
317     p.add_argument("--test", action="store_true",
318                   help="应用预设并打印 rcParams 关键值")
319     p.add_argument("--journal", default="general",
320                   choices=list(JOURNAL_PRESETS))
321     p.add_argument("--lang", default="en", choices=["en", "zh"])
322     p.add_argument("--no-sciplots", action="store_true")
323     p.add_argument("--serif-zh", action="store_true")
324     args = p.parse_args()
325
326     if args.list_fonts:
327         fonts = list_cjk_fonts()
328         if fonts:
329             print("已检测到的 CJK 字体（按优先级排序）：")
330             for f in fonts:
331                 print(f" - {f}")
332         else:
333             print("未检测到任何 CJK 字体。")
334             print(CJK_INSTALL_HINT)
335         return 0
336
337     if args.test:
338         info = setup_style(journal=args.journal, lang=args.lang,
339                           use_sciplots=not args.no_sciplots,
340                           serif_for_zh=args.serif_zh)
341         print(f"applied: {info}")
342         for k in ("figure.figsize", "font.family", "font.size",
343                 "axes.labelsize", "pdf.fonttype", "axes.unicode_minus"):
344             print(f" {k} = {plt.rcParams[k]!r}")
345         return 0
346
347     p.print_help()
348     return 0
349
350
351 if __name__ == "__main__":
352     sys.exit(_cli())

```

```

1 """
2 scipilot-figure-skill :: export_figure.py
3 =====

```

```

4 Unified figure export to multiple formats at exact final size.
5
6 - Vector preferred: PDF / SVG / EPS for line/bar/scatter (lossless, journal-friendly).
7 - Raster for photos / micrographs: PNG / TIFF at >= 300 DPI; never JPEG for data figures.
8 - Embeds TrueType fonts (fonttype 42) so journals don't reject Type-3 PDFs.
9 - Optional grayscale preview to sanity-check colorblind safety.

```

Usage

```

12 -----

```

```

13 from export_figure import export_figure
14 import matplotlib.pyplot as plt
15
16 fig, ax = plt.subplots()
17 ax.plot([0, 1, 2], [3, 1, 4])
18
19 paths = export_figure(
20     fig,
21     basename="figs/fig1_main",
22     formats=["pdf", "svg", "png"],
23     size_inches=(3.5, 2.625), # 强制成 Nature 单栏尺寸
24     dpi=600,
25     grayscale_preview=True,
26 )
27 # -> ['figs/fig1_main.pdf', 'figs/fig1_main.svg',
28 #      'figs/fig1_main.png', 'figs/fig1_main_grayscale.png']

```

```

30 CLI: ``python export_figure.py demo`` 生成一张演示图并多格式导出。

```

```

31 """

```

```

32 from __future__ import annotations
33
34 import argparse
35 import os
36 import sys
37 from typing import Iterable
38
39 import matplotlib.pyplot as plt
40
41
42 VECTOR_FORMATS = {"pdf", "svg", "eps"}
43 RASTER_FORMATS = {"png", "tiff", "tif", "jpg", "jpeg"}
44 SUPPORTED_FORMATS = VECTOR_FORMATS | RASTER_FORMATS
45
46
47 def _ensure_parent(path: str) -> None:
48     parent = os.path.dirname(os.path.abspath(path))
49     if parent and not os.path.exists(parent):
50         os.makedirs(parent, exist_ok=True)

```

```

52
53 def export_figure(
54     fig,
55     basename: str,
56     formats: Iterable[str] | None = None,
57     dpi: int = 300,
58     size_inches: tuple[float, float] | None = None,
59     grayscale_preview: bool = False,
60     tight: bool = True,
61     pad_inches: float = 0.05,
62     transparent: bool = False,
63 ) -> list[str]:
64     """
65     Export a matplotlib Figure to one or more formats at exact final size.
66
67     Args:
68         fig: matplotlib Figure object.
69         basename: 输出路径前缀（不含扩展名）；可包含子目录，自动创建。
70         formats: list/tuple of extensions, e.g. ['pdf', 'svg', 'png'].
71             Default: ['pdf', 'svg', 'png'].
72         dpi: raster 格式分辨率；建议 300（普通）/ 600（IEEE 等）。
73         size_inches: (width, height) 英寸；指定后会 fig.set_size_inches() 强制
74             最终尺寸。强烈建议传入——保证导出后不必在 Word/LaTeX 里二次缩放。
75         grayscale_preview: 额外生成一张 _grayscale.png 供色盲安全检查。
76         tight: 是否走 bbox_inches='tight'（裁掉留白）。
77         pad_inches: tight 模式下保留的边距（英寸）。
78         transparent: 透明背景（PPT/海报可能需要）。
79
80     Returns:
81         实际写出的文件路径列表。
82     """
83     if formats is None:
84         formats = ("pdf", "svg", "png")
85     formats = [f.lower().lstrip(".") for f in formats]
86     unknown = [f for f in formats if f not in SUPPORTED_FORMATS]
87     if unknown:
88         raise ValueError(f"Unsupported formats: {unknown}. "
89                          f"Supported: {sorted(SUPPORTED_FORMATS)}")
90
91     if size_inches is not None:
92         if len(size_inches) != 2:
93             raise ValueError("size_inches must be (width, height)")
94         fig.set_size_inches(*size_inches)
95
96     # 强制嵌入 TrueType 字体 (fonttype 42)；多家期刊明确拒绝 Type-3 PDF。
97     plt.rcParams["pdf.fonttype"] = 42
98     plt.rcParams["ps.fonttype"] = 42
99     plt.rcParams["svg.fonttype"] = "none" # 文本仍可编辑，期刊通常更欢迎

```

```

100
101 saved: list[str] = []
102 for fmt in formats:
103     if fmt in {"jpg", "jpeg"}:
104         print(f"[scipilot-figure-skill] WARNING: skipping {fmt} — "
105               "JPEG is lossy and unsuitable for line/text figures.",
106               file=sys.stderr)
107         continue
108     path = f"{basename}.{fmt}"
109     _ensure_parent(path)
110     kwargs: dict = {
111         "bbox_inches": "tight" if tight else None,
112         "pad_inches": pad_inches,
113         "transparent": transparent,
114     }
115     if fmt in RASTER_FORMATS:
116         kwargs["dpi"] = dpi
117     fig.savefig(path, **kwargs)
118     saved.append(path)
119     print(f"[scipilot-figure-skill] wrote {path}")
120
121 if grayscale_preview:
122     gray_path = _grayscale_from(fig, basename, dpi=dpi)
123     if gray_path:
124         saved.append(gray_path)
125 return saved
126
127
128 def _grayscale_from(fig, basename: str, dpi: int) -> str | None:
129     """
130     导出灰度预览版用于色盲安全检查。
131     优先用 PIL 转灰度；找不到 PIL 时退化为重新画图（关闭颜色）。
132     """
133     try:
134         from PIL import Image
135     except ImportError:
136         print(f"[scipilot-figure-skill] Pillow not available; "
137               "grayscale preview skipped.", file=sys.stderr)
138         return None
139
140     png_path = f"{basename}.png"
141     _ensure_parent(png_path)
142     fig.savefig(png_path, dpi=dpi, bbox_inches="tight")
143
144     gray_path = f"{basename}_grayscale.png"
145     Image.open(png_path).convert("L").save(gray_path)
146     print(f"[scipilot-figure-skill] wrote {gray_path} (grayscale preview)")
147     return gray_path

```

```

148
149
150 def _demo(out_basename: str) -> None:
151     """Generate a small demo figure for quick smoke testing."""
152     import numpy as np
153     rng = np.random.default_rng(7)
154     x = np.linspace(0, 10, 50)
155     y1 = np.sin(x) + rng.normal(0, 0.1, x.size)
156     y2 = np.cos(x) + rng.normal(0, 0.1, x.size)
157
158     fig, ax = plt.subplots(figsize=(3.5, 2.625))
159     ax.plot(x, y1, label="sin", marker="o", markersize=3)
160     ax.plot(x, y2, label="cos", marker="s", markersize=3, linestyle="--")
161     ax.set_xlabel("x")
162     ax.set_ylabel("y")
163     ax.legend(frameon=False)
164
165     paths = export_figure(
166         fig, out_basename,
167         formats=["pdf", "svg", "png"],
168         size_inches=(3.5, 2.625),
169         dpi=300,
170         grayscale_preview=True,
171     )
172     print("\nDemo done. Files:")
173     for p in paths:
174         print(f"  {p}")
175
176
177 def _cli() -> int:
178     p = argparse.ArgumentParser(description="scipilot-figure-skill figure exporter")
179     p.add_argument("cmd", choices=["demo"], help="demo: 跑一张演示图导出 4 种格式")
180     p.add_argument("--out", default="./scipilot_demo",
181                    help="输出 basename (默认 ./scipilot_demo)")
182     args = p.parse_args()
183     if args.cmd == "demo":
184         _demo(args.out)
185     return 0
186
187
188 if __name__ == "__main__":
189     sys.exit(_cli())

```

```

1 """
2 scipilot-figure-skill :: visual_qa.py
3 =====
4 出图后的「程序自检」+「渲染预览」——自检闭环的机器那一层。

```

```

5
6 设计分工：
7 - **程序（本脚本）** 抓**确定性**问题：缺字乱码、文字越界裁切、刻度标签重叠。
8 - **AI 读图**（见 references/visual_review.md）抓**感知性**问题：图例压数据、
9   子图标签是否对齐、配色灰度可分、整体观感。
10
11 两层串起来才是完整的「出图 → 渲 PNG → 程序自检 + AI 读图 → 回改 → 再看」闭环。
12
13 核心能力
14 -----
15 - ``render_preview(fig_or_path, out_png, dpi)`` —— 渲一张中分辨率 PNG，
16   供 AI 用 Read 工具读图复核（矢量 PDF 没法直接“看像素重叠”，必须先栅格化）。
17 - ``audit_layout(fig)`` —— 返回 ``[(severity, msg), ...]``：
18   * **缺字**（FAIL）：渲染时同时拦截 matplotlib 的 warnings 与 logging 两条
19     告警通道，任一报“missing from font”即判定成图会出方框/乱码。
20   * **文字越界裁切**（WARN）：Text 的 window_extent 超出画布边界。
21   * **刻度标签重叠**（WARN）：相邻 tick label 的包围盒水平/垂直相交。
22
23 severity 约定与 check_figure.py 保持一致：INFO < WARN < FAIL。
24
25 Usage
26 -----
27     from visual_qa import render_preview, audit_layout, print_report
28
29     fig, ax = plt.subplots()
30     ...
31     png = render_preview(fig, "figs/_preview.png", dpi=150) # 给 AI 读图
32     issues = audit_layout(fig)
33     print_report(issues)
34
35 CLI:
36     python visual_qa.py demo                    # 跑一遍自检演示
37     python visual_qa.py figs/fig1.png --preview out.png
38     """
39 from __future__ import annotations
40
41 import argparse
42 import io
43 import logging
44 import os
45 import sys
46 import warnings
47
48 import matplotlib.pyplot as plt
49 import matplotlib.text as mtext
50
51 # Windows GBK 终端下 print 中文会 UnicodeEncodeError。用 reconfigure 而非替换

```

```

53 # sys.stdout: 幂等、不创建新对象，多个脚本一起 import 时也不会关闭底层流。
54 if sys.platform == "win32":
55     try:
56         sys.stdout.reconfigure(encoding="utf-8")
57     except (AttributeError, ValueError):
58         pass
59
60 SEVERITY = {"INFO": 0, "WARN": 1, "FAIL": 2}
61 _GLYPH_MARKERS = ("missing from", "Glyph", "findfont")
62
63
64 def _ensure_parent(path: str) -> None:
65     parent = os.path.dirname(os.path.abspath(path))
66     if parent and not os.path.exists(parent):
67         os.makedirs(parent, exist_ok=True)
68
69
70 class _GlyphLogHandler(logging.Handler):
71     """拦截 matplotlib logger 里关于缺字 / 找不到字体的记录。"""
72
73     def __init__(self):
74         super().__init__()
75         self.messages: list[str] = []
76
77     def emit(self, record):
78         msg = record.getMessage()
79         if any(m in msg for m in _GLYPH_MARKERS):
80             self.messages.append(msg)
81
82
83 def _draw_and_collect_glyph_warnings(fig) -> list[str]:
84     """
85     渲染一次 figure，同时从 warnings 和 logging 两条通道收集缺字告警。
86
87     matplotlib 不同版本对缺字的上报方式不一：老版本走 warnings.warn，
88     新版本走 logging。两边都挂上才不会漏。渲染顺便让 renderer 就绪，
89     供后续 window_extent 测量使用。
90     """
91     handler = _GlyphLogHandler()
92     mpl_logger = logging.getLogger("matplotlib")
93     prev_level = mpl_logger.level
94     mpl_logger.setLevel(logging.WARNING)
95     mpl_logger.addHandler(handler)
96
97     collected: list[str] = []
98     try:
99         with warnings.catch_warnings(record=True) as wlist:
100             warnings.simplefilter("always")

```

```

101         # 画到内存即可，不落盘；draw 也会触发缺字告警
102         buf = io.BytesIO()
103         fig.savefig(buf, format="png", dpi=100)
104         buf.close()
105         for w in wlist:
106             s = str(w.message)
107             if any(m in s for m in _GLYPH_MARKERS):
108                 collected.append(s)
109         finally:
110             mpl_logger.removeHandler(handler)
111             mpl_logger.setLevel(prev_level)
112
113         collected.extend(handler.messages)
114         # 去重并保持顺序
115         seen, uniq = set(), []
116         for m in collected:
117             if m not in seen:
118                 seen.add(m)
119                 uniq.append(m)
120         return uniq
121
122
123 def _visible_texts(fig) -> list:
124     out = []
125     for t in fig.findobj(mtext.Text):
126         try:
127             if t.get_visible() and t.get_text().strip():
128                 out.append(t)
129         except Exception:
130             continue
131     return out
132
133
134 def audit_layout(fig, clip_tol_px: float = 2.0, overlap_tol_px: float = 1.0
135                 ) -> list[tuple[str, str]]:
136     """
137     对一张 matplotlib Figure 做版面自检。返回 [(severity, msg), ...]。
138
139     检测项:
140         1. 缺字乱码 (FAIL) —— 中文/特殊符号字体未命中。
141         2. 文字越界裁切 (WARN) —— 标题/轴标签/标注超出画布。
142         3. 刻度标签重叠 (WARN) —— x/y 轴相邻刻度包围盒相交。
143
144     非破坏性: 只渲染测量, 不修改 fig 内容。
145     """
146     issues: list[tuple[str, str]] = []
147
148     # ---- 1. 缺字 (同时触发一次渲染, 让 renderer 就绪) ----

```

```

149 glyph_msgs = _draw_and_collect_glyph_warnings(fig)
150 if glyph_msgs:
151     sample = " | ".join(glyph_msgs[:3])
152     issues.append((
153         "FAIL",
154         f"检测到缺字，成图会出现方框/乱码：{sample[:240]}。"
155         "中文图请先 setup_style(lang='zh') 配置 CJK 字体；"
156         "若是负号方框，确认 axes.unicode_minus=False。"
157     ))
158
159 try:
160     renderer = fig.canvas.get_renderer()
161 except Exception:
162     fig.canvas.draw()
163     renderer = fig.canvas.get_renderer()
164
165 W = float(fig.bbox.width)
166 H = float(fig.bbox.height)
167
168 # ---- 2. 文字越界裁切 ----
169 # 刻度标签的越界由 constrained_layout / bbox_inches='tight' 自动处理，且另有
170 # 刻度重叠检测覆盖——这里跳过它们，只盯 title / 轴标签 / 标注这类用户文字，
171 # 避免 constrained_layout 下刻度贴边造成的误报。
172 tick_ids = set()
173 for ax in fig.axes:
174     for tl in (*ax.get_xticklabels(), *ax.get_xticklabels(minor=True),
175              *ax.get_yticklabels(), *ax.get_yticklabels(minor=True)):
176         tick_ids.add(id(tl))
177
178 clipped: list[str] = []
179 for t in _visible_texts(fig):
180     if id(t) in tick_ids:
181         continue
182     try:
183         bb = t.get_window_extent(renderer)
184     except Exception:
185         continue
186     if (bb.x0 < -clip_tol_px or bb.y0 < -clip_tol_px
187         or bb.x1 > W + clip_tol_px or bb.y1 > H + clip_tol_px):
188         txt = t.get_text().strip().replace("\n", " ")
189         if txt:
190             clipped.append(txt[:24])
191 if clipped:
192     uniq = list(dict.fromkeys(clipped))[:6]
193     issues.append((
194         "WARN",
195         f"以下文字可能超出画布被裁切：{uniq}。"
196         "跑 finalize_figure(fig) 或导出时 bbox_inches='tight' 兜底；"

```

```

197         "标题/标签过长可换行或缩短。"
198     ))
199
200     # ---- 3. 刻度标签重叠 ----
201     overlap_axes = 0
202     for ax in fig.axes:
203         if ax.get_subplotspec() is None:
204             continue
205         if _ticklabels_overlap(ax.get_xticklabels(), renderer,
206                                axis="x", tol=overlap_tol_px):
207             overlap_axes += 1
208             continue
209         if _ticklabels_overlap(ax.get_yticklabels(), renderer,
210                                axis="y", tol=overlap_tol_px):
211             overlap_axes += 1
212     if overlap_axes:
213         issues.append((
214             "WARN",
215             f"{overlap_axes} 个子图存在刻度标签重叠。"
216             "x 轴: ax.tick_params(axis='x', rotation=30) 或减少刻度/缩短标签; "
217             "y 轴: 增大子图高度或减少刻度数。"
218         ))
219
220     return issues
221
222
223 def _ticklabels_overlap(labels, renderer, axis: str, tol: float) -> bool:
224     """相邻刻度标签包围盒是否相交。axis='x' 看水平、'y' 看垂直。"""
225     boxes = []
226     for l in labels:
227         try:
228             if l.get_visible() and l.get_text().strip():
229                 boxes.append(l.get_window_extent(renderer))
230         except Exception:
231             continue
232     if len(boxes) < 2:
233         return False
234     if axis == "x":
235         boxes.sort(key=lambda b: b.x0)
236         return any(a.x1 - b.x0 > tol for a, b in zip(boxes, boxes[1:]))
237     else:
238         boxes.sort(key=lambda b: b.y0)
239         return any(a.y1 - b.y0 > tol for a, b in zip(boxes, boxes[1:]))
240
241
242 def render_preview(fig_or_path, out_png: str = "_preview.png",
243                    dpi: int = 150) -> str:
244     """

```

渲染一张 PNG 预览供 AI 读图。

Args:

fig_or_path: matplotlib Figure 对象（自检闭环主路径），或一个已落盘的图片路径（位图直接返回；PDF 需可选的 PyMuPDF）。

out_png: 输出 PNG 路径。

dpi: 预览分辨率，默认 150（够 AI 看清文字/重叠，又不至于太大）。

Returns:

可供 Read 读取的 PNG 路径（位图来源时返回原路径）。

"""

```
if hasattr(fig_or_path, "savefig"):
```

```
    _ensure_parent(out_png)
```

```
    fig_or_path.savefig(out_png, dpi=dpi, bbox_inches="tight")
```

```
    return out_png
```

```
path = str(fig_or_path)
```

```
if not os.path.exists(path):
```

```
    raise FileNotFoundError(path)
```

```
ext = os.path.splitext(path)[1].lower().lstrip(".")
```

```
if ext in {"png", "tif", "tiff", "jpg", "jpeg", "bmp"}:
```

```
    return path
```

```
if ext == "pdf":
```

```
    try:
```

```
        import fitz # PyMuPDF, 可选
```

```
    except ImportError as e:
```

```
        raise RuntimeError(
```

```
            "把 PDF 渲染预览需要 PyMuPDF (pip install pymupdf)。"
```

```
            "自检闭环更推荐直接把 matplotlib Figure 对象传给 render_preview, "
```

```
            "在导出前就完成读图复核。"
```

```
        ) from e
```

```
    doc = fitz.open(path)
```

```
    pix = doc[0].get_pixmap(dpi=dpi)
```

```
    _ensure_parent(out_png)
```

```
    pix.save(out_png)
```

```
    doc.close()
```

```
    return out_png
```

```
raise RuntimeError(f"不支持从 {ext} 生成预览；请传 Figure 对象或位图。")
```

```
def print_report(issues: list[tuple[str, str]]) -> str:
```

```
    """打印 audit_layout 的结果，返回 verdict (PASS/WARN/FAIL)。"""
```

```
    if not issues:
```

```
        print(" [PASS] 程序自检未发现缺字 / 裁切 / 刻度重叠。")
```

```
        print(" >>> 仍需 AI 读图复核感知性问题（见 visual_review.md）。")
```

```
        return "PASS"
```

```
    max_sev = max(SEVERITY[s] for s, _ in issues)
```

```
    verdict = {2: "FAIL", 1: "WARN", 0: "INFO"}[max_sev]
```

```

293     for sev, msg in sorted(issues, key=lambda x: -SEVERITY[x[0]]):
294         print(f"    [{sev}] {msg}")
295     print(f"    >>> verdict: {verdict} (修完再渲一次 PNG 让 AI 读图复核)")
296     return verdict
297
298
299 def _demo() -> int:
300     """构造一张有版面问题的图，演示 audit_layout 能抓出来。"""
301     import numpy as np
302     rng = np.random.default_rng(1)
303
304     fig, ax = plt.subplots(figsize=(3.0, 2.2))
305     cats = [f"very_long_condition_name_{i}" for i in range(12)]
306     ax.bar(range(12), rng.random(12))
307     ax.set_xticks(range(12))
308     ax.set_xticklabels(cats) # 故意不旋转 -> x 轴刻度必然重叠
309     ax.set_title("An intentionally overlong title that runs off the canvas edge")
310     ax.set_ylabel("value")
311
312     print("=== visual_qa demo: 对一张刻意做坏版面的图自检 ===")
313     issues = audit_layout(fig)
314     print_report(issues)
315     out = render_preview(fig, "./visual_qa_demo.png", dpi=120)
316     print(f"\n预览已写出: {out} (可用 Read 工具读图复核)")
317     print("注: 缺字检测只在中文未配字体等情况下才会 FAIL; 本 demo 主要展示裁切+重叠。")
318     return 0
319
320
321 def _cli() -> int:
322     p = argparse.ArgumentParser(description="scipilot-figure-skill visual QA")
323     p.add_argument("target", nargs="?", help="图片路径; 或 'demo'")
324     p.add_argument("--preview", metavar="OUT.png",
325                    help="把 target 渲成 PNG 预览到此路径")
326     p.add_argument("--dpi", type=int, default=150)
327     args = p.parse_args()
328
329     if args.target == "demo" or args.target is None:
330         return _demo()
331
332     if args.preview:
333         out = render_preview(args.target, args.preview, dpi=args.dpi)
334         print(f"[visual_qa] 预览: {out}")
335     else:
336         # 对已落盘文件, audit_layout 需要 Figure 对象, 这里只能提示
337         print("[visual_qa] 对已落盘图片只支持 --preview 渲图; "
338               "版面自检 audit_layout 请在画图脚本里对 Figure 对象调用。")
339     return 0
340

```

```

341
342 if __name__ == "__main__":
343     sys.exit(_cli())

1 #!/usr/bin/env python3
2 """生成可复现运行清单，并隔离 SKILL_ROOT 与 PROJECT_ROOT。"""
3
4 import argparse
5 import hashlib
6 import importlib.metadata
7 import json
8 import platform
9 import sys
10 from datetime import datetime, timezone
11 from pathlib import Path
12
13
14 SKILL_ROOT = Path(__file__).resolve().parents[4]
15
16
17 def sha256_file(path: Path) -> str:
18     digest = hashlib.sha256()
19     with path.open("rb") as stream:
20         for chunk in iter(lambda: stream.read(1024 * 1024), b''):
21             digest.update(chunk)
22     return digest.hexdigest()
23
24
25 def build_manifest(
26     inputs,
27     seed,
28     parameters,
29     command,
30     packages,
31     runtime_name="python",
32     runtime_version=None,
33     dependencies=None,
34 ):
35     files = []
36     for item in inputs:
37         path = Path(item).resolve()
38         if not path.is_file():
39             raise FileNotFoundError(f"输入文件不存在: {path}")
40         files.append({"path": str(path), "sha256": sha256_file(path), "bytes": path.stat().st_size})
41
42     versions = dict(dependencies or {})
43     if runtime_name == "python":

```

```

44     for package in packages:
45         try:
46             versions[package] = importlib.metadata.version(package)
47         except importlib.metadata.PackageNotFoundError:
48             versions[package] = None
49     runtime_version = runtime_version or sys.version.split()[0]
50
51     return {
52         "schema_version": 1,
53         "created_at_utc": datetime.now(timezone.utc).isoformat(),
54         "random_seed": seed,
55         "input_files": files,
56         "runtime": {
57             "name": runtime_name,
58             "version": runtime_version,
59             "platform": platform.platform(),
60             "dependencies": versions,
61         },
62         "key_parameters": parameters,
63         "reproduce_command": command,
64     }
65
66
67 def _is_within(path: Path, parent: Path) -> bool:
68     try:
69         path.relative_to(parent)
70         return True
71     except ValueError:
72         return False
73
74
75 def resolve_output(project_root, relative_output):
76     project = Path(project_root).resolve()
77     if _is_within(project, SKILL_ROOT):
78         raise ValueError("PROJECT_ROOT 不能位于 SKILL_ROOT 内部")
79     output = (project / relative_output).resolve()
80     if not _is_within(output, project):
81         raise ValueError("输出路径必须位于 PROJECT_ROOT 内部")
82     if _is_within(output, SKILL_ROOT):
83         raise ValueError("输出路径不能位于 SKILL_ROOT 内部")
84     return output
85
86
87 def main() -> int:
88     parser = argparse.ArgumentParser(description="生成数学建模复现清单")
89     parser.add_argument("--project-root", required=True)
90     parser.add_argument("--input", action="append", default=[])
91     parser.add_argument("--seed", required=True, type=int)

```

```

92 parser.add_argument("--parameters", default="{}", help="JSON 对象")
93 parser.add_argument("--command", required=True)
94 parser.add_argument("--package", action="append", default=[])
95 parser.add_argument("--runtime", choices=["python", "matlab"], default="python")
96 parser.add_argument("--runtime-version")
97 parser.add_argument("--dependencies", default="{}", help="依赖或 MATLAB 工具箱版本 JSON 对象")
98 parser.add_argument("--output", default="results/复现清单.json")
99 parser.add_argument("--overwrite", action="store_true")
100 args = parser.parse_args()
101
102 output = resolve_output(args.project_root, args.output)
103 if output.exists() and not args.overwrite:
104     print(f"错误: 输出已存在, 未覆盖: {output}", file=sys.stderr)
105     return 1
106 parameters = json.loads(args.parameters)
107 if not isinstance(parameters, dict):
108     raise ValueError("--parameters 必须是 JSON 对象")
109 dependencies = json.loads(args.dependencies)
110 if not isinstance(dependencies, dict):
111     raise ValueError("--dependencies 必须是 JSON 对象")
112 manifest = build_manifest(
113     args.input,
114     args.seed,
115     parameters,
116     args.command,
117     args.package,
118     runtime_name=args.runtime,
119     runtime_version=args.runtime_version,
120     dependencies=dependencies,
121 )
122 output.parent.mkdir(parents=True, exist_ok=True)
123 output.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
124 print(output)
125 return 0
126
127
128 if __name__ == "__main__":
129     raise SystemExit(main())

```

C 人工核验记录

参赛队提交前应按 README.txt 所列顺序重新运行冒烟测试与必要的全量计算，核对四个 result*.xlsx、关键结果摘要、正文数值和图件数据源的一致性；同时逐项审核 AI 工具使用详情.pdf 中的工具、时间、用途、提示方式与使用过程。支撑材料和论文一经生成最终校验值，不再修改。